Strapi 5 od środka

Praktyczny przewodnik dewelopera: od cyklu żądania po produkcję

Marcin & Claude · wydanie robocze · 14 rozdz.

Spis treści

Wstęp 5
- Dla kogo 5
- Jak ułożona jest książka 5
- Projekt przewodni i kod 5
- Wersja 5

Rozdział 1. Architektura i droga żądania 7
- Co jest pod spodem 7
- Struktura projektu 7
- Droga żądania 8
- Content API i Admin API 9
- register, bootstrap, destroy 9
- Konfiguracja 10
- Obiekt strapi i UID-y 10
- CLI do introspekcji 10
- Gdzie umieścić logikę 11
- Podsumowanie 11
- Co dalej 11
- Źródła 12

Rozdział 2. Modelowanie treści 13
- Schemat od środka 13
- Komponent, relacja czy dynamic zone 14
- Relacje i użytkownik 14
- Dokument i jego wersje 15
- Panel w developmentcie, pliki w produkcji 15
- Zmiana schematu a dane 16
- Podsumowanie 16
- Co dalej 17
- Źródła 17

Rozdział 3. Content API od strony klienta: REST, GraphQL, klient 18
- Kształt odpowiedzi i documentId 18
- populate: nic nie przychodzi samo 18
- fields, filters i sort 19
- Paginacja i limity z config/api.ts 19
- status: szkice i wersje opublikowane 20
- Zapis relacji: connect, disconnect, set 20
- Klient: fetch z qs albo @strapi/client 21
- Specyfikacja OpenAPI 21
- GraphQL 22
- Podsumowanie 23
- Co dalej 23
- Źródła 24

Rozdział 4. Trasy, polityki i middleware 25
- Co dokładnie dzieje się po dopasowaniu trasy 25
- Trasy 25
- Polityki 27
- Middleware 28
- is-owner: polityka czy middleware? 30
- Kolejność wykonania na logach 31
- Polityka, middleware czy kontroler 31

Podsumowanie 32
Co dalej 32
Źródła 32

Rozdział 5. Kontrolery, serwisy, walidacja i błędy 34
Co robi rdzeń 34
Rozszerzanie z super 34
Sanityzacja: wyciek, który widać w curl-u 35
Cienki kontroler, logika w serwisie 36
Kontroler nie ufa ciału żądania 36
Akcja niestandardowa: zatwierdzanie komentarza 37
ctx w kontrolerze 38
Błędy i ich format 38
Walidacja wejścia: schemat, yup czy zod 39
Podsumowanie 39
Co dalej 39
Źródła 39

Rozdział 6. Document Service, Query Engine i reagowanie na zmiany 40
Document Service: pracujesz na dokumentach, nie na wierszach 40
Query Engine: piętro niżej i czego tam nie ma 41
Transakcje: wszystko albo nic 41
Lifecycle hooks czy middleware Document Service 42
Trzy reguły Redakcji jako middleware 43
Pułapki: rekurencja i transakcja, w której siedzi hook 45
Podsumowanie 46
Co dalej 46
Źródła 46

Rozdział 7. Użytkownicy, tokeny i uprawnienia 47
Dwa światy: panel i aplikacja 47
Logowanie w trybie sesji 47
Uprawnienia w kodzie 49
Rozszerzanie users-permissions 49
Tylko moje artykuły 50
Szkice tylko dla uprawnionych 50
API tokeny 51
Logowanie przez GitHub 51
Podsumowanie 51
Co dalej 52
Źródła 52

Rozdział 8. Draft & publish, i18n, media, e-mail, webhooki, cron 53
Draft & publish od strony klienta 53
i18n: tłumaczenia na poziomie pól 53
Media Library: najpierw plik, potem powiązanie 54
E-mail: powiadomienie o nowym komentarzu 55
Webhooki: powiadomienie frontendu o publikacji 56
Cron: komentarze, na które nikt nie odpowiedział 57
Preview: przycisk podglądu w panelu 58
Podsumowanie 58
Co dalej 59
Źródła 59

Rozdział 9. Własny plugin 60
Plugin czy kod w src/api 60
Szkielet z Plugin SDK 60

Strona serwerowa 61
Strona admina 63
Rozszerzanie wbudowanego pluginu: src/extensions 64
Narzędzie MCP z pluginu 65
Publikacja na npm 65
Podsumowanie 65
Co dalej 66
Źródła 66

Rozdział 10. Panel admina i custom fields 67
Konfiguracja w src/admin/app.tsx 67
Akcja „Otwórz na stronie": API Content Managera 68
Widget na stronie głównej 69
Custom field: tytuł SEO z licznikiem 69
Własny edytor treści 71
Panel pod innym adresem 71
Podsumowanie 71
Co dalej 71
Źródła 71

Rozdział 11. Integracja z frontendem (Next.js / Nuxt) 72
Serwer z tokenem, przeglądarka bez 72
Rewalidacja po publikacji 72
Podgląd szkiców 73
Logowanie przez route handler 74
CORS 74
Obrazy 75
Podsumowanie 75
Co dalej 76
Źródła 76

Rozdział 12. TypeScript, testy i narzędzia AI 77
Typy generowane ze schematów 77
Polityka i middleware z typami 77
Testy na prawdziwej instancji 78
Narzędzia AI dla dewelopera 79
Podsumowanie 80
Co dalej 80
Źródła 80

Rozdział 13. Produkcja self-hosted: Docker, Postgres, dane, aktualizacje 81
Konfiguracja produkcyjna 81
Obraz Dockera 82
Compose z Postgresem 83
Za Traefikiem 83
Dane: eksport, import, kopie 84
Zmiany schematu i migracje 85
Aktualizacje 85
Wydajność w skrócie 86
Checklista bezpieczeństwa 86
Podsumowanie 86
Co dalej 86
Źródła 86

Wstęp

Strapi da się nauczyć z panelu. Klikasz typ treści w Content-Type Builderze, zaznaczasz uprawnienia roli Public, frontend dostaje JSON i aplikacja działa. Tak powstaje większość projektów i tak powstały też te, z których wyrosła ta książka. Problem zaczyna się w dniu, w którym trzeba dopisać logikę: autor ma edytować tylko swoje artykuły, komentarz ma czekać na moderację, frontend ma odświeżyć cache po publikacji. Wtedy okazuje się, że Strapi ma pod spodem kilka warstw (trasy, polityki, middleware, kontrolery, serwisy, Document Service, lifecycle hooks), a każda z nich przyjmie twój kod, tylko nie każda jest dobrym miejscem.

Ta książka jest mapą tych warstw. Dokumentacja Strapi 5 ma około siedmiuset stron i opisuje każdą opcję z osobna. Tutaj wybieramy to, czego naprawdę używa się przy budowie aplikacji, układamy w kolejności, w jakiej przechodzi żądanie, i przy każdym mechanizmie odpowiadamy na pytanie, które dokumentacja zostawia czytelnikowi: kiedy tego użyć, a kiedy czegoś innego.

Dla kogo

Dla programisty, który ma już działające aplikacje na Strapi 5, zna panel i REST, a chce wiedzieć, co dzieje się pod spodem. Zakładamy TypeScript, Node.js, podstawy Koa albo Expressa i self-hosting na Dockerze. Nie zakładamy, że wiesz, czym polityka różni się od middleware – od tego pytania ta książka się zaczęła.

Płatnych funkcji (Strapi Cloud, plany Growth i Enterprise: Releases, Review Workflows, Content History, SSO, Audit Logs) nie opisujemy. Wszystko, co jest w książce, działa w darmowej wersji self-hosted.

Jak ułożona jest książka

Rozdział 1 to mapa: struktura projektu, droga żądania od pierwszego middleware do bazy i tabela „gdzie umieścić logikę”. Rozdziały 2 i 3 dotyczą danych z zewnątrz: jak modelować treść i jak klient o nią pyta przez REST i GraphQL. Rozdziały 4–6 to rdzeń książki – trasy, polityki i middleware, potem kontrolery i serwisy, potem Document Service i reagowanie na zmiany. Jeśli masz czas tylko na trzy rozdziały, przeczytaj te.

Rozdziały 7 i 8 opisują to, co Strapi daje gotowe: użytkowników i tokeny, draft & publish, i18n, media, e-mail, webhooki i cron. Rozdziały 9 i 10 to rozszerzanie samego Strapi: własny plugin i panel admina. Rozdział 11 łączy backend z frontendem w Next.js, rozdział 12 dokłada typy, testy i narzędzia AI, a rozdział 13 prowadzi przez produkcję na Dockerze z Postgresem.

Projekt przewodni i kod

Wszystkie rozdziały pracują na jednym projekcie: „Redakcja”, magazyn technologiczny pisany przez wielu autorów. Ma artykuły z draft & publish i dynamic zone, kategorie, komentarze z moderacją i ustawienia strony. Fundament leży w katalogu `code/` (Strapi 5.56, TypeScript, SQLite), a `npm run seed` wypełnia go danymi: trzema kategoriami, czterema artykułami i dwoma autorami, `anna` i `piotr`.

Fundament jest celowo niezabezpieczony – każdy zalogowany autor może edytować cudze artykuły, a komentarz można dodać od razu zatwierdzony. Kolejne rozdziały zamykają te luki i pokazują przy tym, jak wyglądają one w curl-u, zanim zostaną zamknięte. Kod każdego rozdziału jest w `examples/NN-nazwa/` z krótkim README: co skopiować do fundamentu i jak to sprawdzić.

Każdy wynik w książce pochodzi z uruchomionego projektu: odpowiedzi curl, logi, wyniki testów. Tam, gdzie lokalnie nie dało się czegoś uruchomić (prawdziwy S3, pełny OAuth z GitHubem, wysyłka przez zewnętrzny SMTP), blok jest oznaczony jako „Przykładowy wynik”. Ramki „Pułapka” zbierają miejsca, w których zachowanie Strapi 5.56 różni się od tego, co sugeruje dokumentacja, albo od tego, czego intuicyjnie się spodziewasz. Jest ich sporo i są najcenniejszą częścią tej książki.

Wersja

Książka opisuje Strapi 5.56.0 (październik 2026). Strapi wydaje wersje minor co tydzień, więc szczegóły będą się zmieniać – wartości domyślne, nazwy eksportów, drobne zachowania. Architektura (droga żądania, rola Document Service, podział na Content API i Admin API) jest stabilna od 5.0. Gdy coś nie zgadza się z książką, sprawdź najpierw wersję, a potem stronę

dokumentacji w wersji Markdown: każdy adres `docs.strapi.io` z dopisanym `.md` zwraca czysty tekst, a `https://docs.strapi.io/llms.txt` to indeks całości – rozdział 12 pokazuje, jak podłączyć to do asystenta AI.

Rozdział 1. Architektura i droga żądania

> **Cel rozdziału:** zrozumieć, przez jakie warstwy przechodzi żądanie w Strapi 5, i wiedzieć, w której z nich umieścić daną logikę.
> **Kod:** `examples/01-architektura/`. **Pakiety poza fundamentem:** brak.

Redakcja ma już typy treści, uprawnienia i dane z seeda. Za chwilę pojawią się prawdziwe wymagania: autor może edytować tylko swoje artykuły, komentarze czekają na moderację, slug i czas czytania mają się liczyć same, a frontend ma odświeżać cache po publikacji. Każde z nich da się napisać w Strapi na co najmniej trzy sposoby – w polityce, w middleware albo w kontrolerze – i każdy z tych sposobów zachowa się inaczej, gdy redaktorka zmieni artykuł w panelu admina zamiast przez API.

Ten rozdział jest mapą reszty książki: z czego Strapi składa się pod spodem, jak wygląda projekt na dysku i przez jakie warstwy przechodzi żądanie, zanim dotknie bazy – narysowane i sprawdzone w logach uruchomionego projektu. Na koniec dostaniesz tabelę „gdzie umieścić logikę", do której wrócisz przy rozdziałach 4–6.

Co jest pod spodem

Back-end Strapi to serwer HTTP oparty na Koa. Jeśli znasz Express, Koa jest jego młodszym kuzynem: każdy middleware dostaje obiekt kontekstu `ctx` (żądanie, odpowiedź i miejsce na stan) oraz funkcję `next()`, a łańcuch middleware działa jak cebula. Kod przed `await next()` wykonuje się w drodze „do środka", kod po `await next()` – w drodze powrotnej, gdy odpowiedź jest już gotowa. Ta jedna zasada tłumaczy później, dlaczego middleware może zmierzyć czas żądania albo dopisać nagłówek, a polityka już nie.

Na Koa Strapi nakłada własne warstwy. Router zamienia pliki tras na endpointy. Rejestry trzymają kontrolery, serwisy, polityki i middleware pod identyfikatorami (UID). Document Service obsługuje dokumenty, czyli treść z wersjami roboczą i opublikowaną oraz z lokalizacjami. Pod nim leży Query Engine, który przez Knex rozmawia z bazą. Ten sam proces serwuje też panel admina – aplikację React, która komunikuje się z serwerem przez własne, administracyjne API.

Projekt w TypeScripcie nie jest wykonywany wprost ze źródeł: `strapi develop` kompiluje `config/` i `src/` do `dist/` i stamtąd ładuje aplikację, łącznie z konfiguracją. Dlatego błąd w dowolnym pliku `src/` zatrzymuje cały serwer, a nie jeden endpoint.

Struktura projektu

Strapi ładuje kod według konwencji katalogów, a nie przez jawne importy. Plik położony we właściwym miejscu zostaje zarejestrowany pod przewidywalnym UID-em. Projekt Redakcji wygląda tak (część katalogów pojawi się dopiero w kolejnych rozdziałach; pozostałe API powtarzają układ `article`):

```
config/            admin, api, database, middlewares, plugins, server (*.ts)
config/env/<env>/  nadpisania konfiguracji dla NODE_ENV=<env>
src/index.ts       register / bootstrap / destroy
src/api/article/
  content-types/article/schema.json   → api::article.article
  controllers/ services/ routes/      → kontroler, serwis, trasy tego API
  policies/ middlewares/              → api::article.<nazwa>
src/components/shared/*.json          → shared.seo, shared.quote ...
src/middlewares/  src/policies/       → global::<nazwa>
src/extensions/<plugin>/              → rozszerzenia wbudowanych pluginów
src/plugins/<plugin>/                 → lokalne pluginy
types/generated/                      → typy wygenerowane ze schematów
```

Katalogi `policies/` i `middlewares/` tworzysz sam, gdy są potrzebne. Typy w `types/generated/` odświeżają się przy każdym starcie `strapi develop` (w logu widać krok „Generating types"), więc nie edytujesz ich ręcznie; szczegóły w rozdziale 12. Rozszerzanie pluginów i własne pluginy to rozdział 9, panel admina – rozdział 10.

Droga żądania

Zanim kontroler artykułów zobaczy żądanie, przechodzi ono przez kilka bramek. Kolejność poniżej nie pochodzi z dokumentacji, tylko z kodu, który składa endpoint (`@strapi/core/dist/services/server/compose-endpoint.js`), i z logów pokazanych dalej:

```
żądanie HTTP
  → globalne middleware (config/middlewares.ts, po kolei)
    → router: dopasowanie trasy, ctx.state.route
      → authenticate: JWT użytkownika albo API token
      → authorize: czy rola ma uprawnienie do akcji (inaczej 401/403)
      → polityki trasy (true/false)
      → middleware trasy
      → kontroler → serwis
        → Document Service (+ jego middleware) → Query Engine → baza
  ← odpowiedź wraca przez middleware trasy i globalne (kod po await next())
```

Globalne middleware obsługują każde żądanie: do API, do panelu, do plików w `public/`. Uwierzytelnienie i sprawdzenie uprawnień z ról users-permissions dzieje się, zanim ruszy jakikolwiek twój kod związany z trasą – dlatego żądanie bez uprawnienia kończy się 403 i nie dociera nawet do polityki. Polityki tylko przepuszczają albo blokują, middleware trasy może zmieniać żądanie i odpowiedź, a kontroler z serwisem wykonują właściwą pracę przez Document Service.

> **Pułapka:** dokumentacja („Back-end customization", „Routes") opisuje polityki i middleware trasy w jednym kroku i nie mówi, co działa pierwsze, a intuicja z Koa podpowiada, że middleware owija wszystko. W Strapi 5.56 (październik 2026) najpierw wykonują się polityki, potem middleware trasy. Jeśli middleware trasy ma przygotować coś dla polityki (np. wczytać dokument do `ctx.state`), polityka tego nie zobaczy.

Śledzenie żądania na żywo

Najprościej przekonać się o tej kolejności, dokładając do każdej warstwy jedną linię logu. Zacznij od globalnego middleware w `src/middlewares/request-trace.ts`. Plik w tym katalogu jest rejestrowany jako `global::request-trace`, ale działa dopiero po dopisaniu go do `config/middlewares.ts`.

```ts
const requestTrace: Core.MiddlewareFactory<RequestTraceConfig> = (config, { strapi }) => {
  const slowMs = config.slowMs ?? 500;

  return async (ctx, next) => {
    const startedAt = Date.now();
    strapi.log.info(`[trace] → ${ctx.method} ${ctx.url}`);

    await next();

    // The router sets ctx.state.route, so it is only readable after next() resolves
    const route = ctx.state.route;
    const target =
      typeof route?.handler === 'string' ? `${route.info.type} ${route.handler}` : 'no controller';
    const elapsed = Date.now() - startedAt;
    const level = elapsed > slowMs ? 'warn' : 'info';
    strapi.log[level](`[trace] ← ${ctx.status} ${ctx.method} ${ctx.path} (${target}, ${elapsed} ms)`);
  };
};
```

Zewnętrzna funkcja (fabryka) wykonuje się raz, przy starcie, i dostaje konfigurację z `config/middlewares.ts`; zwracana funkcja `(ctx, next)` – przy każdym żądaniu. W drodze do środka middleware nie zna jeszcze trasy, bo `ctx.state.route` ustawia router, który działa „głębiej". Dopiero po `await next()` znamy trasę, status i czas.

Do śladu dochodzą trzy krótsze elementy: middleware Document Service zarejestrowany w `register` (`strapi.documents.use(...)`, pełny plik w `examples/`), demonstracyjna polityka `global::trace-policy`, która zawsze zwraca `true`, oraz middleware trasy, podpięte do akcji `find` w core routerze artykułów:

```ts
export default factories.createCoreRouter('api::article.article', {
  config: {
    find: {
      policies: ['global::trace-policy'],
      middlewares: [traceRouteMiddleware],
    },
  },
});
```

Po starcie (`PORT=1301 npx strapi develop --no-watch-admin --no-build-admin`) wysyłasz dwa żądania: publiczną listę artykułów i anonimową próbę utworzenia artykułu.

```
$ curl -sg 'localhost:1301/api/articles?fields[0]=title&sort=title&pagination[pageSize]=2'
[trace] → GET /api/articles?fields[0]=title&sort=title&pagination[pageSize]=2
[trace]   route policy
[trace]   route middleware
[trace]   documents api::article.article findMany
[trace]   documents api::article.article count
[trace] ← 200 GET /api/articles (content-api api::article.article.find, 21 ms)
http: GET /api/articles?fields[0]=title&sort=title&pagination[pageSize]=2 (21 ms) 200

$ curl -s -X POST localhost:1301/api/articles \
    -H 'Content-Type: application/json' -d '{"data":{"title":"Anonymous draft"}}'
[trace] → POST /api/articles
[trace] ← 403 POST /api/articles (content-api api::article.article.create, 10 ms)
```

Pierwsze żądanie przeszło przez wszystkie warstwy w kolejności z diagramu. Kontroler `find` wywołał Document Service dwa razy: raz po dane, raz po licznik do `meta.pagination`. Linia `http:` pochodzi z wbudowanego `strapi::logger`, który stoi w konfiguracji przed naszym middleware, więc jest „bardziej na zewnątrz" cebuli i loguje jako ostatni. Drugie żądanie odbiło się na etapie `authorize`: rola Public nie ma uprawnienia `create`, więc nie wykonała się żadna polityka ani żadne zapytanie do bazy. To ważne przy projektowaniu: polityki i middleware trasy działają tylko dla żądań, które przeszły już sprawdzenie uprawnień ról.

Content API i Admin API

Strapi wystawia dwa rodzaje tras, a log pokazuje je w polu `route.info.type`. Trasy Content API (`content-api`) to te, z których korzysta frontend Redakcji: domyślnie pod prefiksem `/api` (opcja `api.rest.prefix`), uwierzytelniane JWT użytkownika users-permissions albo API tokenem. Trasy administracyjne (`admin`) obsługują panel: `/admin/*`, `/content-manager/*`, `/upload/*` i trasy admina innych pluginów. Uwierzytelnia je osobny JWT administratora, a dostęp sprawdzają polityki RBAC panelu, takie jak `admin::hasPermissions`.

Tak wygląda ślad, gdy redaktorka otwiera listę artykułów w panelu (żądanie wysłane curl-em z tokenem administratora):

```
[trace] → GET /content-manager/collection-types/api::article.article?page=1&pageSize=2
[trace]   documents api::article.article findMany
[trace]   documents api::article.article count
[trace] ← 200 GET /content-manager/collection-types/api::article.article (admin collection-types.find, 44 ms)
```

Globalny middleware i middleware Document Service zadziałały, ale polityka i middleware trasy artykułów – nie, bo panel nie używa tras z `src/api/article/routes`. Odpowiedź też się różni: panel widzi 4 artykuły (również ten, który ma tylko wersję roboczą), publiczne API – 3. Wniosek: reguła, która ma obowiązywać niezależnie od tego, skąd przyszła zmiana (slug, czas czytania, blokada usuwania kategorii), należy do Document Service, a nie do warstwy tras; to temat rozdziału 6. GraphQL ma własny mechanizm polityk i middleware (rozdział 3).

register, bootstrap, destroy

Plik `src/index.ts` eksportuje trzy funkcje cyklu życia aplikacji. Strapi czeka na każdą z nich (mogą być `async`), a błąd w którejkolwiek przerywa start.

`register` wykonuje się jako pierwsza, po `register` pluginów, ale przed połączeniem z bazą, przed inicjalizacją middleware i tras. To miejsce na rejestrowanie rzeczy, które muszą istnieć, zanim aplikacja ruszy: middleware Document Service, custom fields, rozszerzenia pluginów. `bootstrap` startuje, gdy baza jest zsynchronizowana ze schematami, a middleware i trasy zainicjalizowane, tuż przed otwarciem portu – tu ustawisz uprawnienia ról kodem, utworzysz brakujące dane startowe albo zaplanujesz zadania. `destroy` wywołuje się przy zamykaniu, także przy każdym przeładowaniu w trybie develop (w logu Redakcji widać `[trace] destroy` po każdej zmianie pliku), i służy do zamykania połączeń i zatrzymywania timerów.

> **Pułapka:** zapytanie do bazy w `register` nie działa. Po dopisaniu `await strapi.documents('api::article.article').count({})` do `register` start kończy się błędem `Error: Model api::article.article not found` – modele nie są jeszcze zarejestrowane w warstwie bazy. Ten sam kod w `bootstrap` zwraca liczbę artykułów.

Konfiguracja

Każdy plik w `config/` eksportuje obiekt albo funkcję, która dostaje `{ env }`. Funkcja `env()` czyta zmienne środowiskowe (także z `.env`) i ma warianty z rzutowaniem: `env.int`, `env.float`, `env.bool` (prawda tylko dla napisu `true`), `env.json`, `env.array`, `env.date` i `env.oneOf`. W Redakcji próg „wolnego" żądania z middleware śledzącego trafia do konfiguracji właśnie tak:

```
const config = ({ env }: Core.Config.Shared.ConfigParams): Core.Config.Middlewares => [
  'strapi::logger',
  { name: 'global::request-trace', config: { slowMs: env.int('TRACE_SLOW_MS', 200) } },
  'strapi::errors',
  // rest of the file unchanged
];
```

Własne pliki też są dozwolone. `config/editorial.ts` z polami `commentsPerHour: env.int('COMMENTS_PER_HOUR', 10)` i `moderationEnabled: env.bool('COMMENT_MODERATION', true)` jest dostępny w kodzie jako `strapi.config.get('editorial.commentsPerHour')`. Strapi odrzuca tylko kilka zastrzeżonych nazw plików (m.in. `environment`, `info`, `dirs`, `strapi`), ostrzegając w konsoli. Te same ścieżki z kropkami działają dla plików wbudowanych: `strapi.config.get('api.rest.maxLimit')` zwraca w Redakcji 100.

Konfigurację dla konkretnego środowiska kładziesz w `config/env/<NODE_ENV>/`. Przy `NODE_ENV=production` Strapi wczytuje najpierw `config/*`, potem `config/env/production/*` i scala je głęboko (lodash `merge`). Przydatna opcja na start: `autoOpen: false` w `config/admin.ts` wyłącza otwieranie przeglądarki przy każdym uruchomieniu `strapi develop`.

> **Pułapka:** głębokie scalanie łączy tablice element po elemencie, a nie zastępuje ich. Plik `config/env/production/middlewares.ts` z tablicą `['strapi::logger', 'strapi::errors']` nie skraca listy middleware do dwóch pozycji. W Redakcji `strapi.config.get('middlewares')` zwrócił wtedy 11 elementów, a na początku `['strapi::logger', 'strapi::errors', 'strapi::errors']` – wpis `global::request-trace` został nadpisany, a `strapi::errors` wystąpił dwa razy. Listę middleware dla produkcji podawaj w pełnej długości albo buduj ją w jednym pliku z warunkiem na `env('NODE_ENV')`.

Obiekt strapi i UID-y

W każdym miejscu kodu serwera masz dostęp do obiektu `strapi` – przez parametr (`{ strapi }` w fabrykach, politykach, funkcjach cyklu życia) albo globalnie. To on łączy wszystkie rejestry. Najczęściej będziesz używać kilku getterów: `strapi.documents(uid)` do treści, `strapi.service(uid)` i `strapi.controller(uid)` do warstw API, `strapi.plugin(name).service(name)` do serwisów pluginów (seed Redakcji tworzy użytkowników przez `strapi.plugin('users-permissions').service('user')`), `strapi.config.get(path)` do konfiguracji i `strapi.log` do logów. `strapi.db.query(uid)` schodzi poziom niżej, do Query Engine – kiedy to ma sens, wyjaśnia rozdział 6.

Identyfikatory mają stałe prefiksy, które mówią, skąd element pochodzi: `api::article.article` (typ treści, kontroler i serwis z `src/api`), `api::article.is-owner` (polityka lub middleware w obrębie jednego API), `plugin::users-permissions.user` (element pluginu), `admin::user` (panel), `global::request-trace` (globalna polityka lub middleware z `src/`) i `strapi::cors` (wbudowane middleware). Literówka w UID-zie wychodzi dopiero przy starcie albo przy wywołaniu – sprawdzisz je narzędziami z następnej sekcji.

CLI do introspekcji

Strapi pozwala zajrzeć do swoich rejestrów bez czytania kodu. `npx strapi routes:list` wypisuje wszystkie trasy – w Redakcji 244 wiersze, z czego tylko 18 to trasy czterech typów treści (`/api/articles`, `/api/categories` ...), a reszta należy do panelu i pluginów. `npx strapi content-types:list` pokazuje 22 typy treści, w tym `plugin::content-releases.release` i `plugin::review-workflows.workflow`, które istnieją w bazie, choć same funkcje wymagają płatnej licencji. `policies:list`, `controllers:list`, `services:list` i `hooks:list` działają analogicznie.

```
$ npx strapi middlewares:list (fragment)   $ npx strapi policies:list
global::request-trace                      global::trace-policy
strapi::compression                        plugin::content-manager.hasPermissions
strapi::errors                             admin::isAuthenticatedAdmin
strapi::responseTime                       admin::hasPermissions
plugin::users-permissions.rateLimit        admin::isTelemetryEnabled
```

Lista middleware zawiera wszystko, co jest *zarejestrowane*, a nie to, co działa. `strapi::compression` i `strapi::responseTime` są dostępne, ale nie biorą udziału w żądaniach, dopóki nie dopiszesz ich do `config/middlewares.ts`. To samo dotyczy własnych plików z `src/middlewares/`.

Najmocniejszym narzędziem jest `npx strapi console`: kompiluje projekt, uruchamia serwer w tle (na porcie z `PORT`, więc nie obok `strapi develop` na tym samym porcie) i otwiera REPL Node.js z globalnym obiektem `strapi`. REPL obsługuje `await`, więc sprawdzisz zapytanie, zanim wpiszesz je do kodu:

```
code > strapi.config.get('middlewares.1')
{ name: 'global::request-trace', config: { slowMs: 200 } }
code > strapi.contentType('api::article.article').options
{ draftAndPublish: true }
code > (await strapi.documents('api::article.article').findMany({ status: 'draft', fields: ['title'] })).length
[trace]   documents api::article.article findMany
4
```

Gdzie umieścić logikę

Mapa warstw przekłada się na prostą zasadę: umieść logikę na najniższej warstwie, która widzi wszystkie potrzebne jej informacje, i na najwyższej, która obejmuje wszystkie drogi zmiany danych.

Potrzeba w Redakcji	Miejsce	Rozdział
Nagłówek, czas żądania, CORS, logowanie każdego żądania	globalne middleware	4
Limit komentarzy na godzinę dla jednego endpointu	middleware trasy	4
Autor edytuje tylko swoje artykuły (tak/nie przed kontrolerem)	polityka	4
Nowy endpoint `POST /comments/:id/approve`, kształt odpowiedzi	kontroler + trasa	5
Logika biznesowa wielokrotnego użytku	serwis	5
Slug, `readingTime`, blokada usunięcia kategorii – także z panelu	middleware Document Service	6
Operacje masowe, agregaty bez warstwy dokumentów	Query Engine	6
Uprawnienia ról, dane startowe	`bootstrap`	1, 7

Granica, o którą najczęściej się potkniesz, przebiega między trasami a Document Service. Wszystko powyżej Document Service dotyczy tylko żądań Content API; wszystko w Document Service dotyczy też panelu admina, skryptów i `bootstrap`.

Podsumowanie

- Strapi to aplikacja Koa: globalne middleware działają jak cebula, kod po `await next()` widzi gotową odpowiedź.
- Projekt w TypeScripcie działa z `dist/`; pliki w `src/` są ładowane według konwencji katalogów i dostają UID-y z prefiksami `api::`, `plugin::`, `admin::`, `global::`, `strapi::`.
- Kolejność w obrębie trasy (Strapi 5.56): uwierzytelnienie, uprawnienia ról, polityki, middleware trasy, kontroler – polityki działają przed middleware trasy.
- Żądanie bez uprawnienia roli kończy się 403, zanim wykona się twoja polityka.
- Panel admina używa własnych tras; polityki i middleware tras Content API go nie dotyczą, middleware Document Service – tak.
- `register` nie ma dostępu do bazy, `bootstrap` ma; `destroy` wykonuje się przy każdym przeładowaniu w develop.
- `config/env/<env>/` scala się głęboko z `config/`, a tablice są łączone element po elemencie.
- `routes:list`, `middlewares:list`, `policies:list`, `content-types:list` i `console` pokazują, co naprawdę jest zarejestrowane.

Co dalej

Rozdział 2 schodzi do modelu treści Redakcji: `schema.json` od środka, relacje, komponenty i różnicę między dokumentem a jego wersjami. Bramki z diagramu tej części – trasy, polityki i middleware – rozbierzemy szczegółowo w rozdziale 4, kontrolery i serwisy w rozdziale 5, a Document Service z jego middleware w rozdziale 6.

Źródła

- https://docs.strapi.io/cms/backend-customization
- https://docs.strapi.io/cms/backend-customization/requests-responses
- https://docs.strapi.io/cms/backend-customization/routes
- https://docs.strapi.io/cms/backend-customization/middlewares
- https://docs.strapi.io/cms/backend-customization/policies
- https://docs.strapi.io/cms/api/content-api
- https://docs.strapi.io/cms/api/document-service/middlewares
- https://docs.strapi.io/cms/configurations
- https://docs.strapi.io/cms/configurations/functions
- https://docs.strapi.io/cms/configurations/environment
- https://docs.strapi.io/cms/configurations/guides/access-cast-environment-variables
- https://docs.strapi.io/cms/configurations/middlewares
- https://docs.strapi.io/cms/project-structure
- https://docs.strapi.io/cms/cli

Rozdział 2. Modelowanie treści

> **Cel rozdziału:** zrozumieć, jak `schema.json` zamienia się w tabele i odpowiedzi API, żeby projektować typy treści świadomie: wybierać między komponentem, relacją i dynamic zone oraz wiedzieć, co zmiana schematu robi z istniejącymi danymi.
>
> **Kod:** `examples/02-modelowanie-tresci/`. **Pakiety poza fundamentem:** brak.

W Redakcji model danych powstał w kilkanaście minut w Content-Type Builderze i od tamtej pory działa. Problem zaczyna się przy pierwszej zmianie: ktoś chce przemianować `excerpt` na `lead`, ktoś inny dodać do komentarza pole `status`, a frontendowiec pyta, dlaczego po edycji artykułu jego `id` się zmieniło. Odpowiedzi na te pytania nie ma w panelu. Leżą w plikach `schema.json` i w tym, co Strapi 5 robi z nimi w bazie.

Ten rozdział czyta schematy fundamentu linijka po linijce i tłumaczy, dlaczego wyglądają tak, a nie inaczej. Potem schodzi do bazy SQLite i pokazuje, że jeden dokument to dwa wiersze, a przemianowanie pola potrafi po cichu skasować dane. Zapytania REST (`populate`, `filters`) opisuje rozdział 3, a zapisy przez Document Service rozdział 6; tutaj interesuje nas kształt danych.

Schemat od środka

Typ treści to katalog `src/api/<nazwa>/content-types/<nazwa>/` z jednym plikiem `schema.json`. Strapi czyta go przy starcie, buduje z niego model bazy, trasy i uprawnienia, a w trybie developerskim generuje też typy TypeScript. Panel (Content-Type Builder) jest tylko edytorem tych plików. Oto komentarz (comment) z fundamentu, w skrócie:

```
{
  "kind": "collectionType",
  "collectionName": "comments",
  "info": { "singularName": "comment", "pluralName": "comments", "displayName": "Comment" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "body": { "type": "text", "required": true, "maxLength": 2000 },
    "moderationStatus": {
      "type": "enumeration",
      "enum": ["pending", "approved", "rejected"],
      "default": "pending",
      "required": true
    },
    "article": { "type": "relation", "relation": "manyToOne", "target": "api::article.article", "inversedBy":
"comments" },
    "author": { "type": "relation", "relation": "manyToOne", "target": "plugin::users-permissions.user" }
  }
}
```

`kind` wybiera między kolekcją (`collectionType`, wiele dokumentów) a pojedynczym typem (`singleType`, dokładnie jeden, jak `global` z ustawieniami strony). `singularName` i `pluralName` wyznaczają adresy (`/api/comments`) i UID `api::comment.comment`, a `collectionName` nazwę tabeli. Wartości `required`, `maxLength`, `min`, `unique` i `default` są walidacjami na poziomie schematu, więc obowiązują każde wejście: REST, panel i kod serwera. Sprawdź, jak wyglądają z zewnątrz:

```
$ curl -X POST localhost:1302/api/comments -H "authorization: Bearer eyJhbGciOi..." \
    -d '{"data":{"body":"hi","moderationStatus":"spam"}}'
400 ValidationError: moderationStatus must be one of the following values: pending, approved, rejected
```

Enumeracja odrzuca wartość spoza listy, a `default: "pending"` sprawia, że komentarz bez tego pola trafia do kolejki moderacji. Dlatego pole nazywa się `moderationStatus`, a nie `status`. W Strapi 5 `status` jest parametrem zapytania (`?status=draft`) i argumentem Document Service, więc nazwa jest zarezerwowana. Obok niej lista zawiera między innymi `meta`, `locale`, `localizations`, `document`, `entryId` i wszystko z przedrostkiem `strapi`.

> **Pułapka:** zarezerwowane nazwy pilnuje Content-Type Builder, nie sam serwer. Gdy dopiszesz `status` do `schema.json` artykułu ręcznie, Strapi 5.56 wystartuje i tylko ostrzeże w logu: „The attribute name 'status' on content type 'api::article.article' is reserved when 'draftAndPublish' is enabled". Kolumna powstanie, ale w żądaniach `?status=` i `filters[status]` pole i parametr zaczną się

> mylić. Dokumentacja opisuje tę regułę jako błąd w panelu, a zachowanie serwera jest łagodniejsze, więc w kodzie nie masz siatki bezpieczeństwa. Nazwę `meta` serwer przyjął u mnie bez słowa, mimo że lista ją zabrania. Nazywaj pola od razu inaczej: `moderationStatus`, `approvalState`.

Kilka typów pól wymaga komentarza. `uid` z `targetField: "title"` to unikalny, przyjazny adresom identyfikator, ale generuje go panel, nie API. Wysłany przez REST artykuł bez `slug` dostaje 400 (`slug must be a string type, but the final value was: null`), bo pole jest `required`. Automatyczny slug po stronie serwera to temat rozdziału 6. `blocks` to edytor z blokami w JSON-ie (akapity, nagłówki, listy), który frontend musi samodzielnie wyrenderować; `richtext` to starszy Markdown w jednym polu tekstowym. Pole `media` przechowuje relację do pliku z biblioteki mediów, a `allowedTypes` zawęża je do obrazów, wideo albo plików. Wreszcie `private: true` usuwa atrybut z każdej odpowiedzi API, co przydaje się przy danych wewnętrznych.

Komponent, relacja czy dynamic zone

Fundament używa wszystkich trzech sposobów łączenia danych i warto wiedzieć, dlaczego w tych miejscach. **Komponent** (component) to nazwana grupa pól bez własnej tożsamości: `shared.seo` ma `metaTitle`, `metaDescription` i `shareImage`, a artykuł zawiera dokładnie jedną jego kopię. Nie ma własnego adresu API ani uprawnień, nie da się go współdzielić między artykułami (każdy ma swoją kopię), za to jest tani w odczycie i edytuje się go razem z rodzicem. **Relacja** (relation) wskazuje na osobny dokument z własnym `documentId`, który żyje niezależnie: kategoria istnieje, nawet gdy nie ma w niej artykułów, i można ją edytować w jednym miejscu. **Dynamic zone** to uporządkowana lista komponentów różnych typów.

Reguła kciuka: jeśli dane należą tylko do jednego rodzica i nie potrzebujesz ich wyszukiwać ani chronić osobno, użyj komponentu; jeśli wiele rekordów ma je współdzielić albo rola ma mieć do nich osobne uprawnienia, użyj relacji. Dlatego SEO jest komponentem, a kategoria i komentarz są relacjami.

Pole `content` artykułu to dynamic zone z trzema komponentami: `shared.rich-text`, `shared.quote` i `shared.media`. Można było zrobić jedno pole `blocks`, ale `blocks` ogranicza się do tego, co umie edytor. Dynamic zone pozwala redaktorowi ułożyć tekst, cytat i obraz w dowolnej kolejności, a frontend przełącza się po `__component`. Cena jest realna. Komponenty trafiają do osobnych tabel, a kolejność trzyma tabela łącząca; sprawdź ją w bazie:

```
articles_cmps
 entity_id  cmp_id  component_type      field    order
 1          1       shared.rich-text    content  1
 1          1       shared.quote        content  2
 1          1       shared.seo          seo      null
```

Każdy typ komponentu w dynamic zone to dodatkowe złączenie przy `populate`, a odpowiedź ma strukturę, którą frontend musi rozpoznać po `__component`. Głębokie populate dynamic zone opisuje rozdział 3. Dla treści o stałym kształcie (tytuł, lead, okładka) dynamic zone to przesada; zostaw ją dla ciała artykułu.

Ostatni wybór: kategorie nie mają Draft & Publish (`draftAndPublish: false`), bo to słownik, który nie wymaga szkiców. Komentarze też nie: moderacja ma własne pole stanu, a nie dwie wersje dokumentu.

Relacje i użytkownik

Relacja ma właściciela. Strona z `inversedBy` jest właścicielem (komentarz wskazuje artykuł), a strona z `mappedBy` jedynie odbija ją z drugiej strony (artykuł zbiera swoje komentarze). Do wyboru jest pięć rodzajów: `oneToOne`, `oneToMany`, `manyToOne`, `manyToMany` i `oneWay` (bez drugiej strony). Fundament używa `manyToOne` / `oneToMany` dla pary artykuł–komentarz i artykuł–kategoria.

`author` jest inny, bo celem jest `plugin::users-permissions.user`, czyli typ treści z pluginu. Pole nie ma `inversedBy`, więc użytkownik nie dostaje listy swoich artykułów; to relacja jednokierunkowa w sensie modelu, choć zapisana jako `manyToOne`. Autor jest użytkownikiem aplikacji (rozdział 7), a nie kontem panelu admina, i dlatego komentarz może dostać autora z JWT bez żadnego mapowania. Konsekwencja: odpowiedź z populate `author` zawiera wszystkie pola użytkownika, jeśli nie przejdzie przez sanityzację (rozdział 5 pokazuje, jak łatwo wyciekają e-mail i hash hasła).

Dokument i jego wersje

W Strapi 5 wpis w panelu to **dokument** (document): jeden `documentId` i do kilku wierszy w bazie, po jednym na wersję. Artykuł z włączonym Draft & Publish ma wiersz szkicu (`published_at IS NULL`) i wiersz opublikowany; z i18n dochodzi jeszcze wiersz na każdy język. Zajrzyj do bazy świeżo zasianego projektu (`.tmp/data.db`, tabela `articles`):

```
id  document_id                title                          published
1   aku2wykoymjqm3e7amenfjxp   Policies in Strapi 5           0
2   aku2wykoymjqm3e7amenfjxp   Policies in Strapi 5           1
3   pw48hejujtw1vtmoh4579pa9   Document Service middlewares   0
4   pw48hejujtw1vtmoh4579pa9   Document Service middlewares   1
...
7   (czwarty artykuł)          Self-hosting Strapi with Docker 0
```

Cztery artykuły dają siedem wierszy, bo czwarty nigdy nie został opublikowany. Relacje i komponenty są kopiowane per wersja: tabela `articles_category_lnk` ma osobny wiersz łączący dla szkicu (`article_id = 1`) i dla opublikowanej wersji (`article_id = 2`), a komponent `shared.seo` osobne wiersze dla obu. Dlatego draft i published mogą mieć różne relacje, a dwa artykuły nie dzielą komponentu.

Trzeba też wiedzieć, jakie zachowanie widzi klient. REST domyślnie zwraca wersję opublikowaną, a `?status=draft` szkic. Ta sama odpowiedź ma w obu przypadkach ten sam `documentId` i różne `id`:

```
GET /api/articles?status=published&fields[0]=title   →  id 2, 4, 6
GET /api/articles?status=draft&fields[0]=title       →  id 1, 3, 5, 7
GET /api/articles/aku2wykoymjqm3e7amenfjxp           →  200 (documentId)
GET /api/articles/2                                  →  404 NotFound
```

Adres z numerycznym `id` w Strapi 5 nie istnieje, a pod `documentId` odpowiada wersja zależna od `status`. Rozstrzyga to zapis. Po `PUT` na artykuł (jako `anna`) opublikowany wiersz dostał **nowy** `id`:

```
PUT /api/articles/aku2wykoymjqm3e7amenfjxp  {"data":{"title":"Policies in Strapi 5, revised"}}
id  document_id                title                          published
1   aku2wykoymjqm3e7amenfjxp   Policies in Strapi 5, revised  0
8   aku2wykoymjqm3e7amenfjxp   Policies in Strapi 5, revised  1
```

> **Pułapka:** numeryczne `id` w Strapi 5 nie jest trwałym kluczem. Opublikowana wersja zmieniła `id` z 2 na 8 po zwykłej edycji, a `id` komponentów w szkicu i w wersji opublikowanej też są różne. Do tożsamości używaj wyłącznie `documentId`, także w kluczach cache i w relacjach po stronie frontendu. `id` zostaje wewnętrzną sprawą bazy. Pamiętaj też, że `PUT` na dokumencie z Draft & Publish w REST zmienił obie wersje naraz; sterowanie nimi osobno opisuje rozdział 8.

Przy okazji zwróć uwagę na jedną rzecz z seeda: żądanie anonimowe z `?status=draft` zwróciło szkice, w tym artykuł, który nigdy nie został opublikowany. Uprawnienie `find` roli Public otwiera oba parametry, więc szkice są publiczne, dopóki nie zablokujesz ich polityką albo middleware (rozdział 4). Komentarze, które nie mają Draft & Publish, mają w bazie jeden wiersz i `published_at` ustawione na datę utworzenia.

Panel w developmentcie, pliki w produkcji

Content-Type Builder działa tylko w trybie developerskim (`strapi develop`). W produkcji (`strapi start`) jest wyłączony, co jest zamierzone: schemat to kod, który wchodzi do repozytorium, przechodzi review i jedzie obrazem Dockera. Zmiana typu w panelu na serwerze produkcyjnym nie miałaby gdzie się zapisać, bo pliki w kontenerze znikają przy redeployu. Praca wygląda więc tak: edytujesz model lokalnie (panelem albo plikiem), commitujesz `schema.json` razem z kodem, a produkcja po starcie sama dopasowuje bazę. Podział na środowiska opisuje rozdział 13.

Panel generuje także `types/generated/contentTypes.d.ts` z typami takimi jak `ApiCommentComment`. Dzięki nim `strapi.documents('api::comment.comment')` wie, jakie pola istnieją. Generowanie i użycie typów opisuje rozdział 12; tu wystarczy wiedza, że plik odświeża się przy zmianie schematu i warto go commitować.

Zmiana schematu a dane

To najważniejsza praktyczna wiedza o modelu. Przy każdym starcie Strapi porównuje schematy z bazą i **synchronizuje** ją: dodaje nowe kolumny i tabele, a tabele i kolumny, których już nie ma w schematach, usuwa. Bez ostrzeżenia i bez pytania, w developmencie i w produkcji tak samo. Zmiana nazwy pola dla Strapi to więc usunięcie jednego pola i dodanie drugiego.

Sprawdź to. Przemianuj w `schema.json` artykułu `excerpt` na `lead` i uruchom serwer:

```
$ sed -i 's/"excerpt"/"lead"/' src/api/article/content-types/article/schema.json
$ strapi develop ...
articles columns: id, document_id, title, slug, reading_time, ..., locale, lead
GET /api/articles?fields[0]=lead  →  "lead": null
```

Kolumna `excerpt` zniknęła razem z treścią wszystkich leadów, a `lead` powstała pusta. Log startowy nie zawierał żadnego ostrzeżenia.

> **Pułapka:** zmiana nazwy pola w panelu albo w pliku kasuje dane przy następnym starcie, na każdym środowisku, które dostanie ten kod. Najgorzej, gdy zmiana jedzie na produkcję w zwykłym wdrożeniu. Przed zmianą nazwy zrób kopię bazy, a samą zmianę przeprowadź migracją.

Migracje trzymasz w `database/migrations/`. To pliki JavaScript nazwane datą (kolejność alfabetyczna to kolejność wykonania), każdy z funkcją `up(knex)`, uruchamiane raz, **przed** synchronizacją schematu. Dzięki temu migracja widzi jeszcze starą kolumnę:

```
// database/migrations/2026.10.03T12.00.00.rename-article-excerpt-to-lead.js
'use strict';

// Runs before the schema sync, so it still sees the old `excerpt` column.
async function up(knex) {
  const hasOld = await knex.schema.hasColumn('articles', 'excerpt');
  const hasNew = await knex.schema.hasColumn('articles', 'lead');
  if (hasOld && !hasNew) {
    await knex.schema.alterTable('articles', (table) => {
      table.renameColumn('excerpt', 'lead');
    });
  }
}

module.exports = { up };
```

Po przywróceniu bazy sprzed eksperymentu log pokazał `migrating 2026.10.03T12.00.00.rename-article-excerpt-to-lead.js`, a po starcie `lead` zawierała oryginalne teksty (`A short introduction to policies in strapi 5.`). Funkcja `up` działa w transakcji, więc błąd wycofuje całą migrację. Dokumentacja oznacza mechanizm jako eksperymentalny i zaznacza, że nie ma migracji w dół, więc cofnięcie robisz ręcznie. Strategię migracji na produkcji, kopie zapasowe i `forceMigration` omawia rozdział 13.

Podsumowanie

- `schema.json` jest źródłem prawdy: z niego powstają tabele, trasy i typy. Panel to tylko edytor, a w produkcji jest wyłączony, więc zmiany modelu przechodzą przez repozytorium.
- Walidacje (`required`, `maxLength`, `min`, enum, `default`) w schemacie obowiązują każde wejście. Pole `uid` generuje panel, API wymaga podania wartości.
- Nazwy `status`, `meta`, `locale`, `document` i przedrostek `strapi` są zarezerwowane. Serwer ich w plikach nie blokuje (najwyżej ostrzega), więc pilnuj tego sam i nazywaj pola jak `moderationStatus`.
- Komponent dla danych jednego rodzica (SEO), relacja dla rekordów żyjących własnym życiem (kategoria, komentarz), dynamic zone dla elastycznej listy bloków, ale z kosztem złączeń i populate.
- `inversedBy` oznacza stronę właściciela relacji, `mappedBy` stronę odbitą. Relacja do `plugin::users-permissions.user` bywa jednokierunkowa, a jej populate wymaga sanityzacji.
- Dokument to jeden `documentId` i wiele wierszy: szkic, wersja opublikowana, locale. `id` zmienia się przy edycji, więc tożsamością jest `documentId`.
- Zmiana nazwy pola kasuje dane przy starcie. Chroni przed tym migracja w `database/migrations/` uruchamiana przed synchronizacją schematu.

Co dalej

Masz model, który wiesz, jak czytać i bezpiecznie zmieniać. Rozdział 3 pokazuje, jak klient pobiera te dane przez REST i GraphQL: `populate` dynamic zone, `filters` po relacji i paginację. Do tego czasu możesz sprawdzić własny projekt: zajrzyj do tabel w bazie i policz, ile wierszy ma każdy dokument, oraz zastanów się, które pola w Twoich schematach wymagają migracji, zanim ktoś je przemianuje.

Źródła

- https://docs.strapi.io/cms/backend-customization/models
- https://docs.strapi.io/cms/features/content-type-builder
- https://docs.strapi.io/cms/api/document
- https://docs.strapi.io/cms/migration/v4-to-v5/breaking-changes/attributes-and-content-types-names-reserved
- https://docs.strapi.io/cms/migration/v4-to-v5/breaking-changes/reserved-attributes-status
- https://docs.strapi.io/cms/migration/v4-to-v5/breaking-changes/use-document-id
- https://docs.strapi.io/cms/database-migrations

Rozdział 3. Content API od strony klienta: REST, GraphQL, klient

> **Cel rozdziału:** nauczyć się pobierać i zapisywać treści Redakcji przez REST i GraphQL tak, żeby frontend dostawał dokładnie to, czego potrzebuje, a anonimowy klient nic ponad to.
>
> **Kod:** `examples/03-content-api/`. **Pakiety poza fundamentem:** `npm install @strapi/plugin-graphql` (backend); w kliencie `npm install @strapi/client qs` i `npm install -D @types/qs`.

Frontend Redakcji potrzebuje listy artykułów z okładką i kategorią, strony artykułu z całą strefą dynamiczną (dynamic zone) i danymi SEO oraz podglądu szkiców dla autorów. Pierwsze zapytanie `GET /api/articles` zwraca same tytuły i daty. Drugie, z `populate=*`, zwraca sporo, ale bez autora i obrazka SEO. Trzecie ujawnia coś gorszego: rola Public z samym uprawnieniem `find` czyta szkice, jeśli poprosi o nie w URL-u.

Wszystkie odpowiedzi w tym rozdziale pochodzą z fundamentu po `npm run seed` i skrypcie `scripts/attach-media.js` z `examples/`, który dokłada obrazek do artykułu „Policies in Strapi 5" jako okładkę, `seo.shareImage` i blok `shared.media`. Znaczniki czasu i część pól są skrócone, a długie URL-e złamane dla czytelności. Drogę żądania przez serwer opisuje rozdział 1, a obróbkę parametrów w kontrolerze rozdział 5.

Kształt odpowiedzi i `documentId`

Odpowiedź Strapi 5 jest płaska: pola leżą bezpośrednio w `data`, a nie w `data.attributes` jak w v4.

```
$ curl 'localhost:1303/api/articles?pagination[pageSize]=1'
{"data":[{"id":4,"documentId":"z18pcsia11q8v0uc2dr9h0lt","title":"Document Service middlewares", ...}],
 "meta":{"pagination":{"page":1,"pageSize":1,"pageCount":3,"total":3}}}
```

W URL-ach, relacjach i kodzie frontendu używasz wyłącznie `documentId`. Numeryczne `id` identyfikuje wiersz w bazie, a dokument z draft & publish ma ich kilka: „Policies in Strapi 5" ma `id` 8 w wersji opublikowanej i 1 w szkicu, przy tym samym `documentId` `e5uauilz20m1fx7eaoywnsk3`. Link albo klucz cache zbudowany na `id` rozjedzie się po pierwszej publikacji.

`populate`: nic nie przychodzi samo

REST domyślnie nie dołącza relacji, mediów, komponentów ani stref dynamicznych, bo każde dołączenie to dodatkowe zapytanie do bazy. `populate=*` dołącza wszystko, ale tylko jeden poziom w głąb:

```
$ curl 'localhost:1303/api/articles/e5uauilz20m1fx7eaoywnsk3?populate=*'
{"data":{"id":8, ..., "cover":{"id":1,"name":"policies-cover.png","width":64,"hash":"...","mime":"image/png", ...},
  "seo":{"id":10,"metaTitle":"Policies in Strapi 5","metaDescription":"Read about Policies in Strapi 5"},
  "category":{...}, "comments":[{...}], "content":[...]}, "meta":{}}
```

`seo` nie ma `shareImage`, bo obrazek w komponencie to drugi poziom. Nie ma też `author`: rola Public nie ma uprawnienia `find` na użytkownikach, więc `sanitizeQuery` po cichu usuwa relację z zapytania (rozdział 5). Zamiast błędu dostajesz odpowiedź bez pola, co łatwo przeoczyć. `populate=*` nadaje się do eksploracji w terminalu, nie do kodu: pobiera całe obiekty mediów i wszystkie komentarze, choć karta artykułu potrzebuje dwóch pól.

W kodzie wymieniasz dokładnie to, czego potrzebujesz. Strefa dynamiczna jest polimorficzna, więc Strapi 5 wymaga składni `on` z osobną konfiguracją dla każdego komponentu:

```
$ curl -g 'localhost:1303/api/articles/e5uauilz20m1fx7eaoywnsk3?fields[0]=title
    &populate[cover][fields][0]=url&populate[seo][populate][shareImage][fields][0]=url
    &populate[content][on][shared.rich-text]=true
    &populate[content][on][shared.quote][fields][0]=body
    &populate[content][on][shared.media][populate][file][fields][0]=url'
{"data":{"id":8,"documentId":"e5uauilz20m1fx7eaoywnsk3","title":"Policies in Strapi 5",
  "cover":{"id":1,"documentId":"jd0w1iw84o83s7dx2uc7o5zd","url":"/uploads/policies_cover_791620a0b1.png"},
  "seo":{"id":10,"metaTitle":"...","metaDescription":"...","shareImage":
{"id":1,"url":"/uploads/policies_cover_791620a0b1.png"}},
  "content":[{"id":9,"body":[{"type":"paragraph", ...}],"__component":"shared.rich-text"},
             {"id":9,"body":"Measure before you optimise.","__component":"shared.quote"},
             {"id":2,"file":{"id":1,"url":"/uploads/policies_cover_791620a0b1.png"},"__component":"shared.media"}]},
 "meta":{}}
```

Komponent `seo` to zwykły obiekt, więc jego obrazek dołączasz zagnieżdżonym `populate`. W strefie dynamicznej każdy klucz pod `on` to UID komponentu z własnymi `fields` i `populate`: cytat zwraca samo `body`, blok mediów dołącza plik, a `true` oznacza wszystkie pola skalarne. Literówka w UID daje 400 `Invalid key shared.nope at content.on.shared.nope`, nie pusty wynik.

> **Pułapka:** `populate[0]=content` zwraca bloki mediów jako `{"id":2,"__component":"shared.media"}`, bez pliku. Z kolei `on` działa jak lista dozwolonych komponentów: przy `populate[content][on][shared.quote]=true` akapity `shared.rich-text` znikają z odpowiedzi. Gdy redakcja doda do strefy nowy komponent, frontend nie zobaczy go, dopóki nie dopiszesz go do `on`.

Głębokie `populate` nie ma limitu poziomów, a jego koszt rośnie z każdym poziomem i każdą relacją „do wielu”. Pomiary i przeniesienie domyślnego `populate` na serwer opisuje rozdział 13.

`fields`, `filters` i `sort`

`fields` ogranicza pola skalarne (`documentId` i `id` przychodzą zawsze). `filters` przyjmuje operatory z prefiksem `$` (`$eq`, `$in`, `$containsi`, `$lt`, `$null` i inne). Filtr po relacji to zagnieżdżenie jej nazwy:

```
$ curl -g 'localhost:1303/api/articles?fields[0]=title&filters[category][slug][$eq]=backend'
{"data":[{"title":"Document Service middlewares", ...},{"title":"Policies in Strapi 5", ...}], ...}
$ curl -g 'localhost:1303/api/articles?fields[0]=title&sort=title:asc
    &filters[$or][0][title][$containsi]=policies&filters[$or][1][category][name][$eq]=Frontend'
{"data":[{"title":"Fetching Strapi from Next.js", ...},{"title":"Policies in Strapi 5", ...}], ...}
```

Klucze na jednym poziomie łączą się przez AND, a `$or` przyjmuje tablicę warunków, z których każdy może sięgać przez relację. Pierwszy artykuł pasuje przez kategorię, drugi przez tytuł. Filtry po relacji podlegają tym samym uprawnieniom co `populate`, tylko bez cichego pomijania: `filters[author][username][$eq]=anna` od anonimowego klienta daje 400 `Invalid key author`. `sort` przyjmuje `pole:asc` albo tablicę. Bez niego kolejność zależy od bazy i stronicowana lista może pokazać ten sam artykuł na dwóch stronach.

Paginacja i limity z `config/api.ts`

Paginacja działa po stronach (`pagination[page]`, `pagination[pageSize]`) albo po przesunięciu (`pagination[start]`, `pagination[limit]`). Mieszanie trybów daje 400 `PaginationError: Cannot use both page & offset pagination in the same query`. Fundament ustawia w `config/api.ts` `rest: { defaultLimit: 25, maxLimit: 100, withCount: true, strictParams: true }`.

`defaultLimit` obowiązuje, gdy klient nie poda paginacji. `maxLimit` przycina większe żądania bez błędu: `pagination[pageSize]=500` zwraca w `meta` `"pageSize":100`, więc frontend, który zakłada, że dostał wszystko, po cichu gubi resztę. Nie usuwaj tego wpisu w przekonaniu, że 100 to wartość domyślna, jak podaje tabela w dokumentacji. W Strapi 5.56 (październik 2026) domyślny jest tylko `defaultLimit` równy 25, a bez `maxLimit` to samo żądanie zwraca `"pageSize":500`. `withCount` dokłada `total` i `pageCount`; żądanie może to wyłączyć przez `pagination[withCount]=false` i oszczędzić `COUNT` na dużej tabeli.

`strictParams: true` zamienia nieznany parametr najwyższego poziomu w błąd: `?include=author` daje 400 `Invalid key include`. Literówka nie przechodzi niezauważona, a własne parametry tras rejestrujesz w `register` (dokumentacja, „Custom Content API parameters”). Inna rozbieżność z dokumentacją: `pagination[limit]=-1` miało zwracać „wszystko do `maxLimit`”, a REST zwraca jeden rekord (`"limit":1`). Wartość z query string jest tekstem `"-1"` i porównanie z liczbą `-1` w `@strapi/utils` jej nie rozpoznaje. Potrzebujesz wszystkiego – stronicuj w pętli do `pageCount`.

`status`: szkice i wersje opublikowane

Typ z draft & publish ma dwie wersje dokumentu. REST domyślnie zwraca opublikowane, a `status=draft` szkice, w całym żądaniu łącznie z dołączonymi relacjami. Pełny model opisuje rozdział 8; tu liczy się to, co widzi klient:

```
$ curl -g 'localhost:1303/api/articles?status=draft&fields[0]=title&fields[1]=publishedAt'
{"data":[{"id":1,"documentId":"e5uauilz20m1fx7eaoywnsk3","title":"Policies in Strapi 5","publishedAt":null}, ...,
        {"id":7,"documentId":"vgutht82kwd2mteh07ptyka6","title":"Self-hosting Strapi with
Docker","publishedAt":null}],
 "meta":{"pagination":{"page":1,"pageSize":25,"pageCount":1,"total":4}}}
```

To żądanie bez tokenu. Uprawnienie `find` nie rozróżnia wersji, więc każdy, kto zna parametr, czyta nieopublikowany tekst o Dockerze. Zapis ma odwrotną niespodziankę: w REST `published` jest domyślne także dla `POST` i `PUT`, w przeciwieństwie do Document Service. Anna tworzy szkic z `?status=draft`, a potem poprawia literówkę:

```
POST /api/articles?status=draft                       → 201 "documentId":"xftvajm7vh2ns8f0el2rjxs6","publishedAt":null
GET  /api/articles/xftvajm7vh2ns8f0el2rjxs6 (anonim)    → 404
PUT  /api/articles/xftvajm7vh2ns8f0el2rjxs6 {"data":{"excerpt":"Typo fix"}} → 200 "publishedAt":"2026-10-
03T18:37:42.613Z"
GET  /api/articles/xftvajm7vh2ns8f0el2rjxs6 (anonim)    → 200
```

> **Pułapka:** `PUT` bez `?status=draft` publikuje dokument. Edytor, który zapisuje szkic przez REST i zapomni o parametrze, wypuszcza na stronę niedokończony tekst. Klient pracujący na szkicach powinien przekazywać `status` jawnie w każdym zapisie, także w `create`.

Wyciek szkiców zamyka polityka (policy). Ma działać dla REST i, za chwilę, dla GraphQL, więc sprawdza typ kontekstu:

```
// src/policies/drafts-for-authors-only.ts (bez importów i typu kontekstu)
export default (policyContext: DraftPolicyContext) => {
  const requested = policyContext.is('graphql') ? policyContext.args?.status : policyContext.request.query.status;
  if (String(requested ?? '').toLowerCase() !== 'draft') return true;

  const { user, auth } = policyContext.state;
  if (user || auth?.strategy?.name === 'content-api-token') return true;

  throw new errors.PolicyError('Drafts require authentication');
};
```

W REST polityka czyta `request.query`, bo `policyContext.query` jest w politykach puste (rozdział 4 wyjaśnia dlaczego). W GraphQL argumenty leżą w `args`, a enum ma postać `DRAFT`. Strategia API tokenów nazywa się w Strapi 5.56 `content-api-token`, nie `api-token`. Po podpięciu w core routerze artykułów (`find`, `findOne`, jak w rozdziale 4) anonimowe `?status=draft` dostaje 403 `Drafts require authentication`, a z JWT Anny albo z tokenem read-only (np. podgląd w Next.js z rozdziału 11) – listę szkiców. To świadomy kompromis: każdy zalogowany użytkownik czyta wszystkie szkice, także cudze. Rozdział 7 domyka REST ostrzej, middleware trasy `published-only`, który wymusza `status=published` dla wszystkich poza API tokenem i daje autorom tylko ich własne szkice przez `/articles/mine`. Middleware tras nie obejmują jednak GraphQL, więc polityka zostaje strażnikiem dla obu API.

Jedna trasa to za mało. `GET /api/categories?status=draft&populate[articles][fields][0]=title` dalej zwracało szkic o Dockerze w kategorii DevOps, bo `status` przechodzi na dołączone relacje. Politykę trzeba podpiąć w każdym typie, z którego da się dojść do artykułów (w `examples/`: artykuły, kategorie, komentarze), albo zastąpić ją middleware globalnym z rozdziału 4. Podobnie do `status` działa `locale` (`?locale=pl`); fundament nie ma i18n, więc Strapi go ignoruje, a rozdział 8 pokazuje go na typie z tłumaczeniami.

Zapis relacji: `connect`, `disconnect`, `set`

Relację w `POST` i `PUT` ustawiasz przez `documentId`. Dla relacji „do jednego” wystarczy skrót `"category": "<documentId>"` albo `null`. Relację „do wielu” zmieniasz trzema operacjami: `connect` dokłada, `disconnect` odłącza, `set` zastępuje listę. Anna wysyła kolejne `PUT /api/articles/xftvajm7vh2ns8f0el2rjxs6?status=draft` (`C1`–`C3` to `documentId` jej komentarzy, `FRONTEND` i `DEVOPS` – kategorii); obok stan po zapisie:

```
{"category":{"connect":["FRONTEND"]},"comments":{"connect":["C1","C2"]}}  → Frontend ["First!","Does this work..."]
{"comments":{"connect":[{"documentId":"C3","position":{"start":true}}]}}   → Frontend ["What about
i18n?","First!","Does..."]
{"category":{"disconnect":["FRONTEND"]},"comments":{"disconnect":["C1"]}}  → null     ["What about i18n?","Does..."]
{"category":"DEVOPS","comments":{"set":["C1"]}}                            → DevOps   ["First!"]
{"category":null,"comments":{"set":[]}}                                    → null     []
```

`connect` i `disconnect` to aktualizacje częściowe, więc dwa równoległe zapisy z różnymi komentarzami się nie nadpisują. Długa forma z `position` (`start`, `end`, `before`, `after`) ustala kolejność, co ma sens dla list typu „artykuły polecane". Gołą tablicę Strapi traktuje jak `set`, ale `set` napisane wprost mówi czytelnikowi, że lista jest zastępowana. Nieistniejący `documentId` daje 400 `Document with id "..." not found`.

Pole `comments` to strona `mappedBy`, więc `connect` zmienia w rzeczywistości `article` w komentarzu i zabiera go artykułowi, do którego należał. Anna może też przepiąć cudze komentarze do swojego artykułu, bo uprawnienie `update` na artykułach nie mówi nic o komentarzach. Takie reguły należą do kontrolera albo middleware Document Service (rozdziały 5 i 6).

Klient: fetch z `qs` albo `@strapi/client`

Zapis z nawiasami jest nieczytelny i łatwo w nim o błąd. Dokumentacja zaleca `qs`: piszesz obiekt, a `qs.stringify(..., { encodeValuesOnly: true })` buduje URL. Pełny `client/src/fetch-qs.ts` ma typy odpowiedzi i obsługę błędów:

```ts
// client/src/fetch-qs.ts (fragment)
const query = qs.stringify(
  {
    fields: ['title', 'slug', 'excerpt', 'publishedAt'],
    populate: { cover: { fields: ['url', 'alternativeText'] }, category: { fields: ['name', 'slug'] } },
    filters: { category: { slug: { $eq: categorySlug } } },
    sort: ['publishedAt:desc', 'title:asc'],
    pagination: { page, pageSize: 10 },
  },
  { encodeValuesOnly: true },
);
const response = await fetch(`${baseUrl}/api/articles?${query}`);
```

```
$ STRAPI_URL=http://localhost:1303 node src/fetch-qs.ts backend
Policies in Strapi 5 [Backend] cover=/uploads/policies_cover_791620a0b1.png
Document Service middlewares [Backend] cover=-
```

Typ `ArticleCard` w skrypcie opisuje wynik tego konkretnego zapytania, a nie schemat typu treści. Kształt odpowiedzi zależy od `fields` i `populate`, więc jeden typ „Article" dla całego frontendu byłby fikcją; generowanie typów omawiają rozdziały 11 i 12.

`@strapi/client` (wersja 1.6, październik 2026) to oficjalna, cienka nakładka na `fetch`. Przyjmuje ten sam obiekt zapytania, dokłada nagłówek `Authorization` i zamienia odpowiedzi 4xx/5xx na wyjątki:

```ts
// client/src/strapi-client.ts (fragment)
const authorClient = strapi({ baseURL, auth: jwt });
const { data: draft } = await authorClient.collection('articles').create(
  { title: 'Typed fetching with @strapi/client', slug: 'typed-fetching-with-strapi-client', category: categoryId },
  { status: 'draft', fields: ['title', 'publishedAt'], populate: { category: { fields: ['name'] } } },
);
// update with an unknown field → HTTPBadRequestError; error.response holds Strapi's error body
```

Opcja `auth` jest opisana jako API token, ale to po prostu wartość nagłówka `Bearer`, więc działa też JWT z `/api/auth/local`. Logowanie i odświeżanie tokenów zostaje po stronie aplikacji (rozdziały 7 i 11). Wyniki mają typ `Document` z `[key: string]: any`, więc klient daje wygodę, nie bezpieczeństwo typów. Sięgnij po niego, gdy chcesz ujednolicić błędy i token w wielu miejscach; przy kilku zapytaniach w Server Components fetch z `qs` jest równie krótki.

Specyfikacja OpenAPI

`strapi openapi generate` (funkcja eksperymentalna) tworzy specyfikację OpenAPI 3.1 całego Content API bez startu serwera HTTP. Dla fundamentu to 425 KB i 38 ścieżek: CRUD typów treści, users-permissions (`/auth/local`, `/auth/refresh`, `/users/me`), upload, i18n. Parametry są opisane dokładnie, z enumami pól dla `fields` i `sort`. Gorzej ze schematami:

```
"servers": [{ "url": "", "description": "Default server" }],
"/articles/{id}": { "get": { "parameters": [{ "name": "id",
  "schema": { "type": "string", "format": "uuid", "description": "The document ID, represented by a UUID" } }] } },
"content": { "description": "A dynamic zone field", "type": "array", "items": {} }
```

Ścieżki nie mają prefiksu `/api`, a `servers` jest pusty, bo generator bierze adres z `server.url`. `documentId` udaje UUID, choć prawdziwe identyfikatory wyglądają jak `e5uauilz20m1fx7eaoywnsk3`, więc klient walidujący według specyfikacji odrzuci każdą odpowiedź. Strefa dynamiczna jest tablicą bez typu. Adres naprawia plik `config/openapi.ts` z kluczem `servers` (w `examples/`: `PUBLIC_URL` plus `/api`), który czyta `@strapi/openapi`, choć dokumentacja go nie opisuje. Specyfikacja przyda się do Swagger UI, Postmana i jako mapa API dla agentów AI; generatorom klientów z walidacją na razie nie ufaj. Serwowanie jej pod `/api/openapi.json` (`openapi` w `config/server.ts`) ujawnia całe API, także typy niepubliczne, więc w produkcji zostań przy CLI.

GraphQL

Plugin GraphQL (`npm install @strapi/plugin-graphql`) dodaje endpoint `/graphql` z Apollo Server i schematem wygenerowanym z typów treści (shadow CRUD). Bez konfiguracji start kończy się ostrzeżeniem „Built-in GraphQL operation limits are unbounded or invalid for: depthLimit, maxLimit”. To pierwsza rzecz do ustawienia:

```
// config/plugins.ts (fragment)
graphql: {
  config: { depthLimit: 7, defaultLimit: 25, maxLimit: 100, landingPage: env('NODE_ENV') !== 'production' },
},
```

Zapytanie `articles → category → articles…` głębsze niż 7 dostaje `'' exceeds maximum operation depth of 7`. `maxLimit` przycina `pageSize: 500` do 100, ale domyślnie w GraphQL wynosi `-1`, czyli brak limitu. `landingPage` włącza Apollo Sandbox. Opcja `amountLimit` ze starych poradników w Strapi 5 nie istnieje i jest po cichu ignorowana. Strona artykułu w GraphQL:

```
{ articles(filters: { category: { slug: { eq: "backend" } } }, sort: "title:asc") {
    documentId title
    seo { metaTitle shareImage { url } }
    content { __typename ... on ComponentSharedQuote { body } ... on ComponentSharedMedia { file { url } } }
} }
```

```
{"documentId":"e5uauilz20m1fx7eaoywnsk3","title":"Policies in Strapi 5",
 "seo":{"metaTitle":"Policies in Strapi 5","shareImage":{"url":"/uploads/policies_cover_791620a0b1.png"}},
 "content":[{"__typename":"ComponentSharedRichText"},{"__typename":"ComponentSharedQuote","body":"Measure before you
optimise."},
            {"__typename":"ComponentSharedMedia","file":{"url":"/uploads/policies_cover_791620a0b1.png"}}]}
```

To, co w REST wymaga `populate`, tu wynika z zapytania. Strefa dynamiczna to unia, więc pola komponentu wybierasz fragmentem `... on`, a komponenty bez fragmentu przychodzą z samym `__typename`, zamiast znikać. Identyfikatorem jest tylko `documentId`; `status: DRAFT`, `filters`, `sort` i `pagination` działają jak w REST, a metadane paginacji daje wariant `articles_connection { nodes pageInfo }`. Różni się obsługa uprawnień: `author { username }` od gościa nie znika po cichu, tylko daje `null` i błąd `Forbidden access` w `errors`. Ważniejsze jest to, czego nie widać:

```
REST     trasa → polityki trasy → middleware trasy → kontroler → serwis → Document Service
GraphQL  /graphql → auth i resolversConfig (middlewares, policies) → resolver → Document Service
```

Resolvery shadow CRUD walidują i sanityzują zapytanie, a potem wołają `strapi.documents()` bezpośrednio (`builders/resolvers/query.js` w pluginie), z pominięciem tras, kontrolerów i serwisów.

> **Pułapka:** po instalacji GraphQL wszystko, co zabezpieczyłeś w trasach i kontrolerach, ma obejście. Polityka `is-owner` z rozdziału 4 nie chroni mutacji `updateArticle`, filtr moderacji z rozdziału 5 nie działa w zapytaniu `comments`, a Anna tworzy mutacją `createComment(data: { body: "Buy cheap pills", moderationStatus: approved })` od razu zatwierdzony komentarz. Działa tylko to, co leży w Document Service (rozdział 6) albo w `resolversConfig`.

Redakcja zamyka te drogi w `register` w `src/index.ts`:

```
// src/index.ts (fragment; draftGuards maps nine article/category/comment queries to the draft policy)
const extensionService = strapi.plugin('graphql').service('extension');
extensionService.shadowCRUD('api::comment.comment').disableMutations();
extensionService.shadowCRUD('api::global.global').disableMutations();

extensionService.use(() => ({
  typeDefs: `type Query { articleBySlug(slug: String!): Article }`,
  resolvers: {
    Query: {
      articleBySlug: (_parent: unknown, args: { slug: string }) =>
        strapi.documents('api::article.article').findFirst({ filters: { slug: { $eq: args.slug } }, status:
'published' }),
    },
  },
  resolversConfig: {
    'Query.articleBySlug': { auth: { scope: ['api::article.article.findOne'] } },
    ...draftGuards,
  },
}));
```

`disableMutations()` usuwa mutacje komentarzy ze schematu, więc komentarze powstają tylko przez kontroler REST z walidacją z rozdziału 5; `createComment` kończy się teraz błędem `Cannot query field "createComment" on type "Mutation"`. Węższe opcje to `disableActions(['delete'])` i `field('moderationStatus').disableInput()`.

`articleBySlug` to własny resolver, bo wbudowane `article` przyjmuje tylko `documentId`, a frontend zna slug z URL-a. Zwraca dokument z Document Service, a typ `Article` z shadow CRUD rozwiązuje zagnieżdżone pola razem z kontrolą uprawnień do relacji. Klucz `auth` jest konieczny: bez niego gość dostaje `Forbidden access`, a każdy zalogowany użytkownik dowolnej roli – wynik. `scope` wiąże resolver z uprawnieniem `findOne` artykułów, czyli z tymi samymi rolami co REST. `draftGuards` podpina politykę `drafts-for-authors-only` pod wszystkie zapytania o artykuły, kategorie i komentarze: anonimowe `articles(status: DRAFT)` zwraca `"data": null` z błędem `Drafts require authentication`, a z JWT Anny albo tokenem read-only listę szkiców.

Dla Redakcji REST zostaje domyślnym wyborem: działa bez pluginu, przechodzi przez polityki i kontrolery z rozdziałów 4 i 5 i łatwo go cache'ować po URL-u. GraphQL ma sens, gdy wiele ekranów potrzebuje różnych wycinków tych samych danych albo zespół już pracuje z Apollo. Wtedy traktuj go jak drugie, równorzędne API, z własnymi limitami, politykami w `resolversConfig` i regułami biznesowymi w Document Service.

Podsumowanie

- Odpowiedź v5 jest płaska. W URL-ach i relacjach używaj `documentId`; `id` różni się między szkicem a wersją opublikowaną.
- REST nie dołącza niczego bez `populate`. W kodzie wymieniaj pola, a strefę dynamiczną dołączaj przez `on` z wpisem dla każdego komponentu, bo pominięte znikają z odpowiedzi.
- Relacje bez uprawnień `populate` pomija po cichu, a `filters` odrzuca błędem 400.
- `maxLimit` przycina `pageSize` bez błędu i nie ma wartości domyślnej; `strictParams` odrzuca nieznane parametry; `pagination[limit]=-1` w REST zwraca jeden rekord.
- `status=draft` czyta każdy z `find`, także przez relacje innych typów, a zapis przez REST bez `status=draft` publikuje dokument.
- `connect` i `disconnect` zmieniają relację częściowo, `set` zastępuje listę.
- `qs` i `@strapi/client` przyjmują ten sam obiekt zapytania; klient upraszcza błędy i token, ale nie daje typów.
- Specyfikacja z `strapi openapi generate` to dobra mapa API, ale bez prefiksu `/api` i z błędnym formatem `documentId`.
- GraphQL omija trasy i kontrolery REST: ustaw `depthLimit` i `maxLimit`, wyłącz zbędne mutacje i zabezpiecz zapytania w `resolversConfig`.

Co dalej

Trasy, kontrolery i parametry REST kontrolują tylko jedną z dróg do danych, a GraphQL pokazał, że ta sama treść może przyjść inną. Rozdział 4 wraca na serwer i opisuje trasy, polityki i middleware, w tym middleware globalne – właściwe miejsce na regułę, która ma objąć wszystkie typy naraz. Rozdział 8 rozwija draft & publish i i18n, a rozdział 11 składa całość we frontend w Next.js z podglądem szkiców i rewalidacją cache.

Źródła

- https://docs.strapi.io/cms/api/rest
- https://docs.strapi.io/cms/api/rest/populate-select
- https://docs.strapi.io/cms/api/rest/guides/understanding-populate
- https://docs.strapi.io/cms/api/rest/filters
- https://docs.strapi.io/cms/api/rest/sort-pagination
- https://docs.strapi.io/cms/api/rest/status
- https://docs.strapi.io/cms/api/rest/locale
- https://docs.strapi.io/cms/api/rest/relations
- https://docs.strapi.io/cms/configurations/api
- https://docs.strapi.io/cms/api/client
- https://docs.strapi.io/cms/api/openapi
- https://docs.strapi.io/cms/api/graphql
- https://docs.strapi.io/cms/api/graphql/advanced-policies
- https://docs.strapi.io/cms/plugins/graphql

Rozdział 4. Trasy, polityki i middleware

> **Cel rozdziału:** zrozumieć, co dzieje się z żądaniem między routerem a kontrolerem, i umieć świadomie wybrać miejsce dla reguły dostępu albo obróbki żądania: politykę, middleware trasy, middleware globalne albo kod w kontrolerze.
>
> **Kod:** `examples/04-routes-policies-middlewares/`. **Pakiety poza fundamentem:** brak.

W „Redakcji" rola Authenticated ma uprawnienie `update` na artykułach. To znaczy, że Piotr, zalogowany i z poprawnym tokenem, może zmienić tytuł artykułu Anny. Panel uprawnień users-permissions odpowiada tylko na pytanie „czy ta rola może wywołać tę akcję", a nie „czy ten użytkownik może zmienić ten konkretny dokument". Drugie pytanie wymaga kodu, który zna zarówno użytkownika, jak i dokument, i który uruchomi się, zanim kontroler cokolwiek zapisze.

Strapi daje na to kilka miejsc: polityki (policies), middleware trasy (route middlewares), middleware globalne i middleware Document Service. W dokumentacji i na forach przeczytasz, że „polityka to taki middleware". To prawda tylko w połowie. Polityka rzeczywiście jest funkcją wpiętą w łańcuch Koa, ale ma inną sygnaturę, widzi inny obiekt kontekstu i potrafi tylko przepuścić albo zatrzymać żądanie. Middleware owija resztę łańcucha i może zmienić zarówno żądanie, jak i odpowiedź.

Na działającym projekcie dopiszesz politykę `is-owner`, limit komentarzy na minutę i globalne nagłówki cache, a potem zobaczysz w logach, w jakiej kolejności to wszystko się wykonuje. Kontrolery i serwisy omawia rozdział 5, middleware Document Service i lifecycles – rozdział 6.

Co dokładnie dzieje się po dopasowaniu trasy

Najdokładniejszym opisem tego, co Strapi skleja z każdej trasy, jest plik `@strapi/core/dist/services/server/compose-endpoint.js` w `node_modules`. Dla każdej trasy Strapi tworzy łańcuch Koa w stałej kolejności:

```
config/middlewares.ts (globalne, w kolejności z tablicy)
  → router (dopasowanie metody i ścieżki)
    → routeInfo      ctx.state.route = konfiguracja trasy
    → authenticate   JWT / API token → ctx.state.user, ctx.state.auth
    → authorize      czy rola ma uprawnienie do akcji (403 Forbidden)
    → policies       config.policies, po kolei
    → middlewares    config.middlewares, po kolei
    → kontroler      zwrócona wartość trafia do ctx.body
      → serwis → Document Service (+ jego middleware) → Query Engine
```

Wynikają z tego trzy rzeczy. Polityki działają **po** sprawdzeniu uprawnień roli, więc jeśli rola Public nie ma akcji `update`, twoja polityka w ogóle się nie uruchomi. Middleware trasy działają **po** politykach, a nie przed nimi – żądanie odrzucone przez politykę albo przez `authorize` nigdy do nich nie dotrze, a middleware trasy nie przygotuje też danych dla polityki. I wreszcie: wszystko poniżej routera dotyczy tylko tras Content API. Panel admina ma własne trasy (`/content-manager/...`), na które twoje polityki i middleware tras nie są podpięte. To samo dotyczy GraphQL i wbudowanego serwera MCP: ich resolvery i narzędzia wołają Document Service bezpośrednio (rozdziały 3 i 12).

Trasy

Core router i jego konfiguracja

Każdy typ treści z fundamentu ma plik `src/api/<nazwa>/routes/<nazwa>.ts` z jedną linijką: `factories.createCoreRouter(uid)`. Ta fabryka tworzy pięć tras (`find`, `findOne`, `create`, `update`, `delete`) i przyjmuje opcjonalny obiekt konfiguracji. `only` i `except` wyłączają część tras, `prefix` dodaje przedrostek do ścieżek, a `config` pozwala każdej akcji z osobna przypiąć polityki, middleware i ustawienie `auth`.

Tak wygląda router artykułów po zmianach z tego rozdziału:

```ts
// src/api/article/routes/article.ts
import { factories } from '@strapi/strapi';

const isOwner = { name: 'global::is-owner', config: { uid: 'api::article.article' } };
const trace = { name: 'global::trace', config: { label: 'route middleware' } };

export default factories.createCoreRouter('api::article.article', {
  config: {
    update: { policies: ['global::trace-policy', isOwner], middlewares: [trace] },
    delete: { policies: [isOwner] },
  },
});
```

Polityka i middleware mogą być podane jako sama nazwa (`'global::trace-policy'`) albo jako obiekt `{ name, config }`. Drugi zapis pozwala użyć tej samej polityki z różnymi parametrami na różnych trasach – tu `is-owner` dostaje UID typu treści, którego dotyczy. Elementy z przedrostkiem `trace` to pomoce do obserwowania kolejności wykonania; wrócisz do nich w sekcji o logach.

Trasy własne i kolejność plików

Frontend „Redakcji" potrzebuje na stronie głównej trzech najnowszych artykułów. Możesz za każdym razem wysyłać `?sort=publishedAt:desc&pagination[pageSize]=3`, ale czytelniej jest mieć trasę `GET /api/articles/latest`. Nie potrzebujesz do niej nowego kontrolera: trasa własna może wskazać istniejącą akcję `find` i ustawić zapytanie w middleware przypiętym bezpośrednio jako funkcja.

```ts
// src/api/article/routes/01-custom.ts
import type { Core } from '@strapi/strapi';

// The route owns its query: client parameters (status, filters, populate) are not passed through.
const latestQuery: Core.MiddlewareHandler = async (ctx, next) => {
  ctx.query = {
    fields: ['title', 'slug', 'excerpt', 'publishedAt'],
    sort: 'publishedAt:desc',
    pagination: { pageSize: 3 },
  };
  await next();
};

export default {
  routes: [
    {
      method: 'GET',
      path: '/articles/latest',
      handler: 'api::article.article.find',
      config: { middlewares: [latestQuery] },
    },
  ],
};
```

Przypisanie do `ctx.query` przechodzi przez setter, który Strapi instaluje w middleware `strapi::query`: obiekt zostaje zserializowany przez `qs` z powrotem do query stringa. Kontroler `find` dostaje zapytanie tak, jakby wysłał je klient, łącznie z walidacją parametrów. Trasa korzysta z uprawnienia `find` roli Public, więc działa dla anonimowych czytelników, a zalogowany z ważnym tokenem też dostaje 200.

Przedrostek `01-` w nazwie pliku nie jest ozdobą. Strapi wczytuje pliki tras alfabetycznie, a router dopasowuje trasy w kolejności rejestracji. Gdy plik nazywa się `custom.ts`, trafia za `article.ts` i `GET /api/articles/latest` dopasowuje się najpierw do `GET /api/articles/:id` z `id = "latest"`:

```
$ curl -s 'localhost:1304/api/articles/latest'   # plik custom.ts
{"data":null,"error":{"status":404,"name":"NotFoundError","message":"Not Found","details":{}}}
```

Po zmianie nazwy na `01-custom.ts` trasa zwraca trzy artykuły. Kolejność sprawdzisz poleceniem `npx strapi routes:list`, które wypisuje trasy dokładnie w tej kolejności, w jakiej router je dopasowuje: `/api/articles/latest` musi stać nad `/api/articles/:id`.

> **Pułapka:** pierwsza wersja tej trasy miała `auth: false` i middleware `ctx.query = { ...ctx.query, sort, pagination }`. Oba skróty są ryzykowne. `auth: false` nie znaczy „dostęp dla Public", tylko „pomiń uwierzytelnianie i sprawdzanie roli" (`compose-endpoint.js` i serwis `auth` w `@strapi/core`). Trasa przestaje podlegać uprawnieniom z panelu, więc odebranie roli Public akcji `find` jej nie zamknie. `ctx.state.user` jest zawsze pusty, a śmieciowy token przechodzi z kodem 200 zamiast 401. Z kolei

> rozpakowanie `ctx.query` oddaje klientowi resztę parametrów: wywołanie `/api/articles/latest?status=draft&filters[title][$startsWith]=Self` zwróciło anonimowo szkic „Self-hosting Strapi with Docker” z `publishedAt: null`. Sanityzacja zapytania nadal usuwała `populate[author]` i odrzucała filtry po autorze, ale trasa „trzy najnowsze opublikowane” robiła już coś innego, niż obiecywała. Po poprawce ten sam adres zwraca trzy opublikowane artykuły, a zły token dostaje 401. `auth: false` zostaw dla tras, które same weryfikują wywołującego (webhook z podpisem, health check), a w middleware trasy buduj zapytanie od zera.

Polityki

Czym jest polityka

Polityka to funkcja `(policyContext, config, { strapi })`, którą Strapi wywołuje przed kontrolerem. Jej wynik interpretuje bardzo prosto: `true` albo `undefined` przepuszcza żądanie do następnej polityki, każda inna wartość kończy je błędem `PolicyError` z kodem 403 i komunikatem „Policy Failed”. Zamiast zwracać `false`, możesz rzucić własny `PolicyError` z `@strapi/utils` – wtedy klient dostanie twój komunikat i obiekt `details`. Polityka może być asynchroniczna; Strapi czeka na jej wynik.

Obiekt `policyContext` to płytka kopia kontekstu Koa (`Object.assign({ is, type }, ctx)` w `@strapi/utils`). Dzięki temu `policyContext.state` i `policyContext.params` wskazują na te same obiekty co w kontrolerze. Ale właściwości, które Koa udostępnia przez gettery prototypu, nie zostają skopiowane. W działającym projekcie widać to od razu: w polityce `policyContext.query` i `policyContext.method` mają wartość `undefined`, a `policyContext.request.query` i `policyContext.request.method` działają poprawnie. Polityka decyduje „tak albo nie” i nic więcej.

Polityka `is-owner`

Reguła „autor edytuje tylko swoje artykuły” jest klasycznym zadaniem dla polityki: przed zapisem trzeba pobrać dokument, porównać jego autora z zalogowanym użytkownikiem i albo przepuścić żądanie, albo je zatrzymać.

```ts
// src/policies/is-owner.ts (skrót, pełny plik w examples/)
type IsOwnerConfig = { uid: 'api::article.article' | 'api::comment.comment' };
export default async (
  policyContext: OwnerPolicyContext,
  config: IsOwnerConfig,
  { strapi }: { strapi: Core.Strapi }
): Promise<boolean> => {
  const user = policyContext.state.user;
  const documentId = policyContext.params.id;
  if (!user || !documentId) {
    return false;
  }
  const entry = await strapi.documents(config.uid).findOne({
    documentId,
    fields: ['documentId'],
    populate: { author: { fields: ['id'] } },
  });
  // Unknown document: let the controller answer with its regular 404.
  if (!entry) {
    return true;
  }
  if (entry.author?.id === user.id) {
    return true;
  }
  throw new errors.PolicyError('You can only modify your own entries', {
    policy: 'is-owner',
  });
};
```

W Strapi 5 parametr `:id` w ścieżce REST to `documentId`, czyli identyfikator dokumentu, a nie numeryczne `id` wiersza w bazie. Dlatego polityka szuka dokumentu przez `strapi.documents(uid).findOne({ documentId })`. Autor to relacja do `plugin::users-permissions.user`, więc porównujesz jego numeryczne `id` z `ctx.state.user.id`. Pobierasz tylko `documentId` i `id` autora, bo polityka działa przy każdym zapisie. Gdy dokument nie istnieje, polityka przepuszcza żądanie, a kontroler zwraca zwykłe 404. Odpowiedź 403 zdradzałaby, że coś pod tym identyfikatorem jest.

Typ `OwnerPolicyContext` to `Core.PolicyContext` uzupełniony o `state.user` i `params.id`. W Strapi 5.56 (październik 2026) typ `Core.PolicyContext` nie deklaruje tych pól, a `Core.PolicyHandler` opisuje funkcję synchroniczną, dlatego sygnatura asynchronicznej polityki jest wypisana ręcznie. Rozdział 12 wraca do typowania.

Wynik z uruchomionego projektu (`D` to `documentId` artykułu Anny):

```
PUT /api/articles/$D  (JWT piotra eyJhbGciOi...)  → 403
{"data":null,"error":{"status":403,"name":"PolicyError","message":"You can only modify your own entries","details":
{"policy":"is-owner"}}}
PUT /api/articles/$D  (JWT anny eyJhbGciOi...)    → 200
PUT /api/articles/$D  (bez tokenu)                → 403
{"data":null,"error":{"status":403,"name":"ForbiddenError","message":"Forbidden","details":{}}}
PUT /api/articles/doesnotexist000  (JWT anny)     → 404 NotFoundError
```

Zwróć uwagę na trzecią odpowiedź: to nie `PolicyError`, tylko `ForbiddenError` z etapu `authorize`. Rola Public nie ma uprawnienia `update`, więc żądanie zostało odrzucone, zanim polityka się uruchomiła. Ta sama polityka chroni `DELETE` – Anna próbująca usunąć artykuł Piotra dostaje identyczne 403.

Polityki globalne, API i pluginów

Miejsce pliku decyduje o nazwie polityki. Plik `src/policies/is-owner.ts` to polityka globalna `global::is-owner`, dostępna dla każdej trasy w projekcie. Plik `src/api/comment/policies/article-is-published.ts` to polityka API `api::comment.article-is-published`; na trasach tego samego API możesz ją podać krótko, jako `'article-is-published'`, bo Strapi dopisze przedrostek na podstawie `info.apiName` trasy. Polityki pluginów mają nazwy `plugin::<plugin>.<nazwa>`. Listę zarejestrowanych polityk wypisuje `npx strapi policies:list`.

Sama obecność pliku niczego nie włącza. „Globalna” znaczy „dostępna z każdego miejsca”, a nie „działa na wszystkich trasach” – każdą trasę trzeba wskazać w `config.policies`. Jeśli chcesz regułę dla wszystkich tras, potrzebujesz middleware, nie polityki.

Polityka API dla komentarzy pilnuje, żeby czytelnicy komentowali tylko opublikowane artykuły. Czyta ciało żądania, które o tej porze jest już sparsowane przez `strapi::body`:

```
// src/api/comment/policies/article-is-published.ts (rdzeń)
const articleId = policyContext.request.body?.data?.article;
if (typeof articleId !== 'string') {
  return false;
}
const published = await strapi.documents('api::article.article').count({
  filters: { documentId: articleId },
  status: 'published',
});
return published > 0;
```

Komentarz do szkicu artykułu kończy się teraz odpowiedzią `403` z `{"name":"PolicyError","message":"Policy Failed","details":{}}`. To domyślny komunikat po `return false`. Frontend nie odróżni go od innych polityk, dlatego w `is-owner` lepszy jest jawny `PolicyError` z `details.policy`.

> **Pułapka:** dokumentacja polityk podaje przykład `policies: ['plugin::users-permissions.isAuthenticated']`. W Strapi 5.56 plugin users-permissions nie eksportuje żadnych polityk (`npx strapi policies:list` ich nie pokazuje), a nieznana nazwa polityki wywraca start serwera: `Error creating endpoint GET /articles/mine: Policy plugin::users-permissions.isAuthenticated not found`. Jeśli potrzebujesz takiego sprawdzenia, napisz jednolinijkową politykę globalną `(ctx) => Boolean(ctx.state.user)`. Dobra strona: literówka w nazwie polityki wychodzi przy starcie.

Middleware

Middleware globalne i kod po `await next()`

Middleware to fabryka `(config, { strapi }) => async (ctx, next) => { ... }`. Fabryka działa raz przy starcie (tu przygotujesz stan, na przykład mapę liczników), zwrócona funkcja – przy każdym żądaniu. Wszystko, co jest przed `await next()`, dzieje się w drodze do kontrolera. Wszystko po – w drodze powrotnej, gdy status i ciało odpowiedzi są już ustalone. Ta druga połowa to zasadnicza różnica w stosunku do polityki.

„Redakcja” chce, żeby CDN i frontend mogły cache'ować anonimowe odczyty Content API, a wszystko inne było oznaczone `no-store`. Taka decyzja zależy od statusu odpowiedzi, więc musi zapaść po `await next()`:

```
// src/middlewares/cache-headers.ts (bez importów i typu konfiguracji)
const cacheHeaders: Core.MiddlewareFactory<CacheHeadersConfig> = (config) => {
  const maxAge = config.maxAge ?? 60;
  return async (ctx, next) => {
    await next();
    if (!ctx.path.startsWith('/api/')) {
      return;
    }
    const isPublicRead =
      ctx.method === 'GET' && ctx.status === 200 && !ctx.get('Authorization');
    ctx.set(
      'Cache-Control',
      isPublicRead ? `public, max-age=${maxAge}, stale-while-revalidate=30` : 'no-store'
    );
  };
};
```

Middleware globalne nie działa, dopóki nie wpiszesz go do `config/middlewares.ts`. Ta tablica to dosłownie stos Koa: kolejność elementów jest kolejnością wykonania w drodze „w dół”, a odwrotną kolejnością w drodze powrotnej. Wbudowane elementy `strapi::*` konfigurujesz w tym samym miejscu, zamieniając nazwę na obiekt `{ name: 'strapi::cors', config: { origin: [...] } }`. Tak samo ustawisz limity `strapi::body` (domyślnie `jsonLimit: '1mb'`) albo nagłówki CSP w `strapi::security`. Szczegóły CORS i CSP wracają w rozdziałach 8, 11 i 13.

```
// config/middlewares.ts
const config: Core.Config.Middlewares = [
  'strapi::logger',
  // Before strapi::errors: code after `await next()` sees the final status, also on errors.
  { name: 'global::trace', config: { label: 'global middleware' } },
  { name: 'global::cache-headers', config: { maxAge: 300 } },
  'strapi::errors',
  'strapi::security',
  // rest of the file unchanged
];
```

Anonimowe `GET /api/articles/latest` dostaje teraz `Cache-Control: public, max-age=300, stale-while-revalidate=30`, a każda odpowiedź zapisu albo z tokenem – `no-store`. Middleware globalne działa dla wszystkich żądań, także dla panelu admina i plików z `public/`. Stąd warunek na ścieżkę `/api/`.

> **Pułapka:** dokumentacja radzi dopisywać własne middleware na końcu tablicy. Przy takim ustawieniu odmowa z `is-owner` wróciła bez nagłówka `Cache-Control`, a log z bloku `finally` pokazał `status 404`, choć klient dostał 403. Polityka **rzuca** `PolicyError`, wyjątek przelatuje przez `await next()` twojego middleware i omija kod za nim. Na status 403 zamienia go dopiero `strapi::errors`, który leży wyżej na stosie. Middleware, które ma widzieć ostateczny status także przy błędach, umieść przed `strapi::errors` (albo otocz `next()` blokiem `try/finally` i pamiętaj, że status będzie jeszcze surowy).

Middleware trasy: limit komentarzy

Jeden zalogowany użytkownik ze skryptem może zalać moderację setką komentarzy w minutę. Limit dotyczy jednej trasy (`POST /api/comments`), potrzebuje identyfikacji użytkownika i musi ustawić nagłówki odpowiedzi. To zadanie dla middleware trasy, nie dla polityki: polityka nie ustawi `Retry-After` i nie zwróci statusu 429.

```
// src/api/comment/middlewares/rate-limit.ts (rdzeń)
const rateLimit: Core.MiddlewareFactory<RateLimitConfig> = (config, { strapi }) => {
  const max = config.max ?? 5;
  const windowMs = config.windowMs ?? 60_000;
  const windows = new Map<string, Window>();
  return async (ctx, next) => {
    const user = ctx.state.user as { id: number } | undefined;
    const key = user ? `user:${user.id}` : `ip:${ctx.request.ip}`;
    const now = Date.now();
    let window = windows.get(key);
    if (!window || window.resetAt <= now) {
      window = { count: 0, resetAt: now + windowMs };
      windows.set(key, window);
    }
    window.count += 1;
    ctx.set('X-RateLimit-Remaining', String(Math.max(0, max - window.count)));
    if (window.count > max) {
      const retryAfter = Math.ceil((window.resetAt - now) / 1000);
      ctx.set('Retry-After', String(retryAfter));
      return ctx.tooManyRequests('Too many comments, try again later', { retryAfter });
    }
    await next();
  };
};
```

Plik w `src/api/comment/middlewares/` rejestruje middleware pod nazwą `api::comment.rate-limit`. W routerze komentarzy podpinasz je razem z polityką:

```
// src/api/comment/routes/comment.ts
export default factories.createCoreRouter('api::comment.comment', {
  config: {
    create: {
      policies: ['article-is-published'],
      middlewares: [
        { name: 'api::comment.rate-limit', config: { max: 3, windowMs: 60_000 } },
      ],
    },
  },
});
```

Metoda `ctx.tooManyRequests` nie pochodzi z Koa. Strapi dodaje do kontekstu pomocnika dla każdego statusu 4xx i 5xx z `node:http` (`ctx.badRequest`, `ctx.forbidden`, `ctx.tooManyRequests`...), a odpowiedź ma ten sam format błędu co reszta API. Cztery komentarze Piotra w ciągu minuty:

```
POST /api/comments  → 201  X-RateLimit-Remaining: 2
POST /api/comments  → 201  X-RateLimit-Remaining: 1
POST /api/comments  → 201  X-RateLimit-Remaining: 0
POST /api/comments  → 429  Retry-After: 60
{"data":null,"error":{"status":429,"name":"TooManyRequestsError","message":"Too many comments, try again
later","details":{"retryAfter":60}}}
POST /api/comments (anna) → 201  X-RateLimit-Remaining: 2
```

Anna ma własne okno, bo kluczem jest `id` użytkownika. Ponieważ middleware trasy działa po politykach, komentarz do szkicu odrzucony przez `article-is-published` nie zużywa limitu. Mapa w pamięci procesu wystarcza dla jednej instancji. Przy kilku replikach za load balancerem każda liczy osobno i trzeba wspólnego magazynu, na przykład Redisa. Przy dużym ruchu trzeba też okresowo usuwać wygasłe okna. Strapi stosuje ten wzorzec u siebie: trasy `/api/auth/*` pluginu users-permissions mają middleware trasy `plugin::users-permissions.rateLimit` (domyślnie 10 żądań na 60 sekund, opcja `ratelimit` w konfiguracji pluginu).

`is-owner`: polityka czy middleware?

Strona dokumentacji o middleware zaleca, by „od Strapi v4” zastąpić politykę is-owner middleware trasy. Middleware ma realną przewagę, gdy reguła musi coś zmienić: dopisać filtr, przyciąć odpowiedź, ustawić pole. Dla decyzji „wolno albo nie” przy `update` i `delete` polityka jest prostsza i lepiej nazwana. Ma też jedną własność, której middleware nie ma: działa przed middleware trasy, więc odmowa nie zużywa na przykład limitu żądań.

Uważaj też na sam kod tego przykładu. `isOwner` z dokumentacji middleware wywołuje `strapi.documents(uid).findOne(entryId, { populate: '*' })`. To sygnatura w stylu Strapi 4. W Strapi 5 `findOne` przyjmuje jeden obiekt `{ documentId, ... }`, a wywołanie z ciągiem znaków zwraca po cichu `null`. Kod przykładu odczytuje potem

`entry.author.id` i każde żądanie kończy się błędem 500. Kopiując snippety z dokumentacji, sprawdzaj sygnatury Document Service z rozdziałem 6.

Osobna sprawa to lista. Polityka `is-owner` na `find` nie ma sensu: nie ma `params.id`, a polityka nie potrafi przefiltrować listy, może ją tylko w całości odrzucić. Naturalny odruch to middleware, które dopisuje do `ctx.query` filtr `author.id`. W „Redakcji" kończy się to tak:

```
GET /api/articles/mine  (middleware dopisuje filters[author][id]=<user.id>)
{"data":null,"error":{"status":400,"name":"ValidationError","message":"Invalid key author",...}}
```

Walidacja zapytania w kontrolerze sprawdza filtry względem uprawnień roli, a rola Authenticated nie może czytać użytkowników, więc relacja `author` jest dla niej niewidoczna. Filtr „tylko moje" należy ustawić tam, gdzie nie przechodzi przez walidację parametrów klienta: w kontrolerze (rozdział 5) albo w middleware Document Service (rozdziały 6 i 7).

Kolejność wykonania na logach

Żeby zobaczyć cały łańcuch, w przykładach są trzy pomocnicze elementy: middleware `global::trace` (ten sam plik użyty raz globalnie, raz na trasie, z różnym `label`), polityka `global::trace-policy` i jednolinijkowe middleware Document Service w `register` z `src/index.ts`, które loguje akcję i UID (mechanizm opisuje rozdział 6). Middleware `trace` loguje w bloku `finally`, więc zapisuje się także wtedy, gdy coś niżej rzuci wyjątek.

Anna zmienia swój artykuł (`PUT /api/articles/:id`, 200):

```
[trace] global middleware: before next()
[trace] policy: user=anna params={"id":"jwamfanw4h1oblx64jbjwk8k"}
[trace] document service: findOne api::article.article      ← is-owner pobiera autora
[trace] route middleware: before next()
[trace] document service: update api::article.article       ← kontroler → serwis
[trace] route middleware: after next(), status 200
[trace] global middleware: after next(), status 200
http: PUT /api/articles/jwamfanw4h1oblx64jbjwk8k (118 ms) 200
```

Piotr próbuje zmienić ten sam artykuł (403):

```
[trace] global middleware: before next()
[trace] policy: user=piotr params={"id":"jwamfanw4h1oblx64jbjwk8k"}
[trace] document service: findOne api::article.article
[trace] global middleware: after next(), status 403
http: PUT /api/articles/jwamfanw4h1oblx64jbjwk8k (31 ms) 403
```

Middleware trasy w ogóle się nie uruchomiło, a kontroler nie dostał żądania. Na koniec zmiana tego samego typu treści z panelu admina, wysłana przez Admin API (`PUT /content-manager/collection-types/api::article.article/:documentId` z tokenem administratora):

```
[trace] document service: findOne api::article.article
[trace] document service: update api::article.article
http: PUT /content-manager/collection-types/api::article.article/lgh5n19xmtzc38lwwzl2io09 (235 ms) 200
```

Ani polityka `is-owner`, ani middleware trasy nie zobaczyły tej zmiany, bo dotyczą wyłącznie tras Content API. Middleware globalne co prawda działa także dla tego żądania, ale `trace` pomija ścieżki spoza `/api/`. Jedyną warstwą, którą przechodzi zarówno REST, jak i panel, jest Document Service. Jeśli reguła ma obowiązywać niezależnie od tego, skąd przyszła zmiana (wyliczanie `readingTime`, slug, blokada usunięcia kategorii), jej miejsce jest w middleware Document Service, czyli w rozdziale 6.

Polityka, middleware czy kontroler

	Polityka	Middleware trasy	Middleware globalne	Middleware Document Service	Kontroler
Gdzie	`src/policies`, `src/api/*/policies`	`src/api/*/middlewares`, `src/middlewares`, funkcja w trasie	`src/middlewares` + `config/middlewares.ts`	`strapi.documents.use()` w `register`	`src/api/*/controllers`

	Polityka	Middleware trasy	Middleware globalne	Middleware Document Service	Kontroler
Kiedy działa	po `authorize`, przed middleware trasy	po politykach, wokół kontrolera	wokół całego żądania, każdego	wokół wywołania `strapi.documents()`	na końcu łańcucha trasy
Co widzi	kopia `ctx`: `state`, `params`, `request`	pełny `ctx`	pełny `ctx`, także trasy admina i pliki	`uid`, `action`, `params` – bez HTTP	pełny `ctx`, serwisy
Co może	przepuścić albo zatrzymać (403)	zmienić żądanie i odpowiedź, zakończyć	to samo co trasy, globalnie	zmienić dane i wynik operacji	wszystko, co zwraca odpowiedź
Panel admina	nie	nie	tak	tak	nie
Przykład w „Redakcji”	`is-owner`, komentarz tylko do opublikowanych	limit komentarzy, domyślne parametry `/latest`	`Cache-Control`, pomiar czasu	`readingTime`, slug (rozdz. 6)	akcja `approve`, odpowiedź własnego kształtu (rozdz. 5)

Praktyczna reguła wyboru jest krótka. Pytanie „czy ten użytkownik może wykonać tę akcję na tym dokumencie” to polityka. Coś, co zmienia żądanie albo odpowiedź HTTP dla wybranych tras, to middleware trasy, a dla wszystkich tras – middleware globalne. Reguła o danych, która musi działać także w panelu admina i w cronie, to middleware Document Service. Logika, która tworzy odpowiedź, to kontroler i serwis.

Podsumowanie

- Dla każdej trasy Strapi składa łańcuch: `routeInfo` → `authenticate` → `authorize` → polityki → middleware trasy → kontroler. Polityki działają po sprawdzeniu roli, middleware trasy – po politykach.
- `createCoreRouter(uid, { only, except, prefix, config })` konfiguruje polityki, middleware i `auth` dla każdej akcji z osobna. Trasy własne wczytywane są alfabetycznie, dlatego `01-custom.ts` musi wyprzedzić `article.ts`, gdy ścieżka koliduje z `/:id`.
- `auth: false` wyłącza uwierzytelnianie i uprawnienia roli, a nie „otwiera trasę dla Public”. Middleware trasy powinno budować zapytanie od zera, a nie rozpakowywać `ctx.query` klienta.
- Polityka zwraca `true` / `undefined` albo zatrzymuje żądanie; `PolicyError` z `@strapi/utils` daje własny komunikat i `details`. `policyContext` to płytka kopia `ctx`, więc używaj `request.query` i `request.body`, nie `query` i `body`.
- `is-owner` sprawdza `documentId` z `params.id` przez Document Service. Nieznany dokument zostawia kontrolerowi (404), a cudzy blokuje (403).
- Middleware globalne trzeba wpisać do `config/middlewares.ts`. Kod po `await next()` widzi ostateczny status tylko wtedy, gdy middleware leży przed `strapi::errors`.
- Limit żądań to middleware trasy: ma dostęp do użytkownika, ustawia nagłówki i zwraca 429 przez `ctx.tooManyRequests`.
- Polityki i middleware tras nie widzą zmian z panelu admina; regułę o danych umieszczasz w middleware Document Service.

Co dalej

Polityki i middleware decydują, czy żądanie dotrze do kontrolera i w jakiej postaci. W rozdziale 5 przejdziesz na drugą stronę tej granicy: rozszerzysz kontroler artykułów, przeniesiesz logikę do serwisu, dodasz akcję `POST /comments/:id/approve` z trasą z tego rozdziału i zobaczysz, jak `sanitizeQuery` i `sanitizeOutput` chronią przed wyciekiem danych, które właśnie zablokowały filtr `author` na liście.

Źródła

- https://docs.strapi.io/cms/backend-customization
- https://docs.strapi.io/cms/backend-customization/routes
- https://docs.strapi.io/cms/backend-customization/policies
- https://docs.strapi.io/cms/backend-customization/middlewares
- https://docs.strapi.io/cms/configurations/middlewares

- https://docs.strapi.io/cms/backend-customization/examples/policies
- https://docs.strapi.io/cms/backend-customization/examples/routes
- https://docs.strapi.io/cms/backend-customization/examples/middlewares
- https://docs.strapi.io/cms/error-handling
- https://docs.strapi.io/cms/api/document-service/middlewares
- https://docs.strapi.io/cms/cli

Rozdział 5. Kontrolery, serwisy, walidacja i błędy

> **Cel rozdziału:** nauczyć się rozszerzać kontrolery i serwisy tak, żeby nowe endpointy nie wypuszczały prywatnych danych, nie ufały ciału żądania i zwracały czytelne błędy.
>
> **Kod:** `examples/05-kontrolery-i-serwisy/`. **Pakiety poza fundamentem:** brak (`zod` i `yup` są reeksportowane przez `@strapi/utils`).

Redakcja dostaje pierwsze prośby, których nie da się wyklikać w panelu. Pod artykułem ma się pojawić blok „podobne artykuły", komentarze mają czekać na moderację, a redaktor zatwierdza je jednym przyciskiem. Wszystko to ląduje w kontrolerach (controllers) i serwisach (services), czyli w warstwie, która w fundamencie składa się z trzech linijek `factories.createCore...`.

Kod w tej warstwie pisze się szybko i równie szybko psuje bezpieczeństwo. Zobaczysz w curl-u dwa prawdziwe problemy z typowego projektu: endpoint zwracający hash hasła autora i formularz komentarza, przez który użytkownik sam zatwierdza swój spam. Oba wynikają z założenia „Strapi to za mnie sprawdzi". Trasy i polityki opisuje rozdział 4, Document Service rozdział 6; tutaj zajmujemy się tym, co pomiędzy: kontrolerem, który rozmawia z HTTP, i serwisem, który zna reguły biznesowe.

Co robi rdzeń

`factories.createCoreController('api::comment.comment')` tworzy kontroler z pięcioma akcjami: `find`, `findOne`, `create`, `update` i `delete`. Core router z fundamentu kieruje na nie trasy `GET /api/comments`, `POST /api/comments` i pozostałe. Warto raz zajrzeć do implementacji w `node_modules/@strapi/core/dist/core-api/controller/collection-type.js`, bo każda akcja to ten sam schemat:

```
create(ctx)
  validateQuery(ctx)          → 400, gdy query zawiera niedozwolone pola
  sanitizeQuery(ctx)          → usuwa z query to, czego rola nie może czytać
  validateInput(data, ctx)    → 400 przy nieznanych kluczach i zabronionych relacjach
  sanitizeInput(data, ctx)    → usuwa pola tylko do odczytu
  strapi.service(uid).create  → core service → strapi.documents(uid).create
  sanitizeOutput(entity, ctx) → usuwa pola prywatne i relacje bez uprawnień
  transformResponse(...)      → { data, meta }
```

Core service jest jeszcze cieńszy: dokłada `status: 'published'` jako wartość domyślną i woła Document Service. Cała ochrona danych siedzi więc w kontrolerze. Walidacja (validate) sprawdza i rzuca błąd 400, sanityzacja (sanitize) po cichu wycina to, czego nie powinno być. Obie korzystają ze schematu typu treści i z uprawnień roli, która wykonuje żądanie (`ctx.state.auth`).

Rozszerzanie z `super`

Pierwsze zadanie: publiczna lista komentarzy ma zwracać tylko zatwierdzone. Nie piszesz `find` od zera, tylko modyfikujesz wejście i oddajesz sterowanie rdzeniowi:

```
// src/api/comment/controllers/comment.ts (fragment)
export default factories.createCoreController('api::comment.comment', ({ strapi }) => ({
  async find(ctx) {
    const filters = ctx.query.filters ?? {};
    ctx.query = { ...ctx.query, filters: { $and: [filters, { moderationStatus: 'approved' }] } };
    return super.find(ctx);
  },
}));
```

Filtry klienta trafiają do `$and` razem z warunkiem moderacji, więc nie da się go nadpisać parametrem w URL-u. Próba wyciągnięcia oczekujących komentarzy daje pustą listę:

```
$ curl -g 'localhost:1305/api/comments?filters[moderationStatus]=pending'
{"data":[],"meta":{"pagination":{"page":1,"pageSize":25,"pageCount":0,"total":0}}}
```

`super.find` wykonuje całą sekwencję z diagramu, więc walidację i sanityzację dostajesz za darmo – dlatego dokumentacja zaleca rozszerzanie core controllera zamiast pisania własnego. Gdy akcję nadpisujesz w całości albo dodajesz nową, sekwencję odtwarzasz sam. Gdyby filtr miał działać także dla zapytań z kodu serwera, lepszym miejscem jest middleware Document Service z rozdziału 6.

Sanityzacja: wyciek, który widać w curl-u

Blok „podobne artykuły" to nowa akcja `related`. Najprostsza wersja, jaką pisze się w pośpiechu, woła Document Service i zwraca wynik:

```
async related(ctx) {
  const article = await strapi.documents('api::article.article').findOne({
    documentId: ctx.params.id, status: 'published', populate: ['category'],
  });
  if (!article?.category) return ctx.notFound();
  return strapi.documents('api::article.article').findMany({
    status: 'published',
    filters: { category: { documentId: article.category.documentId }, documentId: { $ne: article.documentId } },
    populate: ['author', 'category'],
    limit: 3,
  });
},
```

Anonimowe żądanie (rola Public ma uprawnienie do `related`, o czym za chwilę):

```
$ curl localhost:1305/api/articles/fn3wajqrwfq0k9pifujh59er/related
[{"documentId":"e8nmfxdpzlhpts6e85ftz6ia","title":"Document Service middlewares",
  "author":{"id":1,"username":"anna","email":"anna@example.com",
    "password":"$2a$10$BwxtogY6E0gkgOEqfmNyaOcF.yqBdF4ZMBmqR/P6Ww6...",
    "resetPasswordToken":null,"confirmed":true,"blocked":false, ...}, ...}]
```

Każdy odwiedzający widzi e-mail i hash hasła autorki. Document Service działa z uprawnieniami serwera: zwraca wszystkie pola, także prywatne (`password` w użytkowniku users-permissions), i każdą relację, o którą poprosisz. Pola prywatne i relacje, do których rola nie ma dostępu, wycina dopiero `sanitizeOutput`. Rola Public nie ma uprawnienia `find` na użytkownikach, więc po sanityzacji relacja `author` znika z odpowiedzi. Brakuje też koperty `{ data, meta }`, której oczekuje frontend.

> **Pułapka:** zwracanie surowego wyniku `strapi.documents()` albo `strapi.db.query()` z kontrolera. Kod przechodzi testy, bo testujesz go zwykle jako właściciel danych, a wyciek widać dopiero z perspektywy anonimowego klienta. Każda akcja, która nie kończy się na `super.*`, powinna kończyć się na `sanitizeOutput`.

Poprawiona wersja ma pięć linii logiki i resztę oddaje serwisowi:

```
// src/api/article/controllers/article.ts
import { factories } from '@strapi/strapi';
import type { Core } from '@strapi/strapi';

// Core helpers (sanitizeOutput, transformResponse) are optional in the factory's `this` type.
type CoreController = Core.CoreAPI.Controller.CollectionType;

export default factories.createCoreController('api::article.article', ({ strapi }) => ({
  async related(this: CoreController, ctx) {
    await this.validateQuery(ctx);
    const query = await this.sanitizeQuery(ctx);

    const articles = await strapi.service('api::article.article').findRelated(ctx.params.id, query);

    const sanitized = await this.sanitizeOutput(articles, ctx);
    return this.transformResponse(sanitized);
  },
}));
```

Adnotacja `this: CoreController` nie jest ozdobnikiem. W typach Strapi metody pomocnicze w `this` są opcjonalne (`Partial`), więc przy `strict: true` z fundamentu `this.sanitizeOutput(...)` kończy się błędem `TS2722: Cannot invoke an object which is possibly 'undefined'`. Przykłady TS w dokumentacji tego nie pokazują.

Teraz `?populate=author` od anonimowego klienta zwraca 200 bez pola `author`, a `?fields[0]=title&populate=category` działa normalnie. `validateQuery` nie sprawdza `populate`, więc zabroniona relacja jest cicho usuwana przez `sanitizeQuery`, a nie odrzucana błędem. Helpery `this.*` działają na modelu kontrolera. Jeśli w kontrolerze artykułów zwracasz kategorie, użyj `strapi.contentAPI.sanitize.output(data, strapi.contentType('api::category.category'), { auth: ctx.state.auth })`, inaczej sanityzacja pójdzie według złego schematu.

Cienki kontroler, logika w serwisie

Kontroler zna HTTP: parametry, query, użytkownika, status odpowiedzi. Serwis zna Redakcję: czym są „podobne artykuły" i kiedy komentarz da się zatwierdzić. Serwis nie dostaje `ctx`, tylko zwykłe argumenty, dzięki czemu wywołasz go z crona, z innego kontrolera albo z testu.

```
// src/api/article/services/article.ts
type FindManyParams = Modules.Documents.ServiceParams<'api::article.article'>['findMany'];
type RelatedQuery = Pick<FindManyParams, 'fields' | 'populate'>;

export default factories.createCoreService('api::article.article', ({ strapi }) => ({
  async findRelated(documentId: string, { fields, populate }: RelatedQuery = {}) {
    const article = await strapi.documents('api::article.article').findOne({
      documentId, status: 'published', populate: ['category'],
    });
    if (!article) throw new errors.NotFoundError('Article not found');
    if (!article.category) return [];

    return strapi.documents('api::article.article').findMany({
      fields, populate, status: 'published',
      filters: { category: { documentId: article.category.documentId }, documentId: { $ne: documentId } },
      sort: 'publishedAt:desc',
      limit: RELATED_LIMIT,
    });
  },
}));
```

Serwis przejmuje z zapytania klienta tylko `fields` i `populate`; filtry, sortowanie i limit definiują „podobne artykuły", więc klient ich nie zmienia. Brak artykułu to `NotFoundError` z `@strapi/utils`, który Strapi zamienia na 404 bez kodu w kontrolerze. Typ parametrów z `Modules.Documents` sprawia, że literówka w nazwie pola wychodzi w `tsc`.

Nowa akcja potrzebuje trasy. Wystarczy plik obok core routera:

```
// src/api/article/routes/01-custom-article.ts
export default {
  routes: [
    { method: 'GET', path: '/articles/:id/related', handler: 'api::article.article.related' },
  ],
};
```

Konfigurację tras, kolejność plików i `auth: false` omawia rozdział 4. Tu zapamiętaj dwie rzeczy. Po pierwsze, nowa akcja pojawia się w users-permissions jako osobne uprawnienie i dopóki go nie nadasz roli, każde żądanie dostaje 403. W `examples/` jest skrypt `scripts/grant-ch05.js`, który nadaje `related` rolom Public i Authenticated oraz tworzy rolę Editor z uprawnieniem `approve` (ustawianie uprawnień kodem rozwija rozdział 7). Po drugie, trasa wskazująca na nieistniejący handler nie daje 404, tylko zatrzymuje start serwera błędem `Handler not found "api::comment.comment.approve"`.

Kontroler nie ufa ciału żądania

Fundament daje roli Authenticated uprawnienie `create` na komentarzach i zostawia domyślny kontroler. Schemat komentarza ma pole `moderationStatus` z wartością domyślną `pending`. Wartość domyślna działa jednak tylko wtedy, gdy klient pola nie poda:

```
$ curl -X POST localhost:1305/api/comments -H "Authorization: Bearer eyJhbGciOi..." \
    -H 'Content-Type: application/json' \
    -d '{"data":{"body":"Buy cheap pills","moderationStatus":"approved","article":"fn3wajqrwfq0k9pifujh59er"}}'
{"data":{"id":4,"documentId":"g4uxdiiiissq016wvx5exp66","body":"Buy cheap pills",
  "moderationStatus":"approved", ...},"meta":{}}
```

Spam jest od razu na publicznej liście. To klasyczne masowe przypisanie (mass assignment): klient ustawia pole, które powinien kontrolować serwer. Z polem `author` jest podobnie. Przy uprawnieniach z seeda `"author":2` kończy się błędem 400 `Invalid key author`, bo rola nie może czytać użytkowników. Wystarczy jednak nadać jej `find` na użytkownikach

(typowe, gdy pod komentarzem chcesz pokazać autora), a Anna publikuje zatwierdzony komentarz jako Piotr.

`sanitizeInput` i `validateInput` tu nie pomogą. Pilnują schematu i uprawnień do relacji, a `moderationStatus` to zwykłe, zapisywalne pole. Strapi nie wie, że w Twojej domenie zmienia je tylko redaktor. Rozwiązanie: kontroler przyjmuje jawną listę pól, a serwis sam ustawia autora i status.

```
// src/api/comment/controllers/comment.ts (fragment)
const parseCommentInput = validateZodSchema(
  z.strictObject({
    body: z.string().trim().min(1).max(2000),
    article: z.string().min(1),
  }),
);

async create(this: CoreController, ctx) {
  const user = ctx.state.user;
  if (!user) return ctx.unauthorized();

  const input = parseCommentInput(ctx.request.body?.data);
  const comment = await strapi.service('api::comment.comment').submit({
    body: input.body, articleId: input.article, authorId: user.id,
  });

  ctx.status = 201;
  return this.transformResponse(await this.sanitizeOutput(comment, ctx));
},
```

```
// src/api/comment/services/comment.ts (fragment)
async submit({ body, articleId, authorId }: CommentSubmission) {
  const article = await strapi.documents('api::article.article').findOne({
    documentId: articleId, status: 'published',
  });
  if (!article) throw new NotFoundError('Article not found');

  return strapi.documents('api::comment.comment').create({
    data: { body, article: articleId, author: authorId, moderationStatus: 'pending' },
  });
},
```

`z.strictObject` odrzuca każdy klucz spoza listy, więc atak kończy się czytelnym 400, a poprawny komentarz zawsze trafia do kolejki:

```
{"data":null,"error":{"status":400,"name":"ValidationError",
  "message":"Unrecognized key: \"moderationStatus\"","details":{"errors":[...]}}}

{"data":{"id":4,"documentId":"x3ojuffzks86bfo2oqq8e36p","body":"Thanks, very useful",
  "moderationStatus":"pending", ...},"meta":{}}
```

Autor pochodzi z `ctx.state.user`, a nie z ciała żądania, więc nie da się go podrobić niezależnie od uprawnień roli. Serwis przy okazji pilnuje reguły, której nie wyrazisz w schemacie: komentować można tylko opublikowane artykuły. Nie da się tego zrobić przez `ctx.request.body.data.author = user.id` przed `super.create(ctx)`, bo `validateInput` w rdzeniu odrzuci klucz `author` tym samym `Invalid key author`. Alternatywą bez własnej akcji `create` jest middleware Document Service ustawiający autora (rozdział 6). Ograniczenie, kto może edytować i usuwać czyje artykuły, to polityka `is-owner` z rozdziału 4.

Ta ochrona obowiązuje tylko na trasie REST. Resolvery GraphQL i panel admina wołają Document Service z pominięciem kontrolera, więc po instalacji GraphQL ten sam atak wraca przez mutację `createComment` (rozdział 3). Regułę, która ma działać niezależnie od wejścia, przenieś do middleware Document Service (rozdział 6).

Akcja niestandardowa: zatwierdzanie komentarza

`POST /comments/:id/approve` składa się z tych samych klocków: trasa w `routes/01-custom-comment.ts`, akcja w kontrolerze, która woła serwis i kończy się na `sanitizeOutput` i `transformResponse` jak `related`, oraz reguła w serwisie:

```
// src/api/comment/services/comment.ts (fragment)
async approve(documentId: string) {
  const comment = await strapi.documents('api::comment.comment').findOne({ documentId });
  if (!comment) throw new NotFoundError('Comment not found');
  if (comment.moderationStatus !== 'pending') {
    throw new ApplicationError('Only pending comments can be approved', {
      moderationStatus: comment.moderationStatus,
    });
  }
  return strapi.documents('api::comment.comment').update({ documentId, data: { moderationStatus: 'approved' } });
},
```

Kto może zatwierdzać, decyduje uprawnienie `approve` w roli, a nie kod akcji:

```
anna (Authenticated):   403 {"name":"ForbiddenError","message":"Forbidden"}
ewa (Editor):           200 {"data":{"documentId":"x3ojuffzks86bfo2oqq8e36p","moderationStatus":"approved",...}}
ewa, drugi raz:         400 {"name":"ApplicationError","message":"Only pending comments can be approved",
                             "details":{"moderationStatus":"approved"}}
ewa, /comments/nope:    404 {"name":"NotFoundError","message":"Comment not found"}
```

`ctx` w kontrolerze

Kontroler dostaje kontekst Koa rozszerzony przez Strapi. W praktyce używasz kilku pól: `ctx.params` (parametry ścieżki; `id` to `documentId`), `ctx.query` (query sparsowane do obiektu z zagnieżdżonymi `filters` i `populate`), `ctx.request.body` (ciało, w Content API w kopercie `data`), `ctx.state.user` (zalogowany użytkownik users-permissions z rolą), `ctx.state.auth` (strategia uwierzytelnienia, którą przekazujesz do sanityzacji) i `ctx.state.route` (konfiguracja trasy). Wartość zwrócona z akcji staje się ciałem odpowiedzi, status ustawiasz przez `ctx.status`. Skróty `ctx.badRequest()`, `ctx.notFound()` i pokrewne (nazwy z http-errors w camelCase) przyjmują komunikat i `details`. Jest też `strapi.requestContext.get()` dostępne z serwisu, ale to wyjście awaryjne: serwis zależny od żądania HTTP nie zadziała w cronie ani w skrypcie.

Błędy i ich format

Każdy błąd Content API ma ten sam kształt:

```
{ "data": null, "error": { "status": 404, "name": "NotFoundError", "message": "Comment not found", "details": {} } }
```

Strapi buduje go z klas w `errors` z `@strapi/utils` (`import { errors } from '@strapi/utils'`). W serwisach i lifecycle'ach rzucasz klasy, w kontrolerach i middleware możesz też użyć skrótów `ctx.*`. Tak wyglądały odpowiedzi z tymczasowej trasy testowej (Strapi 5.56, październik 2026):

Źródło	Status	`name`	Twój `message` w odpowiedzi
`ApplicationError`, `ValidationError`	400	jak klasa	tak, z `details`
`ctx.badRequest('...', details)`	400	`BadRequestError`	tak
`NotFoundError`, `ctx.notFound('...')`	404	`NotFoundError`	tak
`PolicyError`	403	`PolicyError`	tak
`ForbiddenError`	403	`ForbiddenError`	nie, zawsze `Forbidden`
`UnauthorizedError`	401	`UnauthorizedError`	nie, zawsze `Unauthorized`
zwykły `throw new Error(...)`	500	`InternalServerError`	nie; pełny komunikat i stos tylko w logu serwera

Ostatni wiersz oznacza, że przypadkowy wyjątek z treścią zapytania SQL nie trafi do klienta, ale frontend dostaje bezużyteczne 500. Każdy przewidywalny przypadek (brak rekordu, zły stan, niepoprawne dane) zamieniaj więc na klasę z `errors`.

> **Pułapka:** `throw new errors.ForbiddenError('You are not the author')` zwraca klientowi samo `Forbidden`. Dokumentacja wspomina o tym w notatce przy `ForbiddenError`, ale to samo dotyczy `UnauthorizedError`, o czym już nie pisze. Gdy frontend ma pokazać powód odmowy, użyj `PolicyError` (w polityce) albo `ApplicationError` z kodem w `details`.

Walidacja wejścia: schemat, yup czy zod

Walidacja w Strapi ma trzy piętra. Schemat (`required`, `maxLength`, `enum`) sprawdza Document Service przy każdym zapisie, także z panelu: za długi tytuł daje 400 `title must be at most 160 characters`. Kontroler rdzenia sprawdza strukturę: nieznane pole w `data` daje 400 `Invalid key views`. Trzecie piętro to reguły, których schemat nie wyrazi – które pola wolno podać na danym endpoincie, format, zależności między polami. To piszesz sam, w kontrolerze.

`@strapi/utils` reeksportuje oba popularne narzędzia: `yup` z `validateYupSchema` oraz `z` (Zod 4) z `validateZodSchema`. Oba helpery zamieniają błąd biblioteki na `ValidationError` z listą `details.errors`, więc format odpowiedzi jest identyczny jak dla błędów rdzenia. W nowym kodzie wybierz Zod: daje typ wyniku bez dodatkowej deklaracji (`input.body` jest `string`), a Strapi 5 sam używa go do parametrów Content API. Yup ma sens, gdy rozszerzasz kod pluginu, który już go używa. Importuj `z` z `@strapi/utils`, a nie z osobno zainstalowanego `zod`, żeby nie mieć w projekcie dwóch wersji biblioteki.

> **Pułapka:** strona `strapi-utils` w dokumentacji opisuje helper `validateZod`. W Strapi 5.56 taki eksport nie istnieje; funkcja nazywa się `validateZodSchema` (sprawdzone w `node_modules/@strapi/utils/dist/zod.d.ts`). Import według dokumentacji kończy się błędem kompilacji.

Podsumowanie

- Core controller w każdej akcji wykonuje `validateQuery`, `sanitizeQuery`, `validateInput`, `sanitizeInput` i `sanitizeOutput`. Rozszerzając go przez `super`, zachowujesz tę ochronę; pisząc akcję od zera, odtwarzasz ją sam.
- Document Service zwraca dane z uprawnieniami serwera, łącznie z polami prywatnymi. Bez `sanitizeOutput` anonimowy klient widzi e-mail i hash hasła autora.
- Kontroler tłumaczy HTTP na wywołanie serwisu; reguły biznesowe i rzucanie błędów domenowych należą do serwisu, który nie dostaje `ctx`.
- Nowa akcja wymaga trasy i nadania uprawnienia roli; trasa wskazująca na nieistniejący handler blokuje start serwera.
- Sanityzacja wejścia nie chroni pól biznesowych. Pola takie jak `moderationStatus` czy `author` ustawia serwer, a kontroler przyjmuje jawną listę pól (`z.strictObject`).
- Błędy rzucaj klasami z `errors` z `@strapi/utils`; `ForbiddenError` i `UnauthorizedError` ukrywają Twój komunikat, a zwykły `Error` daje 500.
- W TypeScripcie oznacz akcje używające `this.*` typem `this: Core.CoreAPI.Controller.CollectionType`.

Co dalej

Kontroler działa tylko dla żądań, które przyszły przez REST. Gdy redaktor zmieni komentarz w panelu admina albo skrypt zapisze artykuł przez `strapi.documents()`, żaden z pokazanych tu kodów się nie wykona. Rozdział 6 schodzi piętro niżej, do Document Service i Query Engine, i pokazuje, jak reagować na zmiany danych niezależnie od tego, skąd przyszły: od wyliczania `readingTime` po ustawianie autora z kontekstu.

Źródła

- https://docs.strapi.io/cms/backend-customization/controllers
- https://docs.strapi.io/cms/backend-customization/services
- https://docs.strapi.io/cms/backend-customization/requests-responses
- https://docs.strapi.io/cms/error-handling
- https://docs.strapi.io/cms/strapi-utils
- https://docs.strapi.io/cms/migration/v4-to-v5/breaking-changes/default-input-validation
- https://docs.strapi.io/cms/configurations/api

Rozdział 6. Document Service, Query Engine i reagowanie na zmiany

> **Cel rozdziału:** nauczysz się czytać i zapisywać treść przez Document Service, świadomie schodzić do Query Engine i umieszczać reguły „przy każdym zapisie" w middleware Document Service zamiast w lifecycle hookach.
>
> **Kod:** `examples/06-document-service/`. **Pakiety poza fundamentem:** brak.

Redakcja ma kilka reguł, które muszą działać zawsze, niezależnie od tego, kto zapisuje artykuł. Czas czytania (`readingTime`) ma się liczyć sam z treści. Slug ma powstać z tytułu, jeśli autor go nie poda. Kategorii, do której należą artykuły, nie wolno usunąć, bo artykuły straciłyby przypisanie. A artykuł utworzony przez zalogowaną autorkę ma mieć ją jako autora – bez względu na to, co przyśle klient.

Kontroler z rozdziału 5 albo middleware trasy z rozdziału 4 obsłużą te reguły tylko dla REST-a, a redaktor pracuje w panelu, skrypt importu zaś woła serwisy bezpośrednio. Potrzebujesz warstwy, przez którą przechodzi każdy zapis treści – w Strapi 5 to Document Service.

Document Service: pracujesz na dokumentach, nie na wierszach

Dokument to jeden artykuł z punktu widzenia redakcji, identyfikowany stabilnym `documentId`. W bazie ten jeden artykuł to kilka wierszy: wersja robocza (draft), wersja opublikowana, a przy włączonym i18n – po parze na każdy język. Każdy wiersz ma własne numeryczne `id`. Document Service (`strapi.documents(uid)`) ukrywa tę różnicę: pytasz o dokument, podajesz `status` i `locale`, a serwis sam wybiera właściwe wiersze, synchronizuje relacje między wersjami i odpala zdarzenia dla webhooków.

Metody dzielą się na dwie grupy. Odczyt: `findOne`, `findFirst`, `findMany`, `count`. Zapis: `create`, `update`, `delete` oraz – tylko dla typów z draft & publish – `publish`, `unpublish` i `discardDraft`. Parametry wyglądają znajomo, jeśli znasz REST z rozdziału 3, ale są obiektami JavaScript, a nie query stringiem. Lista najnowszych opublikowanych artykułów z kategorii Backend:

```
const latest = await strapi.documents('api::article.article').findMany({
  status: 'published',
  filters: { category: { slug: 'backend' } },
  fields: ['title', 'slug', 'publishedAt'],
  populate: { category: { fields: ['name'] }, author: { fields: ['username'] } },
  sort: 'publishedAt:desc',
  limit: 2,
});
```

```
[{ "publishedAt": "2026-10-03T18:07:59.298Z", "id": 4, "documentId": "j9i5dhs1wiefz0bco65zsw26",
   "title": "Document Service middlewares", "slug": "document-service-middlewares",
   "category": { "id": 1, "documentId": "wfyue085i2lzb8vin2zk3mju", "name": "Backend" },
   "author": { "id": 1, "documentId": "p4k39xc7ey2k9puf6xy0ynaq", "username": "anna" } }]
```

Wynik jest płaski, jak w REST v5, ale **nieprzefiltrowany przez uprawnienia**. Document Service to warstwa dostępu do danych: nie wie nic o rolach. `populate: ['author']` bez `fields` zwrócił mi dla Anny pola `password` i `resetPasswordToken`. Gdy zwracasz taki wynik z własnego kontrolera, musisz go zsanityzować – szczegóły w rozdziale 5.

Domyślny `status` to `draft` (Strapi 5.56, październik 2026), więc bez niego czytasz wersje robocze – także dokumentów opublikowanych, bo każdy ma też draft. Dlatego `count({ status: 'draft' })` zwróciło w moim projekcie 13, a `count({ status: 'published' })` – 9. `status` działa tylko w korzeniu zapytania i obowiązuje też dla relacji z `populate`; próba `populate: { category: { status: 'draft' } }` kończy się błędem `Invalid key status at category`. Domyślnie nie jest populowane nic: ani relacje, ani media, komponenty czy dynamic zones.

Przy zapisie `create()` tworzy draft, `publish({ documentId })` kopiuje go do nowej wersji opublikowanej, `unpublish()` ją usuwa, a `discardDraft()` nadpisuje draft treścią opublikowaną. `status: 'published'` w `create()` lub `update()` zapisuje i od razu publikuje. Wersja opublikowana jest tylko do odczytu – edytujesz zawsze draft. Draft & publish i i18n (parametr `locale`) rozwija rozdział 8.

> **Pułapka:** `strictParams: true` w sekcji `documents` pliku `config/api.ts` fundamentu nie odrzuca nieznanych kluczy, wbrew opisowi w dokumentacji. Kod w `node_modules/@strapi/core` po prostu je wycina. Kod przeniesiony ze Strapi 4 z `publicationState: 'live'` przejdzie bez błędu i zwróci… same drafty (u mnie 13 z 13). Błędem kończy się tylko nieprawidłowa wartość znanego parametru, np. `status: 'live'`: `Invalid parameter at 'status'. Expected 'published' or 'draft'`. Po migracji z v4 przeszukaj kod pod kątem `publicationState` i `entityService` – Entity Service API jest w Strapi 5 przestarzałe i zastąpione przez Document Service.

Query Engine: piętro niżej i czego tam nie ma

Pod Document Service leży Query Engine (`strapi.db.query(uid)`). Operuje na wierszach, nie na dokumentach: używa `where` zamiast `filters`, `select` zamiast `fields`, `orderBy`, `offset` i `limit`, i nie wie nic o draft & publish ani o i18n. Różnicę najlepiej widać na jednym opublikowanym artykule z seeda:

```
// strapi.documents(...).findMany({ filters: { slug: ... } }) returns 1 document
await strapi.db.query('api::article.article').findMany({
  where: { slug: 'policies-in-strapi-5' },
  select: ['id', 'documentId', 'publishedAt'],
});
// → [{ id: 1, documentId: 'r8ef...', publishedAt: null },
//    { id: 2, documentId: 'r8ef...', publishedAt: '2026-10-03T18:07:59.255Z' }]
```

Query Engine zwrócił oba wiersze, a jego `count()` policzył w tej samej bazie 20 wierszy artykułów wobec 11 dokumentów. Gdybyś zaktualizował nim tylko wiersz opublikowany, następna publikacja z panelu nadpisałaby zmianę treścią draftu.

Po co więc schodzić niżej? Przede wszystkim po operacje masowe: `createMany`, `updateMany`, `deleteMany`, których Document Service nie ma. Przykład z Redakcji: zadanie porządkowe odrzuca komentarze, które za długo czekają na moderację. Komentarz nie ma draft & publish ani tłumaczeń, więc jeden wiersz to jeden dokument i Query Engine jest tu bezpieczny:

```
const { count } = await strapi.db.query('api::comment.comment').updateMany({
  where: { moderationStatus: 'pending', createdAt: { $lt: cutoff } },
  data: { moderationStatus: 'rejected' },
});
// → { count: 2 }
```

Jedno zapytanie `UPDATE` zamiast pętli po dokumentach. Cena: żaden middleware Document Service się nie odpali, nie pójdą zdarzenia `entry.update` do webhooków, a z lifecycle hooków zadziała tylko `beforeUpdateMany`/`afterUpdateMany`, nie `beforeUpdate`. Nie dostaniesz też walidacji ze schematu, którą Document Service robi przy każdym zapisie: Query Engine bez mrugnięcia zapisał mi komentarz z 2500 znakami (limit to 2000) i `moderationStatus: 'nonsense'` spoza enumeracji. Schodź więc niżej dla operacji masowych na typach bez draft & publish i i18n oraz dla tabel technicznych (seed tak nadaje uprawnienia rolom). Jeszcze niżej jest knex (`strapi.db.connection`), gdzie nie działają nawet lifecycle hooki.

Transakcje: wszystko albo nic

Import z zewnętrznego systemu tworzy kategorię i jej pierwsze artykuły. Bez transakcji błąd walidacji drugiego artykułu (`title must be at most 160 characters`) zostawił mi w bazie osieroconą kategorię. Rozwiązaniem jest `strapi.db.transaction()` – dokumentacja oznacza ją jako funkcję eksperymentalną, ale to ten sam mechanizm, którego Strapi używa wewnętrznie.

```
export default factories.createCoreService('api::category.category', ({ strapi }) => ({
  // Imports a category with its first articles: all of them land in the database or none does.
  async importWithArticles(name: string, titles: string[]) {
    return strapi.db.transaction(async ({ onCommit }) => {
      const category = await strapi.documents('api::category.category').create({
        data: { name, slug: name.toLowerCase() },
      });
      for (const title of titles) {
        await strapi.documents('api::article.article').create({
          data: { title, slug: slugify(title), category: category.documentId },
        });
      }
      onCommit(() => strapi.log.info(`Imported ${name} with ${titles.length} article(s)`));
      return category;
    });
  },
}));
```

```
before: 3
import failed: ValidationError title must be at most 160 characters
after failed import: 3 0
[2026-10-03 20:16:11.964] info: Imported Security with 2 article(s)
after good import: 4
```

Nie przekazujesz obiektu transakcji do `strapi.documents()` ani `strapi.db.query()`. Strapi trzyma go w `AsyncLocalStorage` i każde wywołanie wewnątrz funkcji automatycznie go używa; jawne `.transacting(trx)` jest potrzebne tylko w surowym knexie. Wyjątek cofa wszystko, a `onCommit` uruchamia efekt uboczny dopiero po zatwierdzeniu. Zagnieżdżone `transaction()` dołącza do zewnętrznej.

`slug: slugify(title)` jest tu mimo middleware generującego slug, bo typy ze schematu wymagają `slug` i `tsc` nie przepuści `create()` bez niego. Transakcji nie dodawaj „na wszelki wypadek”: każda metoda Document Service i tak działa we własnej, a długa transakcja trzyma blokady.

Lifecycle hooks czy middleware Document Service

Strapi daje dwa miejsca, w których możesz zareagować na zapis. Lifecycle hooki (`beforeCreate`, `afterUpdate` itd., w pliku `src/api/article/content-types/article/lifecycles.ts` albo przez `strapi.db.lifecycles.subscribe()`) są częścią warstwy bazy: odpalają się dla każdego **wiersza**, który Query Engine tworzy, zmienia lub usuwa. Middleware Document Service, rejestrowany przez `strapi.documents.use()` w `register()`, owija wywołanie **metody** Document Service: widzi akcję (`create`, `publish` …), uid typu i parametry, zanim cokolwiek trafi do bazy, a po `await next()` – wynik.

W Strapi 5 jedna operacja na dokumencie to kilka operacji na wierszach. Zamiast wierzyć tabeli z dokumentacji, zmierzyłem to.

Pomiar: ile razy co się odpala

Do projektu dodałem dwa liczniki, oba włączane zmienną `TRACE_ARTICLE_EVENTS=true`. Plik `lifecycles.ts` artykułu podpina jedną funkcję `trace(event: Event)` pod `beforeCreate`, `afterCreate`, `beforeUpdate`, `afterUpdate`, `beforeDelete` i `afterDelete`; loguje akcję, `id` wiersza, wersję (draft czy published) i wynik `strapi.db.inTransaction()`. Middleware Document Service (`src/debug/trace-documents.ts`) loguje każde wywołanie na artykułach przed i po `next()`. Pierwszy test: Anna tworzy artykuł przez REST, `POST /api/articles`. Kontroler REST domyślnie tworzy dokument od razu opublikowany:

```
[document]  create status=published trx=false
[lifecycle] beforeCreate id=- version=draft trx=true
[lifecycle] afterCreate  id=8 version=draft trx=true
[lifecycle] beforeCreate id=- version=published trx=true
[lifecycle] afterCreate  id=9 version=published trx=true
[document]  create done trx=false
http: POST /api/articles (61 ms) 201
```

Jedno wywołanie Document Service, dwa wiersze, cztery lifecycle hooki. Kolejne testy dały taki obraz (pomijam hooki `find*`, które publikacja też odpala, bo czyta draft z bazy):

Operacja	Middleware DS	Lifecycle hooki (zapis)
REST `POST /api/articles` (create + publish)	1 × `create`	2 × Create (draft, published)
REST `POST ?status=draft`	1 × `create`	1 × Create
pierwsza publikacja: REST `PUT`	1 × `update`	1 × Update (draft) + 1 × Create (published)
ponowna publikacja: REST `PUT`	1 × `update`	1 × Update + 1 × Delete (stara wersja) + 1 × Create
panel: „Publish”	`findOne` ×2, `update`, `publish`	1 × Update + 1 × Create

(każde „×1” w ostatniej kolumnie to para `before…` i `after…`). `readingTime` liczony w `beforeCreate` policzy się drugi raz dla kopii, a `afterCreate` wysyłający e-mail „nowy artykuł” wyśle go przy każdej publikacji. Wersja opublikowana dostaje też za każdym razem nowe `id` (u mnie 11, potem 12). Middleware widzi jedną intencję: „utwórz”, „zaktualizuj”, „opublikuj”.

Middleware Document Service łapie też panel admina

Middleware trasy z rozdziału 4 działa tylko dla Content API; panel ma własne trasy pluginu Content Manager. Sprawdziłem, czy middleware DS je widzi, na dwa sposoby. W `node_modules/@strapi/content-manager/dist/server/services/document-manager.js` każda akcja panelu (create, update, publish, unpublish, discard, delete, clone) woła `strapi.documents(uid)`. Potem zarejestrowałem pierwszego administratora przez `POST /admin/register-admin`, dostałem JWT admina i z curl-a utworzyłem oraz opublikowałem artykuł przez Admin API panelu (`/content-manager/collection-types/api::article.article`):

```
# after… and "done" lines omitted
[document]  create status=draft trx=false
http: POST /content-manager/collection-types/api::article.article (55 ms) 201
[document]  findOne status=- trx=true
[document]  findOne status=draft trx=true
[document]  update status=draft trx=true
[lifecycle] beforeUpdate id=10 version=draft trx=true
[document]  publish status=- trx=true
[lifecycle] beforeCreate id=- version=published trx=true
http: POST /content-manager/.../gvak.../actions/publish (71 ms) 200
```

Przycisk „Publish” w panelu to w rzeczywistości zapis draftu (`update`) i dopiero potem `publish` – bo formularz wysyła aktualną treść razem z prośbą o publikację. Middleware zobaczył obie operacje, więc reguły z następnej sekcji działają też dla redaktora w panelu. To jest główny argument za middleware Document Service: jedno miejsce dla REST-a, panelu, skryptów i pluginów.

```
Content API (REST) → middleware → polityki → kontroler → serwis ─┐
Panel (Content Manager) → trasy admina → document-manager ──────┤
Skrypt, cron, plugin ────────────────────────────────────────────┤
                                                                 ▼
                    strapi.documents(uid)  ← middleware DS (strapi.documents.use)
                                  │
                    Query Engine (strapi.db.query)  ← lifecycle hooki (per wiersz)
                                  │
                              baza
```

Lifecycle hooki mają jedną przewagę: łapią też zapisy przez Query Engine. Używaj ich do reakcji na zmianę wiersza (np. audyt techniczny tabeli); regułę biznesową o dokumencie umieszczaj w middleware Document Service. Pełne porównanie z politykami i middleware tras jest w tabeli w rozdziale 4.

Trzy reguły Redakcji jako middleware

Middleware dostaje `(context, next)`: `context.uid`, `context.action` i `context.params`, które wolno zmieniać przed `next()`. Jest globalny – odpala się dla każdego typu i akcji – więc zaczynasz od warunku i dla reszty od razu zwracasz `next()`. Dokumentacja podkreśla, że wynik `next()` trzeba zwrócić – zgubiony `return` psuje całą aplikację, bo każda metoda zwróci `undefined`. Rejestracja w `src/index.ts`:

```
register({ strapi }: { strapi: Core.Strapi }) {
  registerDocumentTrace(strapi);
  strapi.documents.use(articleAuthor(strapi));
  strapi.documents.use(articleDerivedFields(strapi));
  strapi.documents.use(categoryDeleteGuard(strapi));
},
```

Middleware wykonują się w kolejności rejestracji. `register()` gwarantuje, że są na miejscu, zanim jakikolwiek `bootstrap()` zapisze pierwszy dokument.

Czas czytania i slug

```
export const articleDerivedFields = (strapi: Core.Strapi): Modules.Documents.Middleware.Middleware => {
  return async (context, next) => {
    if (context.uid !== 'api::article.article') return next();
    if (context.action !== 'create' && context.action !== 'update') return next();

    const data = context.params.data as Record<string, unknown> | undefined;
    if (data) {
      if (Array.isArray(data.content)) {
        data.readingTime = readingTimeOf(data.content as ContentBlock[]);
      }
      if (context.action === 'create' && !data.slug && typeof data.title === 'string') {
        data.slug = await uniqueSlug(strapi, slugify(data.title));
      }
    }
    return next();
  };
};
```

`readingTimeOf` przechodzi po dynamic zone, zlicza słowa w blokach `shared.rich-text` i cytatach i dzieli przez 200 słów na minutę; `uniqueSlug` dokleja `-2`, `-3`, jeśli slug jest zajęty (pełny plik w `examples/`). Czas czytania liczy się tylko wtedy, gdy zapis zawiera `content`, więc aktualizacja samego `excerpt` nie zeruje go. Dwa identyczne żądania REST bez sluga, z tytułem „Łódź meetup: Strapi i Postgres" i treścią o 450 słowach, dostały `lodz-meetup-strapi-i-postgres` i `lodz-meetup-strapi-i-postgres-2`, oba z `readingTime: 3`. Artykuł utworzony w panelu bez sluga dostał `written-in-the-admin-panel`. Slug powstaje tylko przy `create`: zmiana tytułu po publikacji nie zmienia adresu, który ktoś już udostępnił.

Kategoria z artykułami jest nieusuwalna

```
export const categoryDeleteGuard = (strapi: Core.Strapi): Modules.Documents.Middleware.Middleware => {
  return async (context, next) => {
    if (context.uid !== 'api::category.category' || context.action !== 'delete') return next();

    const { documentId } = context.params;
    const articles = await strapi.documents('api::article.article').count({
      filters: { category: { documentId } },
    });
    if (articles > 0) {
      throw new errors.ApplicationError(
        `Category has ${articles} article(s). Move them to another category first.`,
        { documentId, articles }
      );
    }
    return next();
  };
};
```

Rzucony błąd z `@strapi/utils` przerywa łańcuch przed `next()`, więc nic nie zostaje usunięte. `DELETE` kategorii Backend przez Admin API panelu (rola Authenticated nie ma prawa usuwać kategorii) zwrócił 400 z `"name": "ApplicationError"`, komunikatem `Category has 2 article(s)...` i `details: { documentId, articles: 2 }`. Pusta kategoria usuwa się normalnie (200). Masowe usuwanie w Content Managerze też woła `delete` dla każdego dokumentu (w jednej transakcji), więc blokada działa i tam. Polityka z rozdziału 4 nie dałaby tego efektu: chroniłaby tylko trasę `DELETE /api/categories/:id`.

Autor z kontekstu żądania

Middleware Document Service nie dostaje `ctx` Koa, ale Strapi trzyma bieżące żądanie w `strapi.requestContext` (to również `AsyncLocalStorage`). Stamtąd bierzesz zalogowanego użytkownika:

```
type AuthState = { auth?: { strategy?: { name?: string } }; user?: { id: number } };

export const articleAuthor = (strapi: Core.Strapi): Modules.Documents.Middleware.Middleware => {
  return async (context, next) => {
    if (context.uid !== 'api::article.article' || context.action !== 'create') return next();

    // Only Content API users are article authors; admin panel users are a different table.
    const state = strapi.requestContext.get()?.state as AuthState | undefined;
    if (state?.auth?.strategy?.name === 'users-permissions' && state.user) {
      const data = context.params.data as Record<string, unknown>;
      data.author = state.user.id;
    }
    return next();
  };
};
```

Sprawdzenie strategii jest kluczowe. W panelu `ctx.state.user` to administrator z tabeli `admin_users`, a relacja `author` wskazuje na `plugin::users-permissions.user`; przypisanie jego `id` podpięłoby artykuł do przypadkowego użytkownika aplikacji o tym samym numerze. W skrypcie albo cronie `requestContext.get()` zwraca `undefined`. Po utworzeniu artykułu przez Annę oba wiersze mają w `articles_author_lnk` jej identyfikator. Przysłanie `"author": 2` w body REST i tak kończy się błędem `Invalid key author` (Authenticated nie ma uprawnień do typu użytkownika), więc dla REST-a middleware to druga linia obrony. Ustawienie autora w kontrolerze (rozdział 5) zadziała tylko dla jednej trasy.

Pułapki: rekurencja i transakcja, w której siedzi hook

Najczęstszy błąd to zapis przez Document Service do tego samego typu i akcji, którą middleware obsługuje – np. „dopisanie” `readingTime` osobnym `update()`. Uruchomiłem taki kod w `npm run console`, z bezpiecznikiem po pięciu wywołaniach:

```
strapi.documents.use(async (context, next) => {
  if (context.uid === 'api::article.article' && context.action === 'update') {
    console.log(`update middleware, call #${++calls}`);
    if (calls > 5) throw new Error('safety stop after 5 nested updates');
    // wrong: writing through the Document Service re-enters this middleware
    await strapi.documents(context.uid).update({ documentId: context.params.documentId, data: { readingTime: 5 } });
  }
  return next();
});
```

```
update middleware, call #1
update middleware, call #2
...
update middleware, call #6
error: safety stop after 5 nested updates
```

> **Pułapka:** zapis z wnętrza middleware do tej samej akcji tego samego typu wpada w nieskończoną rekurencję – bez bezpiecznika żądanie wisi, aż skończy się stos. Zmieniaj `context.params.data` przed `next()` zamiast robić drugi zapis. Odczyty (`count`, `findFirst`) i zapisy do innych typów są bezpieczne, o ile ich middleware nie wraca do pierwszego typu. To samo dotyczy lifecycle hooków: `afterUpdate`, który woła `strapi.documents().update()` na tym samym artykule, odpala `afterUpdate` ponownie.

Druga rzecz widoczna w logach to kolumna `trx`. Lifecycle hooki zawsze mają `trx=true`: działają wewnątrz transakcji, którą Document Service otwiera dla każdej metody. Middleware przed `next()` i po nim, przy wywołaniu z REST, ma `trx=false`. Panel jest inny: akcja „Publish” opakowuje `update` i `publish` we własną transakcję, więc tam middleware też ma `trx=true`.

> **Pułapka:** `afterCreate` wywołuje się przed zatwierdzeniem transakcji. Wysłany stamtąd e-mail czy żądanie do frontendu może dotyczyć danych, których jeszcze nie ma, a wyjątek w hooku cofa cały zapis – sprawdziłem: rzucenie błędu w `afterCreate` wersji opublikowanej nie zostawiło w bazie nawet draftu. Efekty uboczne przenieś do middleware Document Service za `await next()` albo zarejestruj je przez `onCommit` we własnej transakcji. Do odświeżania cache po publikacji służą też webhooki `entry.publish` (rozdziały 8 i 11).

Podsumowanie

- Document Service (`strapi.documents(uid)`) pracuje na dokumentach z `documentId`; `status` (domyślnie `draft`) i `locale` wybierają wersję, a nic nie jest populowane domyślnie.
- Wynik Document Service nie jest sanityzowany – z własnego kontrolera zwracaj go dopiero po sanityzacji z rozdziału 5.
- Query Engine (`strapi.db.query`) widzi wiersze: draft i wersję opublikowaną osobno, bez middleware DS, webhooków i walidacji. Używaj go do operacji masowych na typach bez draft & publish i do tabel technicznych.
- `strapi.db.transaction()` spina kilka zapisów w całość bez przekazywania obiektu transakcji; efekty uboczne odkładaj do `onCommit`.
- Lifecycle hooki odpalają się per wiersz: utworzenie opublikowanego artykułu to cztery hooki, ponowna publikacja – sześć, a wersja opublikowana za każdym razem dostaje nowe `id`.
- Middleware Document Service (`strapi.documents.use()` w `register()`) odpala się raz na operację i łapie REST, panel admina i skrypty.
- W middleware zmieniaj `context.params` przed `next()`, zawsze zwracaj wynik `next()` i nie zapisuj do tej samej akcji tego samego typu.
- `publicationState` i `entityService` to relikty Strapi 4; to pierwsze jest po cichu ignorowane.

Co dalej

Masz już komplet miejsc na logikę po stronie serwera: trasy, polityki i middleware z rozdziału 4, kontrolery i serwisy z rozdziału 5 oraz Document Service z tego rozdziału. W rozdziale 7 zajmiesz się tym, kim jest `state.user`, z którego korzystał middleware autora: użytkownikami aplikacji, tokenami z odświeżaniem i uprawnieniami ról, a także tym, jak filtrować listy po właścicielu – również w middleware Document Service.

Źródła

- https://docs.strapi.io/cms/api/document-service
- https://docs.strapi.io/cms/api/document-service/status
- https://docs.strapi.io/cms/api/document-service/populate
- https://docs.strapi.io/cms/api/document-service/sort-pagination
- https://docs.strapi.io/cms/api/document-service/middlewares
- https://docs.strapi.io/cms/api/query-engine
- https://docs.strapi.io/cms/api/query-engine/bulk-operations
- https://docs.strapi.io/cms/database-transactions
- https://docs.strapi.io/cms/backend-customization/models (sekcja „Lifecycle hooks”)
- https://docs.strapi.io/cms/migration/v4-to-v5/breaking-changes/lifecycle-hooks-document-service
- https://docs.strapi.io/cms/configurations/api

Rozdział 7. Użytkownicy, tokeny i uprawnienia

> **Cel rozdziału:** rozumieć, kto i czym uwierzytelnia się w Strapi (sesje users-permissions, API tokeny, konto panelu), umieć ustawić uprawnienia kodem i ograniczyć dane do tych, które należą do zalogowanego użytkownika.
>
> **Kod:** `examples/07-uwierzytelnianie/`. **Pakiety poza fundamentem:** brak.

„Redakcja” ma trzy rodzaje gości. Anna loguje się na stronie magazynu, żeby zobaczyć swoje teksty, także szkice. Serwer Next.js potrzebuje dostępu do API bez niczyjego hasła. Redaktorka moderuje komentarze w panelu admina. Każdy ma inny rodzaj tożsamości, inny token i inne miejsce, w którym ustawiasz uprawnienia.

Fundament ma też dziurę, którą widać jednym poleceniem. Rola Public ma uprawnienie `find` na artykułach, a to wystarczy, żeby anonimowo pobrać szkice:

```
$ curl -g 'localhost:1307/api/articles?status=draft&fields[0]=title&fields[1]=publishedAt'
{"data":[ ... {"title":"Self-hosting Strapi with Docker","publishedAt":null}],
 "meta":{"pagination":{"total":4}}}
```

Artykuł Piotra, który nigdy nie został opublikowany, jest publicznie dostępny. Na końcu rozdziału go zamkniesz. Wcześniej poznasz sesje z refresh tokenami, które są domyślne w nowym projekcie, uprawnienia ustawiane kodem i rozszerzanie pluginu users-permissions.

Dwa światy: panel i aplikacja

Administratorzy panelu i użytkownicy aplikacji żyją w osobnych tabelach (`admin_users` i `up_users`), logują się na osobnych trasach i dostają tokeny, których druga strona nie akceptuje. JWT Anny wysłany do `/admin/users/me` kończy się 401, tak samo jak token administratora wysłany do `/api/users/me`.

Uprawnieniami w panelu rządzi RBAC (Role-Based Access Control), dostępny w wersji darmowej. Domyślne role to Super Admin, Editor i Author. Możesz dodać własne, także przez Admin API, z którego korzysta formularz ról. Rola Moderator dostała w ten sposób (`PUT /admin/roles/:id/permissions`) odczyt komentarzy i `explorer.update` z `properties.fields: ["moderationStatus"]`, czyli edycję jednego pola. Nie myl roli Author z polem `author` artykułu: ona ogranicza administratora do wpisów, które sam utworzył w panelu. SSO dla panelu wymaga płatnej licencji. Tokeny dla Admin API (admin tokens, np. dla serwera MCP) omawia rozdział 12.

Logowanie w trybie sesji

Plugin users-permissions ma dwa tryby wydawania tokenów (opcja `jwtManagement`). Dokumentacja i kod pluginu podają jako domyślny `legacy-support`, ale projekt utworzony przez `create-strapi-app` ma w `config/plugins.ts` tryb `refresh` z `sessions.httpOnly: true` – tak jak fundament „Redakcji”. Zobacz, co zwraca logowanie:

```
$ curl -i -X POST localhost:1307/api/auth/local -H 'Content-Type: application/json' \
    -d '{"identifier":"anna","password":"Passw0rd!"}'
HTTP/1.1 200 OK
X-RateLimit-Limit: 10
Set-Cookie: strapi_up_refresh=eyJhbGciOi...; path=/; samesite=lax; httponly
Set-Cookie: strapi_up_refresh.sig=INHz5o...; path=/; samesite=lax; httponly
{"jwt":"eyJhbGciOi...","user":{"id":1,"documentId":"afvh6pf0...","username":"anna",...}}
```

W ciele są tylko `jwt` i `user`. `jwt` to krótko żyjący token dostępu (access token): jego payload to `{ userId: '1', sessionId: '1e8c61f3...', type: 'access' }`, a różnica `exp - iat` wynosi 600 sekund. Token odświeżania (refresh token) trafił do ciasteczka `httpOnly`, którego JavaScript w przeglądarce nie odczyta.

Domyślne wartości (Strapi 5.56, październik 2026): token dostępu 10 minut (`accessTokenLifespan: 600`), token odświeżania wygasa po 14 dniach bez użycia (`idleRefreshTokenLifespan`) i najpóźniej po 30 dniach od zalogowania (`maxRefreshTokenLifespan`). Ciasteczko jest sesyjne (bez `maxAge`), `SameSite=Lax` i dostaje flagę `Secure` automatycznie przy `NODE_ENV=production`. Logowanie ma limit 10 żądań na minutę (`X-RateLimit-Limit: 10`), choć strona REST API dokumentacji podaje „5 na 5 minut”.

Odświeżanie i rotacja

Gdy token dostępu wygaśnie, klient wywołuje `POST /api/auth/refresh`. Strapi szuka tokenu odświeżania najpierw w ciasteczku, potem w polu `refreshToken` ciała. Każde odświeżenie wydaje nowy token odświeżania (rotacja), a stary oznacza jako `rotated`:

```
POST /api/auth/refresh  (bez ciasteczka)       → 400 "Missing refresh token"
POST /api/auth/refresh  (ciasteczko z logowania) → 200 {"jwt":"eyJhbGciOi..."} + nowy Set-Cookie
POST /api/auth/refresh  (to samo stare ciasteczko) → 401 "Invalid refresh token"
```

Ponowne użycie starego tokenu nie unieważnia całej rodziny – nowe ciasteczko działa dalej. W kliencie ma to jedną konsekwencję: dwa równoległe odświeżenia z tym samym ciasteczkiem dają `200` i `401`, a drugi wynik wygląda jak wylogowanie.

```ts
// client/auth-fetch.ts (fragment)
let pendingRefresh: Promise<boolean> | null = null;

// Single flight: a rotated refresh token is invalid, so two parallel refreshes would log the user out.
function refresh(): Promise<boolean> {
  pendingRefresh ??= (async () => {
    const res = await fetch(`${API_URL}/auth/refresh`, { method: 'POST', credentials: 'include' });
    accessToken = res.ok ? ((await res.json()) as { jwt: string }).jwt : null;
    return res.ok;
  })().finally(() => {
    pendingRefresh = null;
  });
  return pendingRefresh;
}
```

Token dostępu trzymasz w pamięci, a żądanie z odpowiedzią 401 powtarzasz raz po odświeżeniu (pełny plik w `examples/`). `credentials: 'include'` wysyła ciasteczko na inny origin, co wymaga też poprawnego `strapi::cors` (rozdział 11, razem z wersją dla Next.js). Przykład z dokumentacji („Authentication flow with JWT") zapisuje `refreshToken` w `localStorage` – dokładnie tam, skąd ciasteczko `httpOnly` miało go zabrać.

Wylogowanie i to, czego ono nie cofa

`POST /api/auth/logout` z tokenem dostępu usuwa bieżącą sesję i czyści ciasteczko. Body `{"scope":"all"}` wylogowuje ze wszystkich urządzeń, a `GET /api/auth/sessions` i `DELETE /api/auth/sessions/:id` pozwalają zbudować ekran „zalogowane urządzenia":

```
POST /api/auth/logout               → 200 {"ok":true}, Set-Cookie: strapi_up_refresh=; expires=1970
GET  /api/users/me (ten sam token)  → 200
POST /api/auth/refresh (ciasteczko) → 401 "Invalid refresh token"
```

Token dostępu jest bezstanowy: Strapi sprawdza tylko podpis, czas i to, czy użytkownik istnieje. Po wylogowaniu działa więc aż do wygaśnięcia. Natychmiast odcina użytkownika dopiero `blocked: true`, bo strategia users-permissions przy każdym żądaniu pobiera użytkownika z bazy (`401 "Invalid credentials"`). Zmiana hasła (`/auth/change-password`) i reset hasła unieważniają wszystkie tokeny odświeżania użytkownika.

> **Pułapka:** „po restarcie serwera token przestał działać" prawie zawsze znaczy „minęło 10 minut". Token dostępu jest podpisany sekretem `JWT_SECRET` (a gdy go brak – `admin.auth.secret`), więc przeżywa restart, a nawet skasowanie bazy i ponowny seed – w obu przypadkach `GET /api/users/me` zwróciło 200. Ten sam token po 10 minutach dał `401 "Missing or invalid credentials"`. Tokeny tracą ważność także po zmianie sekretu i po przełączeniu `jwtManagement`: token z trybu `refresh` w trybie legacy i odwrotnie kończy się 401, więc zmiana trybu wylogowuje wszystkich.

Uwaga na niespójność: `httpOnly: true` nie obejmuje wszystkich tras. `POST /api/auth/local/register` i `/auth/change-password` zwracają `{"jwt","refreshToken","user"}` w ciele i nie ustawiają ciasteczka (sprawdzone w 5.56, widać to w `controllers/auth.js` pluginu). Po rejestracji najprościej od razu wywołać logowanie i zignorować `refreshToken` z odpowiedzi.

Tryb legacy

Po zmianie na `jwtManagement: 'legacy-support'` logowanie zwraca `{ jwt, user }` bez ciasteczka, a payload tokenu to `{ id: 1, iat, exp }` z ważnością 30 dni (`jwt.expiresIn`). `/api/auth/refresh`, `/logout` i `/sessions` odpowiadają 404. Skradziony token odetniesz tylko blokadą użytkownika albo zmianą sekretu dla wszystkich. Tryb legacy zostaw istniejącym

klientom; nowe aplikacje pisz pod `refresh`.

Uprawnienia w kodzie

Uprawnienie users-permissions to wiersz `{ action, role }`, np. `api::article.article.find` dla roli `public`. Klikanie w panelu działa lokalnie, ale na produkcji nikt nie pamięta, które pola zaznaczyć po wdrożeniu nowej akcji. Lepiej trzymać uprawnienia w `bootstrap`, który działa przy każdym starcie:

```
// src/index.ts (fragment)
const ROLE_PERMISSIONS: Record<string, string[]> = {
  authenticated: ['api::article.article.mine'],
};

const ensureRolePermissions = async (strapi: Core.Strapi) => {
  const roles = strapi.db.query('plugin::users-permissions.role');
  const permissions = strapi.db.query('plugin::users-permissions.permission');
  for (const [type, actions] of Object.entries(ROLE_PERMISSIONS)) {
    const role = await roles.findOne({ where: { type } });
    if (!role) throw new Error(`users-permissions role "${type}" does not exist`);
    for (const action of actions) {
      if ((await permissions.count({ where: { action, role: role.id } })) === 0) {
        await permissions.create({ data: { action, role: role.id } });
        strapi.log.info(`Granted ${action} to role ${type}`);
      }
    }
  }
};
```

Kod jest idempotentny: log `Granted api::article.article.mine to role authenticated` pojawia się tylko przy pierwszym starcie, a brak roli zatrzymuje start zamiast cicho pominąć uprawnienie. Plugin przy każdym starcie usuwa uprawnienia do akcji, których nie ma już w żadnym kontrolerze (`syncPermissions`), więc po zmianie nazwy akcji `bootstrap` nada je od nowa. Własną rolę (np. Editor) utworzysz w tym samym miejscu serwisem `strapi.plugin('users-permissions').service('role').createRole({ name, description })`, który sam wylicza `type` z nazwy (`editor`).

Rejestracja przyjmuje domyślnie tylko `username`, `email` i `password`. Próba ustawienia roli przez `POST /api/auth/local/register` z `"role":3` kończy się `400 "Invalid parameters: role"`, a nowy użytkownik dostaje rolę domyślną z ustawień pluginu. Własne pola dopuszczasz jawnie w `config/plugins.ts`: `register: { allowedFields: ['displayName'] }`.

Rozszerzanie users-permissions

Trasy i kontrolery pluginu nie pochodzą z `createCoreRouter`, więc zmieniasz je w `src/extensions/users-permissions/strapi-server.ts`. Plik eksportuje funkcję, która dostaje obiekt pluginu przed rejestracją i musi go zwrócić. Frontend „Redakcji” chce wiedzieć, czy zalogowana osoba jest zwykłym czytelnikiem, czy redaktorem, ale `GET /api/users/me?populate=role` zwraca użytkownika bez roli – sanityzacja usuwa relację, do której rola Authenticated nie ma uprawnień.

```
// src/extensions/users-permissions/strapi-server.ts (bez typów)
export default (plugin: UsersPermissionsPlugin) => {
  // Public profile name; accepted on registration thanks to register.allowedFields.
  plugin.contentTypes.user.schema.attributes.displayName = { type: 'string', maxLength: 60 };

  const originalMe = plugin.controllers.user.me;
  plugin.controllers.user.me = async (ctx) => {
    await originalMe(ctx);
    const role = (ctx.state.user as { role?: RoleSummary } | undefined)?.role;
    if (ctx.body && role) {
      ctx.body = { ...ctx.body, role: { name: role.name, type: role.type } };
    }
  };

  return plugin;
};
```

Wrapper najpierw wywołuje oryginalny `me`, więc walidacja i sanityzacja zostają, a rolę bierze z `ctx.state.user`, który strategia ładuje razem z `role`. Nowe pole `displayName` dodane programowo trafia do schematu jak każde inne; rejestracja z `"displayName":"Ewa K."` tworzy użytkownika z tym polem, a `/users/me` zwraca teraz `"displayName":null,"role":`

`{"name":"Authenticated","type":"authenticated"}`. Kontroler `user` to zwykły obiekt, ale `auth` (logowanie, rejestracja) jest fabryką `({ strapi }) => ({ ... })`. Żeby nadpisać jego akcję, owijasz całą fabrykę, co pokazuje przewodnik z dokumentacji.

Tylko moje artykuły

Rozdział 4 pokazał, że polityka nie przefiltruje listy, a filtr `author` dopisany do `ctx.query` odpada na walidacji (`Invalid key author`). Filtr musi powstać po walidacji zapytania klienta, czyli w kontrolerze:

```
// src/api/article/controllers/article.ts
export default factories.createCoreController('api::article.article', ({ strapi }) => ({
  async mine(this: CoreController, ctx) {
    const user = ctx.state.user as { id: number } | undefined;
    if (!user) {
      return ctx.unauthorized();
    }
    await this.validateQuery(ctx);
    const query = await this.sanitizeQuery(ctx);

    // The ownership filter is added after sanitization, so the client can neither remove it
    // nor trip over the role's missing read access to the user content type.
    const { results, pagination } = await strapi.service('api::article.article').find({
      ...query,
      status: query.status ?? 'draft',
      filters: { $and: [query.filters ?? {}, { author: { id: user.id } }] },
    });
    const sanitized = await this.sanitizeOutput(results, ctx);
    return this.transformResponse(sanitized, { pagination });
  },
}));
```

Trasa `GET /articles/mine` leży w `routes/01-mine.ts`, żeby wyprzedzić `/articles/:id` (rozdział 4), a uprawnienie `mine` nadaje roli Authenticated `bootstrap` z poprzedniej sekcji. Rdzeniowy serwis `find` woła Document Service i zwraca `pagination` dla `meta`. Domyślnie lista pokazuje wersje robocze, bo autor chce widzieć także szkice; `?status=published` daje tylko opublikowane.

```
GET /api/articles/mine?fields[0]=title&fields[1]=publishedAt   (JWT piotra)
{"data":[{"title":"Fetching Strapi from Next.js","publishedAt":null},
         {"title":"Self-hosting Strapi with Docker","publishedAt":null}],
 "meta":{"pagination":{"page":1,"pageSize":25,"pageCount":1,"total":2}}}
GET /api/articles/mine?filters[author][id]=2   (JWT anny)  → 400 "Invalid key author"
GET /api/articles/mine                         (bez tokenu) → 403 Forbidden
```

`publishedAt: null` oznacza tu wersję roboczą, a nie „nieopublikowany”; jeśli frontend ma pokazać status, pobierz go osobnym zapytaniem z `status: 'published'`. Alternatywą jest middleware Document Service (rozdział 6), ale działałoby ono przy każdym `findMany` – także w panelu, cronie i skryptach – i ukryłoby redakcji cudze artykuły. Reguła dotyczy jednej trasy HTTP, więc należy do kontrolera.

Szkice tylko dla uprawnionych

Teraz dziura z początku rozdziału. Uprawnienie `find` nie rozróżnia `status=draft` i `status=published`, więc trzeba wymusić wersję opublikowaną dla każdego, kto nie ma prawa do podglądu. „Redakcja” daje to prawo tylko API tokenom (podgląd w Next.js, rozdział 11), a autorom – przez `/articles/mine`. Middleware trasy wpięte w `find` i `findOne`:

```
// src/api/article/middlewares/published-only.ts
const publishedOnly: Core.MiddlewareFactory = () => {
  return async (ctx, next) => {
    const strategy = (ctx.state.auth as { strategy?: { name?: string } } | undefined)?.strategy?.name;
    if (strategy !== 'content-api-token') {
      ctx.query = { ...ctx.query, status: 'published' };
    }
    await next();
  };
};
```

Tym razem rozpakowanie `ctx.query` jest celowe: zostawiasz klientowi filtry, sortowanie i paginację, a nadpisujesz tylko `status`. Router artykułów podpina je w `config` akcji `find` i `findOne`. Anonimowe `?status=draft` zwraca teraz trzy opublikowane artykuły z ustawionym `publishedAt`, `GET /api/articles/<documentId szkicu>?status=draft` daje 404, a API

token nadal widzi cztery wersje robocze. Szczegóły draft & publish są w rozdziale 8.

> **Pułapka:** strategia API tokenów nazywa się w Strapi 5.56 `content-api-token`, nie `api-token`. Pierwsza wersja middleware porównywała z `'api-token'` i token podglądu dostawał po cichu wersje opublikowane, bez żadnego błędu. Nazwę sprawdzisz w `@strapi/admin/dist/server/server/src/strategies/content-api-token.js`, a uwierzytelniony użytkownik ma strategię `users-permissions` – tę samą co anonimowy gość, tylko z `credentials` różnym od `null`.

API tokeny

API token to klucz dla programu, nie dla osoby: serwer Next.js, skrypt importu, build statycznej strony. Nie ma użytkownika (`ctx.state.user` jest pusty), więc `/api/users/me` i `/api/articles/mine` zwracają z nim 401. Token tworzysz w *Settings → API Tokens* albo przez Admin API, z którego korzysta ten formularz:

```
$ npx strapi admin:create-user -e admin@example.com -p '...' -f Ada -l Admin
$ curl -X POST localhost:1307/admin/login ...   → {"data":{"token":"eyJhbGciOi...",...}}
$ curl -X POST localhost:1307/admin/api-tokens -H "Authorization: Bearer eyJhbGciOi..." \
    -H 'Content-Type: application/json' -d '{"name":"frontend-read-only","type":"read-only","lifespan":null}'
  → {"data":{"kind":"content-api","type":"read-only","accessKey":"da7ceba808...",...}}
```

Nowy projekt ma już dwa tokeny, „Read Only" i „Full Access". Dzięki `secrets.encryptionKey` w `config/admin.ts` klucze da się później odczytać w panelu; bez niego tylko raz, przy tworzeniu. Zmiana `API_TOKEN_SALT` unieważnia wszystkie tokeny. Z tokenem read-only `POST /api/articles` daje 403, z full-access – 201.

> **Pułapka:** read-only znaczy „wszystkie akcje `find` i `findOne`", także na użytkownikach. Token read-only zwrócił `GET /api/users?fields[0]=username&fields[1]=email` z adresami e-mail wszystkich trzech użytkowników, a full-access z `populate[author]=true` dołącza do artykułu pełny rekord autora. Token, który trafia do frontendu albo do zmiennej środowiskowej wielu serwisów, twórz jako typ Custom z zaznaczonymi tylko potrzebnymi akcjami i nigdy nie wysyłaj go do przeglądarki.

Logowanie przez GitHub

Frontend kieruje użytkownika na `/api/connect/github`, GitHub po zalogowaniu wraca do Strapi, a Strapi przekazuje `access_token` na adres frontendu. Ten wymienia go na sesję przez `GET /api/auth/github/callback?access_token=...`; konto jest dopasowane po e-mailu albo utworzone. Konfiguracja leży w magazynie pluginu (klucz `grant`) i zwykle ustawia się ją w panelu. Możesz ją pisać w `bootstrap` ze zmiennych środowiskowych:

```
// src/index.ts (fragment; key and secret come from GITHUB_CLIENT_ID / GITHUB_CLIENT_SECRET)
const store = strapi.store({ type: 'plugin', name: 'users-permissions', key: 'grant' });
const grant = (await store.get()) as Record<string, GrantProvider>;
grant.github = { ...grant.github, enabled: true, key, secret, callback: frontendCallback };
await store.set({ value: grant });
```

Z fikcyjnym `client_id` lokalnie sprawdzisz tylko pierwszy krok: `GET /api/connect/github` odpowiada `302` na `https://github.com/login/oauth/authorize?client_id=dummy-client-id&...&redirect_uri=http://localhost:1307/api/connect/github/callback&scope=user,user:email`. Reszta wymaga aplikacji OAuth w GitHubie i publicznego adresu (GitHub nie akceptuje `localhost`). Callback w trybie `refresh` odpowiada tą samą funkcją co logowanie hasłem (`sendRefreshAuthResponse`). Przykładowy wynik: `{"jwt":"eyJhbGciOi...","user":{"username":"anna-gh","provider":"github",...}}` plus ciasteczko `strapi_up_refresh`.

Reset hasła (`/auth/forgot-password` → e-mail z kodem → `/auth/reset-password`) i potwierdzanie adresu e-mail (*Advanced settings → Enable email confirmation*) wymagają działającego providera e-maila, który konfiguruje rozdział 8. Przy włączonym potwierdzaniu rejestracja zwraca samego `user` bez tokenu, a niepotwierdzone konto nie zaloguje się.

Podsumowanie

- Administratorzy panelu (RBAC, darmowe własne role) i użytkownicy aplikacji (users-permissions) mają osobne tabele, trasy i tokeny.
- Nowy projekt działa w trybie `refresh`: token dostępu na 10 minut w ciele odpowiedzi, token odświeżania w ciasteczku `httpOnly`, rotowany przy każdym `/auth/refresh`. Klient odświeża jednym, wspólnym żądaniem.

- Wylogowanie i usunięcie sesji nie unieważniają wydanego tokenu dostępu; natychmiast działa tylko `blocked: true`. „401 po restarcie” to zwykle wygasły token.
- Uprawnienia ról nadawaj idempotentnym kodem w `bootstrap`; users-permissions rozszerzasz w `src/extensions/users-permissions/strapi-server.ts`, owijając oryginalne akcje.
- Lista „tylko moje” to akcja kontrolera z filtrem dopisanym po `sanitizeQuery`, a nie polityka ani filtr w `ctx.query`.
- Uprawnienie `find` obejmuje `status=draft`; middleware trasy wymusza wersję opublikowaną dla wszystkich poza API tokenem podglądu.
- API token read-only czyta także użytkowników; dla frontendu twórz token Custom.

Co dalej

Masz już komplet narzędzi do kontroli dostępu: uprawnienia ról, polityki, kontrolery i tokeny. Rozdział 8 przechodzi do funkcji wbudowanych, które z nich korzystają: draft & publish od strony API, i18n, media, e-mail (którego potrzebuje reset hasła), webhooki i cron.

Źródła

- https://docs.strapi.io/cms/features/users-permissions
- https://docs.strapi.io/cms/features/users-permissions/rest-api
- https://docs.strapi.io/cms/backend-customization/guides/customizing-users-permissions-plugin-routes
- https://docs.strapi.io/cms/backend-customization/examples/authentication
- https://docs.strapi.io/cms/plugins-development/plugins-extension
- https://docs.strapi.io/cms/features/api-tokens
- https://docs.strapi.io/cms/features/admin-tokens
- https://docs.strapi.io/cms/features/rbac
- https://docs.strapi.io/cms/configurations/users-and-permissions-providers
- https://docs.strapi.io/cms/configurations/users-and-permissions-providers/github

Rozdział 8. Draft & publish, i18n, media, e-mail, webhooki, cron

> **Cel rozdziału:** nauczysz się korzystać z darmowych funkcji wbudowanych w Strapi 5 od strony kodu i klienta API – wersji roboczych, tłumaczeń, plików, poczty, webhooków, zadań cyklicznych i podglądu – oraz poznasz miejsca, w których zachowują się inaczej, niż sugeruje panel.
>
> **Kod:** `examples/08-funkcje-wbudowane/`. **Pakiety poza fundamentem:** `npm install @strapi/provider-email-nodemailer`; opcjonalnie `@strapi/provider-upload-aws-s3` (tylko konfiguracja produkcyjna).

Redakcja ma już reguły zapisu z rozdziałów 4–6. Teraz potrzebuje rzeczy, których nie pisze się od zera: autorka chce poprawić lead opublikowanego artykułu tak, żeby czytelnicy nie widzieli wersji w połowie edycji. Część tekstów ma wersję polską. Okładki trafiają do biblioteki mediów, autor dostaje e-mail o nowym komentarzu, frontend odświeża cache po publikacji, a komentarze, których nikt nie zmoderował przez miesiąc, same przechodzą w stan odrzucony.

Każda z tych funkcji ma swoje „ale", którego nie widać w panelu. Mechanikę Document Service (`status`, `publish()`, lifecycle hooki i middleware DS) znasz z rozdziału 6, więc tutaj tylko z niej korzystasz.

Draft & publish od strony klienta

Typ z draft & publish ma dla każdego dokumentu wersję roboczą i – po publikacji – wersję opublikowaną. REST domyślnie czyta wersje opublikowane, odwrotnie niż Document Service. Domyślnie też **publikuje przy zapisie**: `POST` i `PUT` bez parametru `status` zapisują draft i od razu go publikują. Żeby poprawić tylko wersję roboczą, autorka dodaje `?status=draft`:

```
curl -X PUT "localhost:1308/api/articles/$DOC?status=draft" -H "Authorization: Bearer $ANNA" \
  -H 'content-type: application/json' -d '{"data":{"excerpt":"Updated intro, not yet live."}}'
curl -g "localhost:1308/api/articles/$DOC?fields[0]=excerpt"                    # still the old one
curl -g "localhost:1308/api/articles?publicationFilter=modified&fields[0]=title" # edited since publication
curl -X PUT "localhost:1308/api/articles/$DOC" -H "Authorization: Bearer $ANNA" \
  -H 'content-type: application/json' -d '{"data":{}}'                          # publish the draft
```

```
{"excerpt":"Updated intro, not yet live.","publishedAt":null}
"A short introduction to policies in strapi 5."
["Policies in Strapi 5"]
{"excerpt":"Updated intro, not yet live.","publishedAt":"2026-10-03T18:29:43.140Z","id":8}
```

Pusty `PUT` to w REST jedyny sposób na publikację: Content API nie ma tras `publish` ani `unpublish` (`POST /api/articles/:id/actions/publish` zwrócił 405). Wycofanie z publikacji robi redaktor w panelu albo twój kod przez `strapi.documents(uid).unpublish()` – na przykład w akcji niestandardowej z rozdziału 5. Parametr `publicationFilter` (`modified`, `never-published` i inne) wybiera dokumenty według relacji między ich wersjami.

Najważniejsza rzecz dotyczy jednak czytelnika, nie autora. Uprawnienie `find` w roli Public obejmuje oba statusy, więc anonimowy klient czyta szkice:

```
curl -g "localhost:1308/api/articles?status=draft&fields[0]=title&fields[1]=publishedAt"
```

```
{"id":7,"documentId":"fhjgx99rqsclaci4fraebtzj","title":"Self-hosting Strapi with Docker","publishedAt":null}
... 4 documents, including the one that was never published
```

Draft & publish to więc przepływ pracy redakcji, a nie mechanizm poufności. Jeśli szkice nie mogą wyciec przed premierą, zablokuj `status=draft` dla ról bez tokena – jak to zrobić, pokazuje rozdział 7.

i18n: tłumaczenia na poziomie pól

Internacjonalizacja (i18n) jest w Strapi 5 wbudowana. Włączasz ją w schemacie typu i decydujesz pole po polu, co jest tłumaczone. W Redakcji tłumaczone są `title`, `slug`, `excerpt`, `content` i `seo`; `readingTime` i `cover` są wspólne dla wszystkich języków:

```
"pluginOptions": { "i18n": { "localized": true } },
"attributes": {
  "title": { "type": "string", "required": true, "maxLength": 160,
             "pluginOptions": { "i18n": { "localized": true } } },
  "readingTime": { "type": "integer", "min": 0,
                   "pluginOptions": { "i18n": { "localized": false } } }
}
```

Domyślnym językiem nowego projektu jest `en`; istniejące artykuły po włączeniu i18n dostały `locale: "en"`. Język polski trzeba dodać – w panelu (Settings → Internationalization) albo w `bootstrap()`, żeby każde środowisko miało go z kodu: `strapi.plugin('i18n').service('locales').create({ code: 'pl', name: 'Polish (pl)' })`, poprzedzone sprawdzeniem `findByCode('pl')`. Tłumaczenie istniejącego dokumentu to `PUT` z parametrem `locale`:

```
curl -X PUT "localhost:1308/api/articles/$DOC?locale=pl" -H "Authorization: Bearer $ANNA" \
  -H 'content-type: application/json' \
  -d '{"data":{"title":"Polityki w Strapi 5","slug":"polityki-w-strapi-5","excerpt":"Krótkie wprowadzenie do
polityk."}}'
```

```
{"id":11,"documentId":"zww8ba9zgusomej4xxkrium6","locale":"pl","title":"Polityki w Strapi 5","publishedAt":"2026-10-
03T18:30:10.255Z"}
```

Ten sam `documentId`, nowy wiersz, od razu opublikowany. W Document Service odpowiednikiem jest `update({ documentId, locale: 'pl', data })`, które tworzy tłumaczenie jako draft; publikujesz je osobno przez `publish({ documentId, locale: 'pl' })`. Wersje językowe żyją niezależnie: publikacja polskiej nie publikuje angielskiej.

Przy odczycie zwróć uwagę na trzy zachowania. Po pierwsze, nie ma fallbacku: `GET /api/articles/:id?locale=pl` dla artykułu bez polskiej wersji zwraca 404, a nie wersję angielską – frontend musi sam zdecydować, co pokazać. Po drugie, nieznany język (`locale=de`) nie jest błędem, tylko pustą listą. Po trzecie, wszystkie języki naraz daje `locale=*` – zadziałało w REST i w Document Service, choć strona REST dokumentacji wymienia gwiazdkę tylko przy Document Service; `locale=all` zwraca pustą listę, bo „all” jest traktowane jak nazwa języka.

Relacje i pola `uid` są w i18n **zawsze** tłumaczone, niezależnie od `pluginOptions` (kod `isLocalizedAttribute` w pluginie i18n). Polska wersja startuje więc bez kategorii i bez autora: przy tłumaczeniu musisz je podać w `data`, inaczej filtr po kategorii jej nie znajdzie.

> **Pułapka:** pola nietłumaczone są kopiowane między językami tylko w obrębie **tego samego statusu**: zapis draftu aktualizuje drafty innych języków, publikacja – ich wersje opublikowane. REST-owy `PUT` publikujący zwraca wersję opublikowaną, więc synchronizuje tylko wersje opublikowane. W moim teście: `PUT ?status=draft z readingTime: 9` dla `en`, potem publikujący `PUT z readingTime: 6` dla `en` (polski draft dalej ma 9), a na koniec zmiana samego leadu w wersji `pl`. Publikacja polskiej wersji skopiowała jej nieaktualne `readingTime: 9` do opublikowanej wersji angielskiej, która miała 6. W panelu, sądząc po kodzie (nie klikałem tego), problem nie powstaje: „Publish” najpierw zapisuje draft (rozdział 6), więc drafty innych języków dostają nową wartość. Jeśli edytujesz pola wspólne przez REST, zapisuj je jako draft i dopiero potem publikuj każdy język.

Media Library: najpierw plik, potem powiązanie

Pliki trafiają do biblioteki mediów osobnym żądaniem `multipart/form-data` na `POST /api/upload`, a do artykułu podpinasz je zwykłym zapisem. Rola potrzebuje uprawnienia `plugin::upload.content-api.upload`, którego seed nie nadaje (anonimowy upload dostaje 403); w przykładzie dodaje je `bootstrap()`. Anna wysyła okładkę 1200×800 px z tekstem alternatywnym:

```
curl -X POST localhost:1308/api/upload -H "Authorization: Bearer $ANNA" \
  -F "files=@cover.jpg" -F 'fileInfo={"alternativeText":"Gradient z napisem Redakcja"}'
```

```
[{ "id": 1, "documentId": "pczbswhmeh4bfuk1y6ziuy3z", "name": "cover.jpg", "mime": "image/jpeg",
   "size": 21.12, "width": 1200, "height": 800, "url": "/uploads/cover_b62e14258a.jpg", "provider": "local",
   "formats": {
     "thumbnail": { "width": 234, "height": 156, "url": "/uploads/thumbnail_cover_b62e14258a.jpg" },
     "large":     { "width": 1000, "height": 667, "url": "/uploads/large_cover_b62e14258a.jpg" },
     "medium":    { "width": 750, "height": 500, "url": "/uploads/medium_cover_b62e14258a.jpg" },
     "small":     { "width": 500, "height": 333, "url": "/uploads/small_cover_b62e14258a.jpg" } } }]
```

Strapi wygenerował miniaturę i trzy wersje responsywne według domyślnych breakpointów 1000/750/500 px (Strapi 5.56, październik 2026); zmienisz je kluczem `breakpoints` w konfiguracji `upload` w `config/plugins.ts`, ale tylko dla nowych plików. `size` jest w kilobajtach. Powiązanie to `PUT /api/articles/:documentId` z `{"data":{"cover":1}}` – przyjęło zarówno numeryczne `id`, jak i `documentId` pliku. Okładkę zobaczysz dopiero z `populate[cover]`.

`allowedTypes` i `deniedTypes` z fundamentu działają: SVG skończył się `400 File type 'image/svg+xml' is not allowed`, a skrypt powłoki wysłany jako `text/plain` – `File type 'application/x-sh' is not allowed`, bo Strapi rozpoznaje typ binarny po zawartości. Tekstowego SVG zawartość nie zdradza: ten sam plik nazwany `fake.png` z typem `image/png` przeszedł. Ryzyko jest tu małe, bo serwer oddaje go jako `image/png` z nagłówkiem `X-Content-Type-Options: nosniff`, więc przeglądarka nie wykona go jako SVG – ale nie traktuj `deniedTypes` jako skanera treści.

Provider S3-kompatybilny

Lokalny dysk wystarcza na jednym serwerze z wolumenem (rozdział 13). Przy kilku instancjach albo CDN przenosisz media do bucketu: MinIO, Cloudflare R2, Hetzner Object Storage czy AWS S3 obsługuje ten sam provider `@strapi/provider-upload-aws-s3`. Konfiguracja produkcyjna w `config/env/production/plugins.ts` (scalana z bazową, więc `security` i e-mail zostają):

```
upload: { config: {
  provider: 'aws-s3',
  providerOptions: {
    baseUrl: env('S3_PUBLIC_URL'), // e.g. https://media.redakcja.example
    s3Options: {
      credentials: { accessKeyId: env('S3_ACCESS_KEY_ID'), secretAccessKey: env('S3_ACCESS_SECRET') },
      region: env('S3_REGION', 'auto'),
      endpoint: env('S3_ENDPOINT'),
      forcePathStyle: env.bool('S3_FORCE_PATH_STYLE', true), // MinIO needs it, R2 does not
      params: { Bucket: env('S3_BUCKET') },
} } } },
```

Plik przechodzi `tsc`, ale nie uruchamiałem go z prawdziwym bucketem. Drugi krok to CSP: panel admina ładuje miniatury z bucketu, więc `strapi::security` w `config/middlewares.ts` zamieniasz na obiekt z `contentSecurityPolicy.directives`, dopisując domenę mediów do `img-src` i `media-src` (obok `'self'`, `data:`, `blob:` i `market-assets.strapi.io`). Przykładowy wynik: po uploadzie `url` w odpowiedzi to `https://media.redakcja.example/cover_b62e14258a.jpg`, a `provider` to `aws-s3`. R2 nie obsługuje ACL, więc nie ustawiaj tam `params.ACL`.

E-mail: powiadomienie o nowym komentarzu

Domyślny provider e-mail to `sendmail`, który w produkcji się nie sprawdza. Dokumentacja poleca `@strapi/provider-email-nodemailer`, czyli zwykłe SMTP. Konfiguracja w `config/plugins.ts`:

```
email: { config: {
  provider: 'nodemailer',
  providerOptions: {
    host: env('SMTP_HOST', 'localhost'),
    port: env.int('SMTP_PORT', 2525),
    secure: env.int('SMTP_PORT', 2525) === 465, // implicit TLS only on 465
    ...(env('SMTP_USERNAME') ? { auth: { user: env('SMTP_USERNAME'), pass: env('SMTP_PASSWORD') } } : {}),
  },
  settings: { defaultFrom: env('EMAIL_FROM', 'Redakcja <no-reply@redakcja.example>') },
} },
```

W developmencie wygodny jest Mailpit w Dockerze; ja użyłem 36-liniowego „zlewu” SMTP na samym module `net` z Node (`tools/smtp-sink.mjs`), który przyjmuje każdą wiadomość i wypisuje ją na konsolę.

Gdzie wysłać powiadomienie? Komentarz może powstać przez REST, panel albo import, więc – zgodnie z rozdziałem 6 – w middleware Document Service, po `await next()`, kiedy zapis jest już w bazie:

```
export const newCommentEmail = (strapi: Core.Strapi): Modules.Documents.Middleware.Middleware => {
  return async (context, next) => {
    const result = await next();
    if (context.uid !== 'api::comment.comment' || context.action !== 'create') return result;

    const { documentId } = result as { documentId: string };
    const comment = await strapi.documents('api::comment.comment').findOne({
      documentId,
      fields: ['body'],
      populate: { article: { fields: ['title'], populate: { author: { fields: ['username', 'email'] } } } },
    });
    const article = comment?.article;
    const author = article?.author;
    if (!comment || !article || !author?.email) return result;

    // Fire and forget: a slow or broken SMTP server must not fail or delay the comment request.
    strapi.plugin('email').service('email')
      .sendTemplatedEmail({ to: author.email }, template, {
        comment: { body: comment.body }, article: { title: article.title }, author: { username: author.username },
      })
      .catch((error: Error) => strapi.log.error(`New comment email failed: ${error.message}`));
    return result;
  };
};
```

`sendTemplatedEmail(options, template, data)` wymaga w szablonie wszystkich trzech kluczy: `subject`, `text` i `html`, a składnię bierze z szablonów lodasha. Piotr komentuje artykuł Anny, wpisując w treść znacznik `<script>`. Zlew SMTP odebrał (treść zdekodowana z quoted-printable):

```
From: Redakcja <no-reply@redakcja.example>
To: anna@example.com
Reply-To: redakcja@redakcja.example
Subject: Nowy komentarz do „Policies in Strapi 5”
Content-Type: multipart/alternative

[text/plain] Świetny tekst! <script>alert(1)</script> Czy będzie część o middleware?
[text/html]  <blockquote>Świetny tekst! &lt;script&gt;alert(1)&lt;/script&gt; Czy będzie część o middleware?
</blockquote>
```

W części HTML szablon używa `<%- comment.body %>`, które escapuje HTML. Strapi ogranicza `<%= ... %>` do kluczy z `data`, ale **nie escapuje** – w części tekstowej widać surowy znacznik, a w HTML byłby to wstrzyknięty kod w skrzynce autora. Wysyłka bez `await` jest celowa: przy wyłączonym serwerze SMTP komentarz dalej dostał 201 w 21 ms, a w logu pojawiło się `New comment email failed: connect ECONNREFUSED ::1:2588`. Ceną jest brak ponowień – gdy powiadomienia są ważne, zapisz je w kolejce (tabela + cron niżej) zamiast wysyłać w żądaniu. Pamiętaj też, że middleware działa dla każdego zapisu: seed tworzący komentarze też próbował wysłać maile.

Webhooki: powiadomienie frontendu o publikacji

Webhook to żądanie `POST`, które Strapi wysyła na twój adres po zdarzeniu w treści: `entry.create`, `entry.update`, `entry.delete`, `entry.publish`, `entry.unpublish`, `entry.draft-discard` oraz `media.*`. Frontend Redakcji chce wiedzieć o publikacji, wycofaniu i usunięciu artykułu, żeby odświeżyć cache (rozdział 11). Zamiast wyklikać webhook w panelu (Settings → Webhooks) w każdym środowisku osobno, w przykładzie `bootstrap()` tworzy webhook `frontend-revalidate` z adresu w `FRONTEND_WEBHOOK_URL`, korzystając z tych samych usług co kontroler panelu:

```
const store = strapi.get('webhookStore') as WebhookStore;
const runner = strapi.get('webhookRunner') as WebhookRunner;
const existing = (await store.findWebhooks()).find((webhook) => webhook.name === NAME);
if (!existing) {
  // The store only saves the row; the runner must be told about it, like the admin controller does.
  runner.add(await store.createWebhook({ name: NAME, url, headers: {}, events: EVENTS }));
}
```

`webhookStore` zapisuje wiersz w tabeli `strapi_webhooks`, a `webhookRunner` nasłuchuje zdarzeń. Runner wczytuje webhooki z bazy w swoim `bootstrap`, który działa przed twoim, więc nowy webhook trzeba mu przekazać ręcznie (pełny plik z aktualizacją adresu jest w `examples/`). To API wewnętrzne – typy `WebhookStore` w przykładzie opisałem sam na podstawie kodu `@strapi/core`.

Sekret dokładasz nagłówkiem. `webhooks.defaultHeaders` w `config/server.ts` trafia do każdego webhooka, a nagłówek ustawiony w samym webhooku go nadpisuje:

```
webhooks: {
  defaultHeaders: { 'X-Webhook-Secret': env('WEBHOOK_SECRET', '') },
},
```

Odbiorca (`tools/webhook-receiver.mjs`, 28 linii na `node:http`) porównuje nagłówek przez `timingSafeEqual` i bez niego odpowiada 401. Po publikacji artykułu przez REST odebrał:

```
{ "headers": { "x-webhook-secret": "local-test-secret", "x-strapi-event": "entry.publish",
               "content-type": "application/json", "user-agent": "node" },
  "body": { "event": "entry.publish", "createdAt": "2026-10-03T18:29:43.169Z", "model": "article",
            "uid": "api::article.article",
            "entry": { "id": 8, "documentId": "zww8ba9zgusomej4xxkrium6", "locale": "en",
                       "slug": "policies-in-strapi-5", "publishedAt": "2026-10-03T18:29:43.140Z",
                       "category": { "name": "Backend", "...": "..." }, "comments": ["..."],
                       "author": { "username": "anna", "email": "anna@example.com", "...": "..." } } } }
```

Publikacja polskiej wersji przyszła jako osobne zdarzenie z `locale: "pl"` i `slug: "polityki-w-strapi-5"`, a wycofanie przez Admin API panelu – jako `entry.unpublish`. Payload zawiera cały wpis z relacjami, komponentami i dynamic zones i nie da się tego skonfigurować. Opcja `webhooks.populateRelations`, którą fundament ma w `config/server.ts`, pochodzi ze Strapi 4 – w `node_modules` nie ma żadnego kodu, który by ją czytał, więc w przykładzie ją usunąłem.

> **Pułapka:** dokumentacja zapewnia, że webhooki nie działają dla typu użytkownika „ze względu na prywatność”, a pola prywatne nie trafiają do payloadu. Ale relacja `author` jest wypełniona i niesie `email`, `provider`, `confirmed` i `blocked` autorki – wycięte są tylko hasło i tokeny. Jeśli webhook idzie do usługi zewnętrznej, wysyłasz jej dane osobowe autorów. Gdy odbiorcy wystarczy `documentId` i `slug`, wyślij własne, chude żądanie z middleware Document Service.

Runner jest „wystrzel i zapomnij”: kolejka po pięć żądań naraz, limit 10 sekund, bez ponowień (Strapi 5.56, październik 2026). Gdy wyłączyłem odbiorcę i opublikowałem artykuł, w logu Strapi nie pojawiło się nic – błąd jest połykany. Frontend musi więc mieć rewalidację awaryjną (np. `revalidate` co kilka minut), a nie polegać wyłącznie na webhooku. W produkcji (`NODE_ENV=production`) panel odrzuca też adresy, które rozwiązują się na localhost albo adres prywatny (10.x, 172.16–31.x, 192.168.x) – czyli także nazwę usługi Dockera w tej samej sieci. To ochrona przed SSRF. Kod z `bootstrap()` tej walidacji nie przechodzi, więc jeśli świadomie kierujesz webhook do kontenera obok, zrobisz to tylko z kodu.

Cron: komentarze, na które nikt nie odpowiedział

Zadania cykliczne to funkcje uruchamiane przez Strapi według reguły w formacie crona (z opcjonalnym polem sekund na początku). Zadanie Redakcji odrzuca komentarze czekające na moderację dłużej niż 30 dni. Masowy `updateMany` przez Query Engine i jego koszty omówił rozdział 6; tutaj ważne jest opakowanie:

```
export default {
  rejectStaleComments: {
    async task({ strapi }: { strapi: Core.Strapi }) {
      const cutoff = new Date(Date.now() - pendingDays * DAY_MS);
      const { count } = await strapi.db.query('api::comment.comment').updateMany({
        where: { moderationStatus: 'pending', createdAt: { $lt: cutoff } },
        data: { moderationStatus: 'rejected' },
      });
      strapi.log.info(`[cron] rejectStaleComments: ${count} comment(s) older than ${pendingDays} days rejected`);
    },
    options: { rule: process.env.COMMENT_REJECT_RULE ?? '0 0 3 * * *', tz: 'Europe/Warsaw' },
  },
};
```

Plik `config/cron-tasks.ts` nie jest wczytywany automatycznie – importujesz go w `config/server.ts` i włączasz: `cron: { enabled: env.bool('CRON_ENABLED', false), tasks: cronTasks }`. Bez tego klucza Strapi przyjmuje `enabled: false`; z kodu providera wynika, że flaga dotyczy tylko zadań z konfiguracji – te dodane przez `strapi.cron.add()` ruszą i tak. Do testu uruchomiłem serwer z `COMMENT_REJECT_RULE='*/15 * * * * *'` i przestawiłem w SQLite datę utworzenia dwóch oczekujących komentarzy o 40 dni wstecz:

```
[2026-10-03 20:32:30.008] info: [cron] rejectStaleComments: 0 comment(s) older than 30 days rejected
[2026-10-03 20:32:45.038] info: [cron] rejectStaleComments: 2 comment(s) older than 30 days rejected
→ GET /api/comments: [[4,"rejected"],[5,"pending"],[6,"rejected"]]
```

Komentarz numer 5 był świeży i został w kolejce. Nazwane zadanie możesz później usunąć przez `strapi.cron.remove('rejectStaleComments')`. Dokumentacja wciąż opisuje bibliotekę `node-schedule`, ale Strapi 5.56 używa `croner` – składnia reguł jest zgodna, a wyjątek rzucony w zadaniu kończy się wpisem `Cron job "..." failed` w logu zamiast awarii procesu.

Cron w Strapi działa w każdym procesie: przy dwóch replikach zadanie wykona się dwa razy. `updateMany` jest idempotentne, więc tu to nie szkodzi, ale wysyłka newslettera już tak. Takie zadania uruchamiaj na jednej instancji (`CRON_ENABLED=true` tylko tam) albo w zewnętrznym harmonogramie wołającym trasę z API tokenem.

Preview: przycisk podglądu w panelu

Funkcja Preview dodaje w edycji wpisu przycisk otwierający stronę frontendu z aktualnym szkicem. W darmowej wersji podgląd otwiera się na pełnym ekranie panelu; widok obok formularza i edycja w samym podglądzie to Live Preview z planów Growth i Enterprise. Konfiguracja leży w `config/admin.ts`: `handler` zamienia dokument na adres frontendu, a `null` ukrywa podgląd dla typu:

```
preview: {
  enabled: true,
  config: {
    allowedOrigins: [env('CLIENT_URL', 'http://localhost:3000')],
    async handler(uid, { documentId, locale, status }) {
      if (uid !== 'api::article.article') return null;
      const article = await strapi.documents(uid).findOne({
        documentId, locale, status: status === 'published' ? 'published' : 'draft', fields: ['slug'],
      });
      if (!article) return null;
      const params = new URLSearchParams({ url: `/artykuly/${article.slug}`, secret: env('PREVIEW_SECRET', ''),
status: status ?? 'draft' });
      return `${env('CLIENT_URL', 'http://localhost:3000')}/api/preview?${params}`;
    },
  },
},
```

Panel woła handler przez Admin API, więc sprawdziłem go z JWT administratora (`GET /content-manager/preview/url/api::article.article?documentId=...&locale=pl&status=draft`): odpowiedź to `{"data":{"url":"http://localhost:3000/api/preview?url=%2Fartykuly%2Fpolityki-w-strapi-5&secret=&status=draft"}}`, a dla kategorii – 204 bez treści. Strapi dopisał też `CLIENT_URL` do `frame-src` w nagłówku CSP, żeby panel mógł osadzić frontend w ramce. Trasę `/api/preview` po stronie Next.js, która sprawdza sekret i włącza draft mode, buduje rozdział 11.

Podsumowanie

- REST domyślnie czyta i **zapisuje** wersje opublikowane; `?status=draft` edytuje sam szkic, a publikacja w REST to `PUT` bez `status` – tras publish/unpublish nie ma.
- Public z uprawnieniem `find` czyta szkice przez `status=draft`; ochronę opisuje rozdział 7.
- i18n włączasz w schemacie per pole; relacje i `uid` są zawsze tłumaczone, nie ma fallbacku języka, `locale=*` zwraca wszystkie wersje.
- Pola nietłumaczone synchronizują się tylko w obrębie jednego statusu – publikacja jednego języka przez REST może nadpisać je w innym.
- Upload to osobne `POST /api/upload`, potem powiązanie przez `id` pliku; `deniedTypes` rozpoznaje typy binarne po zawartości, tekstowe po deklaracji.
- E-mail wysyłaj z middleware Document Service po `next()`, bez `await` na SMTP, a dane użytkownika w HTML wstawiaj przez `<%- %>`.
- Webhook niesie pełny wpis łącznie z e-mailem autora, nie ponawia prób i nie loguje błędów; sekret dodaj w `webhooks.defaultHeaders`.
- Cron wymaga `server.cron.enabled: true` i działa w każdej replice.

Co dalej

Funkcje z tego rozdziału zbudowałeś w `src/` i `config/` jednego projektu. Gdy ten sam mechanizm – na przykład kolejka moderacji komentarzy z powiadomieniami – ma działać w kilku projektach, przenosisz go do pluginu z własnymi trasami, serwisami i stroną w panelu. Tym zajmuje się rozdział 9.

Źródła

- https://docs.strapi.io/cms/features/draft-and-publish
- https://docs.strapi.io/cms/api/rest/status
- https://docs.strapi.io/cms/api/rest/publication-filter
- https://docs.strapi.io/cms/features/internationalization
- https://docs.strapi.io/cms/api/rest/locale
- https://docs.strapi.io/cms/api/document-service/locale
- https://docs.strapi.io/cms/features/media-library
- https://docs.strapi.io/cms/api/rest/upload
- https://docs.strapi.io/cms/configurations/media-library-providers
- https://docs.strapi.io/cms/configurations/media-library-providers/amazon-s3
- https://docs.strapi.io/cms/features/email
- https://docs.strapi.io/cms/configurations/email-nodemailer
- https://docs.strapi.io/cms/backend-customization/webhooks
- https://docs.strapi.io/cms/configurations/cron
- https://docs.strapi.io/cms/configurations/server
- https://docs.strapi.io/cms/features/preview

Rozdział 9. Własny plugin

> **Cel rozdziału:** zbudować plugin „comment-moderation” ze stroną serwerową i ekranem w panelu admina, zrozumieć, jak Strapi ładuje plugin lokalny, i wiedzieć, kiedy plugin jest lepszy od kodu w `src/api`.
>
> **Kod:** `examples/09-pluginy/`. **Pakiety poza fundamentem:** `npx @strapi/sdk-plugin init` (SDK 6.1.1) i `npm install --legacy-peer-deps` w katalogu pluginu.

W rozdziale 5 zatwierdzanie komentarza stało się akcją `POST /api/comments/:id/approve`. Działa, ale redaktorka nie pracuje w curl-u, tylko w panelu admina. Chce jednego ekranu: lista komentarzy czekających na moderację, przy każdym „Approve” i „Reject”. Czytelnik ma też móc zgłosić opublikowany komentarz, a zgłoszony wraca do kolejki.

Drugi powód jest organizacyjny. Wydawnictwo planuje drugi magazyn na osobnej instancji Strapi i chce tam tej samej moderacji. Kopiowanie kontrolerów, tras i ekranu między repozytoriami kończy się dwiema wersjami, które rozjadą się po pierwszej poprawce. Plugin pakuje serwer i panel w jedną paczkę z własną konfiguracją i wersją.

Zbudujesz plugin `comment-moderation` od szkieletu z Plugin SDK do działającej kolejki, a potem rozszerzysz wbudowany plugin przez `src/extensions`, dodasz narzędzie do serwera MCP i przygotujesz paczkę do publikacji.

Plugin czy kod w `src/api`

Plugin nie daje mocy, której nie ma kod aplikacji: trasy, kontrolery, serwisy i polityki działają jak w rozdziałach 4 i 5, z przedrostkiem `plugin::comment-moderation.` zamiast `api::`, a ekran w panelu dodasz też w `src/admin/app.tsx` (rozdział 10). Różnica leży w granicach: plugin ma własny `package.json`, build, konfigurację i uprawnienia, a projekt widzi go jako jedną zależność.

Logika jednego projektu zostaje więc w `src/api`. Plugin opłaca się, gdy kod ma żyć w kilku projektach, łączy serwer z ekranem w panelu albo ma mieć własny cykl wydań. Uproszczenie Redakcji: plugin pracuje na typie `api::comment.comment` z aplikacji; plugin dla obcych projektów dostarczyłby własny typ w `server/src/content-types`.

Szkielet z Plugin SDK

Dokumentacja zaleca tworzenie pluginów przez Plugin SDK. Polecenie zadaje kilkanaście pytań (nazwa, opis, admin i serwer, TypeScript, ESLint) i generuje szkielet:

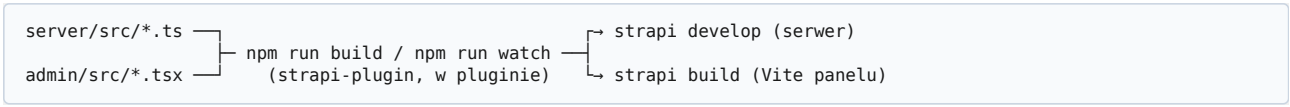
```
$ npx @strapi/sdk-plugin init src/plugins/comment-moderation
src/plugins/comment-moderation/
  package.json   exports: ./strapi-server → dist/server, ./strapi-admin → dist/admin
  server/src/    index.ts, register.ts, bootstrap.ts, config/, routes/, controllers/,
                 services/, policies/, middlewares/, content-types/
  admin/src/     index.ts (menu, rejestracja), pages/, components/, translations/
```

Na koniec SDK podpowiada wpis do `config/plugins.ts`. Dodajesz go z konfiguracją pluginu:

```
// config/plugins.ts (fragment)
'comment-moderation': {
  enabled: true,
  // Strapi loads the built files from package.json "exports" (dist/), not the sources.
  resolve: './src/plugins/comment-moderation',
  config: { pageSize: 10 },
},
```

Przewodnik dokumentacji opisuje głównie plugin jako osobną paczkę: w jej katalogu działa `npm run watch:link`, który przebudowuje kod i publikuje go do lokalnego magazynu `yalc`, a projekt dołącza go przez `npx yalc add --link comment-moderation`, bez `resolve`. Dla pluginu w `src/plugins` kroki są według dokumentacji „takie same”, poza wpisem z `resolve`. Tę wersję wybiera Redakcja: plugin jest wersjonowany z aplikacją, a globalny `yalc` nie jest potrzebny.

Kluczowe jest pole `exports`. Loader w `@strapi/core` szuka wejścia serwera pod `./strapi-server`, czyli w `dist/server/index.js`, a build panelu importuje `./strapi-admin` z `dist/admin`. Strapi nie czyta `server/src` ani `admin/src`, więc plugin trzeba zbudować:



> **Pułapka:** plugin włączony w `config/plugins.ts`, ale niezbudowany, jest pomijany bez słowa – loader sprawdza, czy plik z `exports` istnieje, i jeśli nie, przechodzi dalej. Bez `dist/` polecenie `npx strapi routes:list` nie pokazało żadnej trasy pluginu, po `npm run build` – pięć. Do tego watcher `strapi develop` ignoruje `**/dist/**`, ale reaguje na zmiany w `server/src`. Restart po zapisie pliku źródłowego potrafi wyprzedzić `npm run watch`: w logu serwer wstał 0,4 s przed zapisem nowego `dist/server/index.js` i działał na starym kodzie. Po przebudowie zrestartuj Strapi ręcznie.

Zależności pluginu w projekcie

Szkielet ma w `devDependencies` i `peerDependencies` `@strapi/strapi`, Reacta i Design System. Dokumentacja radzi usunąć `@strapi/strapi` z `devDependencies`, ale npm od wersji 7 i tak instaluje zależności typu peer: `npm install` w katalogu pluginu dodał 1627 pakietów (1,1 GB), w tym drugą kopię `@strapi/strapi` i `@strapi/utils`. To nie tylko strata miejsca. Middleware `strapi::errors` rozpoznaje błędy przez `instanceof errors.ApplicationError`, a klasa z kopii pluginu jest inną klasą niż ta z projektu. `NotFoundError` rzucony przez serwis pluginu dla `PUT /comment-moderation/comments/nope000/approve` wrócił jako `500 InternalServerError`, a `PolicyError` – też jako 500.

Dla pluginu lokalnego rozwiązaniem jest `@strapi/utils` w `peerDependencies` i instalacja bez peerów: `npm install --legacy-peer-deps`. Zostaje 412 pakietów (246 MB) – SDK, TypeScript i typy – a `@strapi/*` rozwiązują się z `node_modules` projektu. Ten sam błąd daje wtedy poprawne `404 NotFoundError`. Po stronie panelu duplikaty bolą mniej, bo konfiguracja Vite w Strapi 5.56 (październik 2026) deduplikuje Reacta, Design System i `@strapi/strapi`.

Strona serwerowa

`server/src/index.ts` eksportuje obiekt z kluczami `register`, `bootstrap`, `destroy`, `config`, `controllers`, `routes`, `services`, `contentTypes`, `policies` i `middlewares`. Nazwa elementu staje się częścią UID-u: serwis `moderation` to `plugin::comment-moderation.moderation`. W pluginie sięgasz po niego przez `strapi.plugin('comment-moderation').service('moderation')`, z aplikacji także przez `strapi.service('plugin::comment-moderation.moderation')`.

Konfiguracja z walidacją

Plugin ma dwie opcje: rozmiar strony kolejki i długość podglądu komentarza. Strapi łączy `default` z `config` z `config/plugins.ts` i przekazuje wynik do `validator`, który może zatrzymać start. Zod z `@strapi/utils` (rozdział 5) daje walidację i typ w jednym:

```ts
// server/src/config/index.ts (bez importu)
const configSchema = z.object({
  pageSize: z.number().int().min(1).max(100),
  previewLength: z.number().int().min(20),
});
export type PluginConfig = z.infer<typeof configSchema>;

export default {
  default: { pageSize: 20, previewLength: 200 } satisfies PluginConfig,
  validator(config: unknown) {
    configSchema.parse(config);
  },
};
```

`pageSize: 500` w `config/plugins.ts` kończy start błędem `Error regarding comment-moderation config: [... "code": "too_big", "maximum": 100, "path": ["pageSize"] ...]` – przy wdrożeniu, a nie przy pierwszym kliknięciu moderatora. Wartości czytasz w serwisie przy każdym wywołaniu, przez `strapi.plugin('comment-moderation').config('pageSize')`, a nie raz przy imporcie modułu.

Serwis na Document Service

Serwis `moderation` pobiera kolejkę, zmienia status i zwraca zatwierdzone komentarze artykułu. Rdzeń kolejki:

```
// server/src/services/moderation.ts (fragment findPending)
const [rows, total] = await Promise.all([
  comments().findMany({
    filters,
    sort: 'createdAt:asc',
    fields: ['body', 'createdAt'],
    populate: { article: { fields: ['title'] }, author: { fields: ['username'] } },
    start: (page - 1) * pageSize,
    limit: pageSize,
  }),
  comments().count({ filters }),
]);
return {
  results: commentRows.parse(rows).map((row) => ({
    documentId: row.documentId,
    body: preview(row.body),
    createdAt: row.createdAt,
    article: row.article?.title ?? null,
    author: row.author?.username ?? null,
  })),
  pagination: { page, pageSize, total },
};
```

`comments()` to `strapi.documents('api::comment.comment')`, a `commentRows` – schemat Zod z polami, których plugin oczekuje od komentarza. Plugin kompiluje się osobno i nie widzi `types/generated` aplikacji, więc wynik `findMany` to dla niego `AnyDocument`, a `tsc` odrzucił pierwszą wersję z rzutowaniem na własny typ. Parsowanie na granicy zastępuje rzutowanie i sprawdza, czy aplikacja ma komentarz w oczekiwanym kształcie. Odpowiedź składa się z jawnie wybranych pól (z autora tylko `username`), więc nie potrzebuje sanityzacji z rozdziału 5.

Trasy admina i Content API

Szkielet eksportuje nazwane routery – format, który dokumentacja zaleca, gdy plugin ma oba rodzaje tras:

```
// server/src/routes/admin/index.ts (fragment)
const canModerate = {
  name: 'admin::hasPermissions',
  config: { actions: ['plugin::comment-moderation.moderate'] },
};
const onlyPending = {
  name: 'plugin::comment-moderation.comment-in-status',
  config: { statuses: ['pending'] },
};
// one of the routes of the { type: 'admin' } router
{ method: 'PUT' as const, path: '/comments/:id/approve', handler: 'moderation.approve',
  config: { policies: ['admin::isAuthenticatedAdmin', canModerate, onlyPending] } },
```

Router `content-api` ma `GET /articles/:articleId/comments` i `POST /comments/:id/report`. `npx strapi routes:list` pokazuje, gdzie Strapi je zamontował:

```
GET   /api/comment-moderation/articles/:articleId/comments
POST  /api/comment-moderation/comments/:id/report
GET   /comment-moderation/comments/pending
PUT   /comment-moderation/comments/:id/approve
PUT   /comment-moderation/comments/:id/reject
```

Tylko Content API stoi za `/api`. Trasy admina uwierzytelnia JWT administratora i sprawdzają polityki RBAC. Trasy Content API podlegają users-permissions z akcjami w rodzaju `plugin::comment-moderation.comments.findApproved` (plugin, kontroler, akcja). Redakcja nadaje je rolom w `bootstrap` aplikacji wzorcem z seeda (rozdział 7) – o otwarciu tras dla Public decyduje projekt, nie plugin.

Polityka pluginu

Zatwierdzić i odrzucić można tylko komentarz z kolejki, a zgłosić – tylko zatwierdzony. To jedna reguła z różnym parametrem, więc jedna polityka z konfiguracją per trasa:

```
// server/src/policies/comment-in-status.ts (rdzeń)
const allowed = config.statuses ?? [];
const documentId = policyContext.params.id;
if (!documentId || allowed.length === 0) return false;
const status = await strapi.plugin('comment-moderation')
  .service<ModerationService>('moderation').getStatus(documentId);
// Unknown comment: the controller answers with its regular 404.
if (status === null || allowed.includes(status)) return true;
throw new errors.PolicyError('Comment is not in an allowed moderation status', {
  policy: 'comment-in-status', moderationStatus: status, allowed,
});
```

Polityka trafia do obiektu `policies` pod kluczem `comment-in-status` i ma pełną nazwę `plugin::comment-moderation.comment-in-status`, której dokumentacja każe używać także wewnątrz pluginu. Middleware pluginu deklarujesz tak samo, w obiekcie `middlewares`; mechanika jest ta z rozdziału 4.

> **Pułapka:** strony dokumentacji o trasach i politykach pluginów zapisują konfigurację polityki jako `{ name, options: { ... } }`. W Strapi 5.56 walidator tras przepuszcza `options`, ale rejestr polityk (`registries/policies.js`) przekazuje polityce tylko pole `config`. Z `options` polityka dostała pusty obiekt, odrzuciła każde zgłoszenie i klient zobaczył ogólne `403 "Policy Failed"`. Pisz `config`, jak w rozdziale 4 – także dla `admin::hasPermissions`.

Uprawnienia RBAC w `bootstrap`

W panelu o dostępie decydują role administratorów (RBAC), a nie role users-permissions. Plugin rejestruje akcje w `bootstrap`, gdy serwis uprawnień admina jest gotowy:

```
// server/src/bootstrap.ts (fragment)
await strapi.admin.services.permission.actionProvider.registerMany([
  { section: 'plugins', displayName: 'Read the moderation queue', uid: 'read', pluginName: 'comment-moderation' },
  { section: 'plugins', displayName: 'Approve and reject comments', uid: 'moderate', pluginName: 'comment-moderation'
},
]);
```

Akcje `plugin::comment-moderation.read` i `.moderate` pojawiają się w Settings → Roles → Plugins; podział pozwala dać stażyście podgląd kolejki bez prawa decyzji. Test z Ewą jako Super Admin i Olą z rolą Editor – to konta panelu, niezależne od kont aplikacji (dwa światy użytkowników opisuje rozdział 7):

```
GET /comment-moderation/comments/pending  bez tokenu / JWT Anny (users-permissions) → 401
GET ...   JWT Ewy (admin)       → 200 {"results":[...3...],"pagination":{"page":1,"pageSize":10,"total":3}}
GET ...   Ola, Editor           → 403 PolicyError "Policy Failed";  po nadaniu Editorowi akcji read → 200
PUT /comment-moderation/comments/:id/approve   Ola (tylko read)              → 403 "Policy Failed"
PUT ...approve   Ewa, komentarz odrzucony → 403 {"policy":"comment-in-status","moderationStatus":"rejected","allowed":
["pending"]}
PUT ...approve   Ewa, nieznany documentId → 404 NotFoundError "Comment not found"
POST /api/comment-moderation/comments/:id/report   bez tokenu → 403;  Piotr → 200 {"moderationStatus":"pending"}
```

`pageSize` równe 10 pochodzi z `config/plugins.ts` projektu. Token users-permissions na trasie admina daje 401, a nie 403, bo trasy panelu nie znają strategii JWT aplikacji.

> **Pułapka:** uprawnienia RBAC pluginu żyją w bazie tylko tak długo, jak plugin jest ładowany. Przy każdym starcie `bootstrap` panelu (`cleanPermissionsInDatabase`) usuwa z ról i tokenów administratora akcje, których nikt nie zarejestrował. Wystarczyło jedno `npx strapi routes:list` z niezbudowanym pluginem, żeby z tabeli `admin_permissions` zniknęły wszystkie cztery wpisy `plugin::comment-moderation.*`: akcja `read` roli Editor, uprawnienie tokenu MCP i obie akcje Super Admina. Super Admin odzyskuje je po przywróceniu pluginu, role i tokeny – nie. Wdrożenie z niezbudowanym albo chwilowo wyłączonym pluginem kasuje ręczną konfigurację uprawnień.

Strona admina

Wejście panelu (`admin/src/index.ts`) w `register` wywołuje `app.addMenuLink()` z adresem `plugins/comment-moderation`, leniwie ładowanym komponentem `() => import('./pages/App')` i `permissions: PERMISSIONS.read`, więc rola bez akcji `read` nie zobaczy linku. `PERMISSIONS` to plik z tablicami `[{ action: 'plugin::comment-moderation.read', subject: null }]`, wspólny dla menu, strony i przycisków. Ukryty link to wygoda, a nie zabezpieczenie: `App` owija trasy w `<Page.Protect`

`permissions={PERMISSIONS.read}>`, bo adres strony można wpisać ręcznie, a ostatnią linią obrony są polityki tras serwera. Strona kolejki rozmawia z serwerem przez `useFetchClient`, który dokleja token administratora, odświeża go przy 401 i anuluje żądania przy odmontowaniu. Ścieżki tras admina pluginu podajesz bez `/api`:

```
// admin/src/pages/HomePage.tsx (fragment)
const { get, put } = useFetchClient();
const { toggleNotification } = useNotification();
const {
  isLoading: isLoadingPermissions,
  allowedActions: { canModerate },
} = useRBAC({ moderate: PERMISSIONS.moderate });
// load() calls get('/comment-moderation/comments/pending') and stores the result in state

const decide = async (documentId: string, decision: Decision) => {
  try {
    await put(`/comment-moderation/comments/${documentId}/${decision}`);
    toggleNotification({ type: 'success', message: decision === 'approve' ? 'Comment approved' : 'Comment rejected'
});
  } catch {
    toggleNotification({ type: 'danger', message: 'The comment could not be updated' });
  }
  await load();
};
```

Reszta to Design System: `Layouts.Header` z tytułem i liczbą komentarzy w kolejce, `Table` z kolumnami Comment, Article, Author, Created i Decision, a w ostatniej przyciski „Approve” i „Reject” z `disabled={!canModerate}` – Editor z samą akcją `read` widzi kolejkę, ale nie podejmie decyzji. Pusta kolejka pokazuje `EmptyStateLayout`, a po decyzji pojawia się powiadomienie i lista pobiera się od nowa.

Ten opis pochodzi z kodu i buildu, nie z klikania. Przeszły `tsc` pluginu, jego build i `npm run build` projektu (panel w 25–30 s), a bundle panelu zawiera chunk pluginu z adresem `/comment-moderation/comments/pending`. Usuń ze szkieletu `admin/custom.d.ts`: deklaruje `@strapi/design-system` jako moduł bez typów, a bez niego `tsc` sprawdza prawdziwe propsy komponentów.

Rozszerzanie wbudowanego pluginu: `src/extensions`

Czasem potrzebujesz innego zachowania istniejącego pluginu. Redakcja pozwala zalogowanym wgrywać pliki przez `POST /api/upload`, ale nie chce stu plików w jednym żądaniu. Trasa należy do pluginu Upload, więc nie podepniesz do niej middleware w `src/api`. Plik `src/extensions/<plugin>/strapi-server.ts` dostaje obiekt pluginu po załadowaniu i zwraca zmieniony:

```
// src/extensions/upload/strapi-server.ts (rdzeń; limitFiles to middleware trasy z ctx.badRequest)
export default (plugin: UploadPlugin) => {
  const uploadRoute = plugin.routes['content-api'].routes.find(
    (route) => route.method === 'POST' && route.path === '/'
  );
  // Internal structure of a built-in plugin: fail loudly after an upgrade instead of silently.
  if (!uploadRoute) {
    throw new Error('upload extension: content-api route POST / not found');
  }
  uploadRoute.config = {
    ...uploadRoute.config,
    middlewares: [...(uploadRoute.config?.middlewares ?? []), limitFiles],
  };
  return plugin;
};
```

Anna wysyła cztery pliki i dostaje `400 {"name":"BadRequestError","message":"Upload at most 3 files at once","details":{"count":4}}`, pojedynczy plik przechodzi z 201. W Upload 5.56 `routes['content-api']` jest fabryką z getterem `routes`, dzięki któremu zapis z dokumentacji (`routes['content-api'].routes.push(...)`) i powyższe `find` działają. Rozszerzenie opiera się jednak na wewnętrznej strukturze pluginu, której przewodniki migracji nie obejmują – dlatego kod rzuca błąd, gdy trasy nie znajdzie. Rozszerzenia users-permissions opisuje rozdział 7, a panel wbudowanych pluginów zmienisz tylko łatką `patch-package`.

Narzędzie MCP z pluginu

Strapi 5.56 ma wbudowany serwer MCP (Model Context Protocol), włączany w `config/server.ts` opcją `mcp: { enabled: true }` i uwierzytelniany tokenami administratora. To funkcja darmowa: usługa `strapi.ai.mcp` siedzi w `@strapi/core` i nie sprawdza licencji; konfigurację i podłączenie klienta opisuje rozdział 12. Plugin może dołożyć narzędzie, ale tylko w `register()`, póki serwer MCP jest bezczynny:

```
// server/src/register.ts (fragment)
strapi.ai.mcp.registerTool({
  name: 'list_pending_comments',
  title: 'List pending comments',
  description: 'Returns the oldest comments waiting for moderation in the Redakcja magazine.',
  auth: { policies: [{ action: 'plugin::comment-moderation.read' }] },
  resolveInputSchema: () => z.object({ page: z.number().int().min(1).default(1) }),
  resolveOutputSchema: () => z.object({ total: z.number(), comments: z.array(commentSchema) }),
  createHandler: (strapi) => async ({ args }) => {
    const { results, pagination } = await strapi
      .plugin('comment-moderation').service<ModerationService>('moderation').findPending(args.page);
    const comments = results.map(({ documentId, body, article }) => ({ documentId, body, article }));
    return {
      content: [{ type: 'text', text: JSON.stringify(comments) }],
      structuredContent: { total: pagination.total, comments },
    };
  },
});
```

Narzędzie używa tej samej akcji RBAC co ekran w panelu. Token z akcją `read` widzi je w `tools/list` obok narzędzia `log` (dostępnego tylko w trybie develop), a `tools/call` zwraca `{"total":3,"comments":[{"documentId":"hb8mrlw5of8ysd93lyrst7y4","body":"Great article!",...}]}`. Token bez tej akcji widzi tylko `log`, a wywołanie kończy się błędem JSON-RPC `-32602 "Tool list_pending_comments disabled"`.

Publikacja na npm

Gdy drugi magazyn ma zainstalować plugin z rejestru, `npm run build && npm run verify` buduje `dist/` i sprawdza, czy `exports` wskazuje na istniejące pliki. `files: ["dist"]` ze szkieletu trzyma źródła poza paczką, biblioteki aplikacji (`@strapi/*`, React, Design System) zostają w `peerDependencies`, a wersja dla Strapi 5 powinna mieć inny numer główny niż wersja dla Strapi 4. `verify` przeszedł bez uwag; `npm publish` nie był uruchamiany. Po instalacji z npm wpis w `config/plugins.ts` traci `resolve`, a dalszą pracę prowadzisz z `watch:link`.

Podsumowanie

- Plugin to te same trasy, kontrolery, serwisy i polityki co w `src/api`, z przedrostkiem `plugin::<nazwa>.` i własnym `package.json`, buildem, konfiguracją oraz uprawnieniami. Opłaca się przy współdzieleniu między projektami i przy połączeniu serwera z ekranem w panelu.
- `npx @strapi/sdk-plugin init src/plugins/<nazwa>` tworzy szkielet. Plugin lokalny włączasz przez `resolve` w `config/plugins.ts`, paczkę rozwijasz przez `watch:link` i `yalc`.
- Strapi ładuje plugin z `dist/` według `exports`. Niezbudowany plugin jest pomijany po cichu, a watcher `develop` ignoruje `dist/`.
- W `src/plugins` instaluj zależności pluginu z `--legacy-peer-deps` i trzymaj `@strapi/*` w `peerDependencies`; druga kopia `@strapi/utils` zamienia 404 i 403 w 500.
- Konfiguracja to `default` i `validator`; walidacja Zodem zatrzymuje start z czytelnym błędem.
- Router `admin` trafia pod `/<plugin>` i podlega RBAC, router `content-api` – pod `/api/<plugin>` i rolom users-permissions. Konfiguracja polityki w trasie to `config`, nie `options`.
- Akcje RBAC rejestrujesz w `bootstrap` pluginu i chronisz nimi menu, stronę (`Page.Protect`) i przyciski (`useRBAC`). Start aplikacji bez pluginu usuwa te akcje z ról i tokenów.
- `src/extensions/<plugin>/strapi-server.ts` zmienia wbudowany plugin kosztem zależności od jego wnętrza. Narzędzie MCP rejestrujesz w `register()` przez `strapi.ai.mcp.registerTool`, z akcją RBAC jako bramką.

Co dalej

Plugin dodał do panelu własną stronę, ale panel da się zmieniać także bez pluginu i w miejscach, które już istnieją. W rozdziale 10 zajmiesz się `src/admin/app.tsx`, strefami wstrzykiwania w widoku edycji artykułu, API Content Managera i custom fields – polami z własnym komponentem React, rejestrowanymi po stronie serwera i panelu podobnie jak akcje i link w menu tego pluginu.

Źródła

- https://docs.strapi.io/cms/plugins-development/developing-plugins
- https://docs.strapi.io/cms/plugins-development/create-a-plugin
- https://docs.strapi.io/cms/plugins-development/plugin-sdk
- https://docs.strapi.io/cms/plugins-development/plugin-structure
- https://docs.strapi.io/cms/plugins-development/server-api
- https://docs.strapi.io/cms/plugins-development/server-configuration
- https://docs.strapi.io/cms/plugins-development/server-routes
- https://docs.strapi.io/cms/plugins-development/server-policies-middlewares
- https://docs.strapi.io/cms/plugins-development/server-controllers-services
- https://docs.strapi.io/cms/plugins-development/server-getters-usage
- https://docs.strapi.io/cms/plugins-development/server-lifecycle
- https://docs.strapi.io/cms/plugins-development/admin-panel-api
- https://docs.strapi.io/cms/plugins-development/admin-navigation-settings
- https://docs.strapi.io/cms/plugins-development/admin-fetch-client
- https://docs.strapi.io/cms/plugins-development/guides/admin-permissions-for-plugins
- https://docs.strapi.io/cms/plugins-development/plugins-extension
- https://docs.strapi.io/cms/plugins-development/extend-mcp-server
- https://docs.strapi.io/cms/features/strapi-mcp-server
- https://docs.strapi.io/cms/configurations/plugins

Rozdział 10. Panel admina i custom fields

> **Cel rozdziału:** dopasować panel Redakcji do zespołu (język, marka, strona startowa), dodać akcję i wskazówkę w edycji artykułu oraz zbudować własne pole z licznikiem znaków, zarejestrowane po stronie serwera i panelu.
>
> **Kod:** `examples/10-panel-admina/`. **Pakiety poza fundamentem:** brak.

Redaktorki Redakcji pracują po polsku i każdego ranka robią to samo: sprawdzają, ile komentarzy czeka na moderację, poprawiają tytuły SEO i otwierają opublikowany artykuł na stronie, żeby zobaczyć go oczami czytelnika. Panel wita je angielskim interfejsem z logo Strapi, a pole `metaTitle` przyjmie 70 znaków, choć wyszukiwarka utnie tytuł wcześniej.

W rozdziale 9 panel dostał stronę z pluginu. Tym razem zmieniasz go z poziomu projektu: konfiguracją, API Content Managera, strefą wstrzykiwania, widgetem i custom fieldem – polem z własnym komponentem React, które dla bazy pozostaje zwykłym tekstem. Poza custom fieldem wszystko trafia do paczki JavaScript panelu przy `npm run build` (albo `strapi develop`), nie do serwera.

Konfiguracja w `src/admin/app.tsx`

Strapi ignoruje `src/admin/app.example.tsx` z fundamentu, dopóki nie zmienisz nazwy na `app.tsx`. Obiekt `config` opisuje to, co da się zmienić bez pisania komponentów:

```tsx
// src/admin/app.tsx (fragment)
import logo from './extensions/logo.svg';

export default {
  config: {
    locales: ['pl'],
    translations: {
      pl: { 'Auth.form.welcome.title': 'Witaj w Redakcji', 'app.components.LeftMenu.navbrand.title': 'Redakcja' },
    },
    auth: { logo },
    menu: { logo },
    theme: {
      light: { colors: { primary100: '#fdf0ef', primary200: '#f6cfcc', primary500: '#d9534c', primary600: '#b4231f',
primary700: '#8f1a17' } },
      dark: { colors: { primary100: '#3a1514', primary200: '#6b2523', primary500: '#e0726c', primary600: '#e88a85',
primary700: '#f2b3af' } },
    },
    tutorials: false,
    notifications: { releases: false },
  },
};
```

`locales` dodaje języki do wyboru w profilu. Angielskiego nie usuniesz, bo jest domyślny i zapasowy dla brakujących kluczy – a te są: w Strapi 5.56 (październik 2026) polski plik rdzenia panelu ma około 850 z 915 kluczy, Media Library około 245 z 392. `translations` nadpisuje pojedyncze klucze; klucze pluginów mają przedrostek, np. `content-type-builder.plugin.name`. `theme` jest głęboko scalany z motywem Design Systemu, więc podajesz tylko zmienione kolory. Logo z `src/admin/extensions/` Vite wstawił do paczki jako `data:image/svg+xml`; `tsc` potrzebuje do tego importu pliku `src/admin/vite-env.d.ts` z `/// <reference types="vite/client" />`. Logo wgrane w Settings → Global settings → Overview ma pierwszeństwo przed konfiguracją.

Bundler konfigurujesz w `src/admin/vite.config.ts` przez `mergeConfig` z Vite (rozdział 9).

Plik eksportuje też `register(app)` i `bootstrap(app)`. `register` wywołuje się po rejestracji pluginów i dostaje pełny `StrapiApp` – tu dodajesz custom fields i widgety. `bootstrap` działa, gdy pluginy są gotowe, więc tu sięgasz po Content Managera. Dokumentacja radzi zaczynać od `app.tsx` i przenosić kod do pluginu, gdy powtarza się w kilku projektach.

> **Pułapka:** `bootstrap` nie dostaje `StrapiApp`, tylko wycinek: `addComponents`, `addFields`, `addMenuLink`, `addReducers`, `addSettingsLink(s)`, `getPlugin` i `registerHook` (`StrapiApp.bootstrap` w `@strapi/admin`). `app.widgets.register()` w `bootstrap` kończy się „Cannot read properties of undefined”. Dlatego w przykładzie parametr ma typ `Pick<StrapiApp, 'getPlugin'>`.

Akcja „Otwórz na stronie”: API Content Managera

Preview (rozdziały 8 i 11) pokazuje szkic w ramce panelu, a redaktorka chce zwykłego linku do opublikowanej wersji na stronie magazynu. Content Manager ma na to punkty rozszerzeń: `addEditViewSidePanel` (panel w bocznej kolumnie edycji), `addDocumentAction` (akcja w panelu, w menu „More actions” albo w wierszu listy), `addDocumentHeaderAction` (przycisk w nagłówku), `addBulkAction` (akcja na zaznaczonych wpisach) i `addRichTextBlocks` (bloki edytora Blocks). Każde przyjmuje „komponenty opisu” – funkcje, które dostają kontekst dokumentu i zwracają obiekt z etykietą i akcją albo `null`:

```
// src/admin/extensions/open-on-site-action.tsx (bez importów)
const FRONTEND_URL = process.env.STRAPI_ADMIN_FRONTEND_URL ?? 'http://localhost:3000';

export const OpenOnSiteAction: HeaderActionComponent = ({ model, document }) => {
  // Header actions are rendered for every content type, so the action filters itself out.
  if (model !== 'api::article.article') {
    return null;
  }
  const slug: unknown = document?.slug;
  const hasPublishedVersion = document !== undefined && document.status !== 'draft';
  return {
    label: 'Otwórz na stronie',
    icon: <ExternalLink />,
    disabled: typeof slug !== 'string' || !hasPublishedVersion,
    onClick: () => {
      window.open(`${FRONTEND_URL}/articles/${String(slug)}`, '_blank', 'noopener');
    },
  };
};
```

W `bootstrap` rzutujesz `app.getPlugin('content-manager').apis` (w typach `unknown`) na `ContentManagerPlugin['config']['apis']` z `@strapi/content-manager/strapi-admin` i wywołujesz `addDocumentHeaderAction([OpenOnSiteAction])`. Content Manager renderuje opis jak komponent React, więc wolno w nim używać hooków. Przycisk trafia do nagłówka edycji każdego typu treści, stąd `null` dla wszystkiego poza artykułem. Pole `document.status` przyjmuje `draft`, `published` albo `modified`; curl na `/content-manager/collection-types/api::article.article` pokazał dla artykułów z seeda trzy razy `published` i raz `draft` – ten ostatni ma przycisk wyszarzony. Przykłady w dokumentacji piszą `onClick: ({ documentId }) => ...`, ale w 5.56 `onClick` dostaje zdarzenie (`React.SyntheticEvent`); kontekst przekazujesz przez domknięcie.

Adres frontendu daje zmienna `STRAPI_ADMIN_FRONTEND_URL`. Build wstawia do paczki obiekt `process.env` złożony tylko ze zmiennych z przedrostkiem `STRAPI_ADMIN_` (i kilku własnych Strapi). Po `STRAPI_ADMIN_FRONTEND_URL=https://redakcja.example npm run build` adres był w `dist/build/strapi-*.js`. Zmiana wymaga nowego buildu, a wszystko z tym przedrostkiem jest publiczne – żadnych sekretów.

Strefa wstrzykiwania

API Content Managera zastąpiło w v5 większość stref wstrzykiwania (injection zones), w które plugin dokłada dowolny komponent. Zostały tam, gdzie API nie sięga, np. `editView.right-links`. Redakcja wstawia tam przypomnienie o okładce:

```
// src/admin/extensions/cover-reminder.tsx (rdzeń)
export const CoverReminder = () => {
  const { model } = useContentManagerContext(); // unstable_useContentManagerContext
  const { value: cover } = useField('cover');
  if (model !== 'api::article.article' || cover) return null;
  return <Typography variant="pi" textColor="warning600">Artykuł nie ma okładki...</Typography>;
};
```

W `bootstrap` podpinasz go przez `contentManager.injectComponent('editView', 'right-links', { name: 'redakcja-cover-reminder', Component: CoverReminder })`. Komponent strefy dostaje tylko prop `slug`, a dane bierze z hooków: `useField` czyta bieżącą, niezapisaną wartość pola, więc komunikat znika po wybraniu okładki. Kolumny listy wpisów i układ formularza zmieniają z kolei predefiniowane hooki panelu (`registerHook`).

> **Pułapka:** dokumentacja umieszcza `editView.right-links` „między przyciskami Configure the view i Edit”, co pasuje do układu ze Strapi 4. W 5.56 `Panels.mjs` Content Managera renderuje strefę w panelu „Entry”, pod przyciskami akcji dokumentu, a `editView.informations` nie renderuje nigdzie, choć jest na liście stref. Zanim zaplanujesz ekran pod strefę, sprawdź w `node_modules/@strapi/content-manager`, gdzie pada `InjectionZone area=`.

Widget na stronie głównej

Domyślnych widgetów strony głównej (ostatnie wpisy, profil, statystyki) nie usuniesz, ale możesz dodać własne. Redakcja dokłada kolejkę moderacji:

```
// src/admin/app.tsx, register() (fragment)
app.widgets.register({
  id: 'pending-comments',
  icon: Discuss,
  title: { id: 'redakcja.widget.pending-comments', defaultMessage: 'Komentarze do moderacji' },
  component: async () => (await import('./extensions/pending-comments-widget')).default,
  link: { label: { id: 'redakcja.widget.link', defaultMessage: 'Wszystkie komentarze' },
    href: '/content-manager/collection-types/api::comment.comment' },
  permissions: [{ action: 'plugin::content-manager.explorer.read', subject: 'api::comment.comment' }],
});
```

Strapi rysuje ramkę, ikonę, tytuł i link, a twój komponent wypełnia środek. `icon` jest obowiązkowa (bez niej: `An icon must be provided`). Komponent ładuje się leniwie, `permissions` ukrywa widget przed rolą bez odczytu komentarzy, a bez `pluginId` widget dostaje UID `global::pending-comments`. Dane pobiera przez `useFetchClient` (rozdział 9) z API Content Managera, czyli z uprawnieniami zalogowanej osoby, a stany ładowania, błędu i pustej kolejki rysują `Widget.Loading`, `Widget.Error` i `Widget.NoData` z `@strapi/strapi/admin`. Zapytanie widgetu (`filters[moderationStatus][$eq]=pending`, `sort=createdAt:ASC`) wysłane curl-em z tokenem administratora po dodaniu dwóch komentarzy zwróciło `"total": 2` i oba komentarze, od najstarszego.

Custom field: tytuł SEO z licznikiem

Redaktorka chce widzieć przy `metaTitle` licznik znaków i ostrzeżenie po przekroczeniu zalecanych 60, bez blokowania zapisu. To zadanie dla custom fielda: pola, które w Content-Type Builderze i formularzu wygląda jak nowy typ, a w bazie jest jednym z istniejących. Nowego typu danych nie doda, nie oprze się też na relacji, mediach, komponencie ani dynamic zone.

Pole rejestrujesz dwa razy. Serwer musi je znać, zanim wczyta schematy, więc rejestracja trafia do `register` w `src/index.ts`:

```
// src/index.ts
register({ strapi }: { strapi: Core.Strapi }) {
  strapi.customFields.register([
    { name: 'seo-title', type: 'string', inputSize: { default: 12, isResizable: true } },
    { name: 'seo-description', type: 'text', inputSize: { default: 12, isResizable: true } },
  ]);
},
```

Pola bez pluginu dostają UID-y `global::seo-title` i `global::seo-description`. Typ bazowy jest stały dla rejestracji, dlatego tytuł (`string`) i opis (`text`) to dwa pola z jednym komponentem. W `shared.seo` atrybut dostaje `type: "customField"` i UID pola, a walidacje zostają:

```
"metaTitle": { "type": "customField", "customField": "global::seo-title",
  "required": true, "maxLength": 70, "options": { "recommendedLength": 60 } }
```

Przy starcie Strapi zamienia `customField` na typ bazowy (`convertCustomFieldType` w `@strapi/core`), więc kolumna się nie zmienia i tytuły z seeda przetrwały bez migracji. W panelu rejestracja idzie w `register` z `app.tsx`, z leniwie ładowanym komponentem:

```
// src/admin/app.tsx, register(), for each of the two fields (fragment)
app.customFields.register({
  name: field.name,
  type: field.type,
  icon: Search,
  intlLabel: { id: `redakcja.${field.name}.label`, defaultMessage: field.label },
  intlDescription: { id: `redakcja.${field.name}.description`, defaultMessage: 'Pole z licznikiem znaków' },
  components: {
    Input: async () => {
      const { default: SeoLengthInput } = await import('./extensions/seo-length-input');
      // The registry types Input without props; the Content Manager passes them at runtime.
      return { default: SeoLengthInput as ComponentType };
    },
  },
});
```

`components.Input` jest otypowany jako komponent bez propsów, więc `tsc` odrzucił komponent z wymaganymi propsami – stąd rzutowanie. Pełny plik ma jeszcze `options.base` z ustawieniem „Zalecana długość” (`name: 'options.recommendedLength'`, `type: 'number'`) w Content-Type Builderze. Rejestr wymaga nazw opcji z przedrostkiem `options.`, ale typ `CustomFieldOptionName` w 5.56 zna tylko nazwy wbudowane, jak `maxLength`, więc linia ma `@ts-expect-error`, który sam zgłosi się do usunięcia po poprawce typu.

Komponent czyta i zmienia wartość przez formularz Content Managera:

```
// src/admin/extensions/seo-length-input.tsx (rdzeń)
const { value, error, onChange } = useField<string>(name);
const fieldRef = useFocusInputField<HTMLInputElement & HTMLTextAreaElement>(name);
const text = value ?? '';
const recommended = attribute.options?.recommendedLength ?? DEFAULT_RECOMMENDED_LENGTH;
const inputProps = { ref: fieldRef, name, value: text, disabled, onChange: handleChange };
return (
  <Field.Root name={name} hint={hint} error={error} required={required}>
    <Flex justifyContent="space-between">
      <Field.Label>{label}</Field.Label>
      <Typography variant="pi" textColor={text.length > recommended ? 'danger600' : 'neutral600'}>
        {text.length} / {recommended}
      </Typography>
    </Flex>
    {attribute.type === 'text' ? <Textarea {...inputProps} /> : <TextInput {...inputProps} />}
    <Field.Hint />
    <Field.Error />
  </Field.Root>
);
```

`handleChange` wywołuje `onChange(name, event.target.value)`. Dokumentacja opisuje propsy `value`, `onChange` i `intlLabel` oraz radzi `forwardRef` dla fokusu, ale `InputRenderer` w 5.56 podaje etykietę jako tekst `label` i nie przekazuje `ref`. Hooki `useField` i `useFocusInputField` z `@strapi/strapi/admin` dają wartość i fokus jawnie, z typami. `attribute` zawiera typ bazowy i opcje: `/content-manager/init` zwrócił dla `metaTitle` `{"type":"string","customField":"global::seo-title","required":true,"maxLength":70,"options":{"recommendedLength":60}}`.

Walidację robi serwer na podstawie typu bazowego; licznik to tylko wskazówka, a `recommendedLength` serwer pomija. Zapis przez REST z pełnym tokenem API:

```
PUT /api/articles/i7cdjb2elborwn2s23dr4e7h  {"data":{"seo":{"metaTitle":"Policies in Strapi 5: who may edit what",
...}}}
200 "seo": {"id": 10, "metaTitle": "Policies in Strapi 5: who may edit what", "metaDescription": "How route policies
..."}
PUT ... {"data":{"seo":{"metaTitle":"<71 znaków>"}}}  → 400 "seo.metaTitle must be at most 70 characters"
PUT ... {"data":{"seo":{"metaDescription":"no title"}}}  → 400 "seo.metaTitle must be a `string` type, but the final
value was: `null`."
```

> **Pułapka:** obie rejestracje muszą się zgadzać. Bez serwerowej Strapi nie wstaje: `Error: Could not find Custom Field: global::seo-title`. Bez rejestracji w panelu serwer działa, a formularz pokazuje zablokowane pole z tekstem `Unsupported field type: global::seo-title` (`NotSupportedField` w `@strapi/admin`). Pole z pluginu ma UID `plugin::<plugin>.<nazwa>`, a `plugin` na serwerze i `pluginId` w panelu muszą mieć tę samą wartość.

Build (około 20 s) wydzielił chunki `seo-length-input-*.js` i `pending-comments-widget-*.js`. Opis zachowania panelu pochodzi z kodu, typów i buildu, nie z klikania.

Własny edytor treści

Edytor WYSIWYG pola `richtext` (Markdown) wymienisz pluginem z Marketplace (np. z CKEditorem) albo custom fieldem na typie `richtext`, zbudowanym jak licznik SEO. Pole `blocks` z `shared.rich-text` rozszerzasz przez `addRichTextBlocks` w `register`: dodajesz bloki albo usuwasz wbudowane, jak `code` – a frontend musi umieć je wyrenderować.

Panel pod innym adresem

Inną ścieżkę ustawiasz w `config/admin.ts` przez `url: '/dashboard'`. Od Strapi 5.51 ciasteczko sesji panelu ma domyślnie ścieżkę `/admin`, więc dodaj `auth.cookie.path: '/dashboard'` i przebuduj panel – inaczej, według dokumentacji, każde żądanie po zalogowaniu wraca do ekranu logowania.

Panel na osobnym hoście to statyczne pliki z `dist/build`, np. na CDN, i Strapi jako samo API: absolutne `url` w `config/server.ts` oraz `url: '/'` i `serveAdminPanel: false` w `config/admin.ts`. Z `serveAdminPanel: false` adres `/admin` i pliki panelu zwracały 404, a API panelu działało (`/admin/users/me` z tokenem administratora – 200). Wdrożenia na dwóch domenach nie uruchamiałem; sprawdź przy nim `auth.cookie` (`domain`, `sameSite`), bo sesja panelu żyje w ciasteczkach httpOnly. Reverse proxy i HTTPS – rozdział 13.

Podsumowanie

- `config` w `src/admin/app.tsx` ustawia języki, tłumaczenia, motyw, logo i powiadomienia; polskie braki spadają do angielskiego.
- `register` panelu dostaje pełny `StrapiApp` (custom fields, widgety), `bootstrap` – wycinek z `getPlugin` dla Content Managera.
- Akcje Content Managera to komponenty zwracające opis, renderowane dla każdego typu treści – filtruj po `model`. Położenie stref wstrzykiwania w 5.56 różni się od dokumentacji.
- Widget to leniwy komponent z ikoną i uprawnieniami; dane bierze z API Content Managera przez `useFetchClient`.
- Custom field to istniejący typ danych z własnym komponentem, rejestrowany na serwerze i w panelu; walidację robi serwer według typu bazowego.
- Zmienne `STRAPI_ADMIN_*` są wpisywane w paczkę przy buildzie i publiczne. `serveAdminPanel: false` wyłącza serwowanie plików panelu, nie jego API.

Co dalej

Przycisk „Otwórz na stronie” prowadzi na stronę magazynu, której jeszcze nie ma. W rozdziale 11 zbudujesz ją w Next.js, z odświeżaniem cache po publikacji i podglądem szkiców.

Źródła

- https://docs.strapi.io/cms/admin-panel-customization
- https://docs.strapi.io/cms/admin-panel-customization/locales-translations
- https://docs.strapi.io/cms/admin-panel-customization/theme-extension
- https://docs.strapi.io/cms/admin-panel-customization/logos
- https://docs.strapi.io/cms/admin-panel-customization/bundlers
- https://docs.strapi.io/cms/admin-panel-customization/extension
- https://docs.strapi.io/cms/admin-panel-customization/homepage
- https://docs.strapi.io/cms/admin-panel-customization/wysiwyg-editor
- https://docs.strapi.io/cms/admin-panel-customization/host-port-path
- https://docs.strapi.io/cms/configurations/admin-panel
- https://docs.strapi.io/cms/features/admin-panel
- https://docs.strapi.io/cms/features/custom-fields
- https://docs.strapi.io/cms/plugins-development/admin-panel-api
- https://docs.strapi.io/cms/plugins-development/content-manager-apis
- https://docs.strapi.io/cms/plugins-development/admin-injection-zones
- https://docs.strapi.io/cms/plugins-development/admin-hooks

Rozdział 11. Integracja z frontendem (Next.js / Nuxt)

> **Cel rozdziału:** połączyć Strapi z frontendem w Next.js tak, żeby strona była szybka dzięki cache, odświeżała się po publikacji, pokazywała szkice redaktorom i logowała czytelników bez wystawiania tokenów w przeglądarce.
>
> **Kod:** `examples/11-frontend/`. **Pakiety poza fundamentem:** w projekcie Next.js `npm install qs` i `npm install -D @types/qs`.

Lista artykułów „Redakcji" nie zmienia się co sekundę, więc nie ma powodu pytać Strapi przy każdym wejściu. Redakcja oczekuje jednak, że opublikowany tekst pojawi się od razu, a przycisk podglądu w panelu (rozdział 8) pokaże szkic, którego nikt inny jeszcze nie widzi. Do tego czytelnicy logują się, a API token frontendu nie może trafić do przeglądarki.

Przykład to minimalny projekt Next.js 16.3 z App Routerem (październik 2026), uruchomiony przez `next build` i `next start` na porcie 3011 obok Strapi z rozdziałów 7 i 8. Wszystkie wyniki poniżej pochodzą z tego zestawu. Zapytania REST i `qs` opisuje rozdział 3, sesje z refresh tokenami – rozdział 7.

Serwer z tokenem, przeglądarka bez

Kod w Server Components i route handlerach wykonuje się na serwerze Next.js, więc może używać API tokenu. Token podglądu jest potrzebny, bo middleware `published-only` z rozdziału 7 przepuszcza `status=draft` tylko dla strategii `content-api-token`. Ten sam rozdział pokazał, że token read-only czyta adresy e-mail użytkowników, dlatego frontend dostaje token typu Custom z samymi `find` i `findOne` na artykułach i kategoriach (`GET /api/users` z nim daje 403). Całe pobieranie siedzi w jednym module:

```
// lib/strapi.ts (fragment)
import 'server-only';

async function strapiGet<T>(path: string, query: object, tags: string[]): Promise<T> {
  const { isEnabled: isDraft } = await draftMode();
  const search = qs.stringify({ ...query, status: isDraft ? 'draft' : 'published' }, { encodeValuesOnly: true });
  const response = await fetch(`${STRAPI_URL}/api/${path}?${search}`, {
    headers: { Authorization: `Bearer ${API_TOKEN}` },
    cache: 'force-cache',
    next: { tags, revalidate: 300 },
  });
  if (!response.ok) {
    throw new Error(`Strapi ${response.status} for /api/${path}: ${await response.text()}`);
  }
  return (await response.json()) as T;
}
```

`import 'server-only'` sprawia, że import modułu w komponencie klienckim przerywa build, więc token nie wycieknie przez pomyłkę. W Next 16 `fetch` domyślnie niczego nie cache'uje; `force-cache` i `next.tags` trzeba podać jawnie. `next build` oznaczył stronę główną jako statyczną z `Revalidate 5m`, a odpowiedź ma nagłówek `x-nextjs-cache: HIT`.

Typy odpowiedzi opisują zapytania, nie typy treści: `ArticleCard` dla listy, `ArticleDetail` ze strefą dynamiczną jako unią po `__component`. Typy generowane przez Strapi (rozdział 12) opisują schemat serwera, a kształt odpowiedzi zależy od `fields` i `populate`; specyfikacja OpenAPI ma usterki z rozdziału 3. Ręczny typ obok zapytania to realistyczny wybór. `@strapi/client` też tu nie pasuje: metody `collection()` przyjmują tylko parametry zapytania, więc nie przekażesz im `next.tags`.

Komentarze pod artykułem ładuje komponent kliencki (`comments.tsx`) zwykłym `fetch` na `NEXT_PUBLIC_STRAPI_URL`, bez tokenu. Widzi tylko to, na co pozwala rola Public, i jako pierwszy potrzebuje CORS.

Rewalidacja po publikacji

Webhook z rozdziału 8 wysyła `entry.publish`, `entry.unpublish` i `entry.delete` z nagłówkiem `X-Webhook-Secret`. Odbiera go route handler, który unieważnia tag, a następne żądanie strony pobiera świeże dane:

```
// app/api/revalidate/route.ts (fragment)
export async function POST(request: Request) {
  if (!secretMatches(request.headers.get('x-webhook-secret'), process.env.STRAPI_WEBHOOK_SECRET)) {
    return Response.json({ error: 'Invalid secret' }, { status: 401 });
  }
  const payload = (await request.json()) as StrapiWebhook;
  if (!payload.model) {
    return Response.json({ revalidated: false, reason: 'no model in payload' });
  }
  // One tag per content type: a publication refreshes the list and every page of that type.
  const tag = `strapi:${payload.model}`;
  revalidateTag(tag, { expire: 0 });
  return Response.json({ revalidated: true, tag });
}
```

`secretMatches` porównuje skróty SHA-256 przez `timingSafeEqual`. HMAC z sekcji „Verifying signatures" dokumentacji musiałby liczyć nadawca, a runner webhooków Strapi wysyła tylko statyczne nagłówki. Jeden tag na typ treści odświeża listę i wszystkie artykuły naraz – przy kilkuset tekstach to tańsze niż pilnowanie tagów per slug, który może się zmienić. Piotr publikuje zmieniony tytuł przez REST:

```
GET  localhost:3011/          → "Fetching Strapi from Next.js"         x-nextjs-cache: HIT
PUT  localhost:1311/api/articles/stjg4i8olo7vp0jlfofgw74b {"data":{"title":"Fetching Strapi from Next.js 16"}} → 200
log Next: [revalidate] entry.publish fetching-strapi-from-next-js -> strapi:article
GET  localhost:3011/          → "Fetching Strapi from Next.js 16"      x-nextjs-cache: MISS, potem HIT
POST localhost:3011/api/revalidate (zły sekret) → 401 {"error":"Invalid secret"}
```

Zapis szkicu (`?status=draft`) nie wysyła webhooka i nie zmienia strony, co jest zgodne z intencją.

> **Pułapka:** dokumentacja Next.js zaleca `revalidateTag(tag, 'max')`, a jednoargumentowe `revalidateTag(tag)` jest przestarzałe. Profil `max` oznacza stale-while-revalidate: po publikacji z `'max'` pierwsze żądanie zwróciło stary tytuł, dopiero drugie nowy. Redaktor, który odświeża stronę zaraz po publikacji, uzna, że webhook nie działa. Dla webhooka z CMS-a użyj `{ expire: 0 }`.

Druga niespodzianka dotyczy self-hostingu. Domyślny cache handler trzyma unieważnione tagi w pamięci procesu, a dane `fetch` na dysku w `.next/cache`. Po webhooku i restarcie `next start` (a nawet po ponownym `next build` na tej samej maszynie) strona wróciła do tytułu sprzed publikacji, choć Strapi miał już nowy – aż do kolejnej rewalidacji czasowej. Przy kilku replikach `revalidateTag` działa tylko na tej, która odebrała webhook. Dlatego `revalidate: 300` zostaje jako siatka bezpieczeństwa, razem z tym, że Strapi nie ponawia nieudanych webhooków. Pełne rozwiązanie to wspólny cache handler, np. na Redisie (dokumentacja Next.js, „Self-hosting").

W Dockerze Strapi wysyła webhook na adres w sieci wewnętrznej, np. `http://frontend:3000/api/revalidate`. W produkcji panel odrzuci taki adres jako prywatny (rozdział 8), więc tworzysz webhook kodem w `bootstrap` albo kierujesz go na publiczny URL frontendu.

Podgląd szkiców

Handler `preview` w `config/admin.ts` (rozdział 8) zwraca panelowi adres `/api/preview?url=/artykuly/<slug>&secret=...&status=draft`. Route handler w Next sprawdza sekret, włącza draft mode i przekierowuje:

```
// app/api/preview/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  if (!secretMatches(searchParams.get('secret'), process.env.PREVIEW_SECRET)) {
    return new Response('Invalid preview secret', { status: 401 });
  }
  // Only paths of this site, never another origin: the URL comes from a link that can be copied.
  const url = searchParams.get('url') ?? '/';
  if (!url.startsWith('/') || url.startsWith('//')) {
    return new Response('Invalid preview url', { status: 400 });
  }
  const draft = await draftMode();
  if (searchParams.get('status') === 'published') draft.disable();
  else draft.enable();
  redirect(url);
}
```

Draft mode to ciasteczko `__prerender_bypass`. Z nim Next pomija cache `fetch`, a `strapiGet` dokłada `status=draft`. Adres wziąłem z Admin API panelu, a stronę pobrałem z ciasteczkiem i bez niego. Zły sekret daje 401, a `url=//evil.example` – 400:

```
GET /content-manager/preview/url/api::article.article?documentId=stjg4i8o...&status=draft
  → http://localhost:3011/api/preview?url=%2Fartykuly%2Ffetching-strapi-from-next-js&secret=...&status=draft
GET (ten adres) → 307 do /artykuly/fetching-strapi-from-next-js,
  set-cookie: __prerender_bypass=...; Path=/; Secure; HttpOnly; SameSite=none
GET /artykuly/fetching-strapi-from-next-js → z ciasteczkiem "...Next.js (draft v2)", bez "...Next.js 16"
GET /artykuly/self-hosting-strapi-with-docker → z ciasteczkiem 200, bez 404
```

Przykład trasy w dokumentacji Strapi (z demo LaunchPad) woła `draftMode()` bez `await`, czyli API sprzed Next 15, i przekierowuje na `url` bez sprawdzenia. Komentarz w nim ostrzega przed otwartym przekierowaniem, ale kod robi dokładnie to. Ciasteczko `SameSite=none; Secure` pozwala działać podglądowi w ramce panelu. Next nie wysyła `X-Frame-Options`, więc ramka działa bez konfiguracji; jeśli dodasz CSP, wpisz adres panelu do `frame-ancestors`. Baner „Podgląd szkiców” w `layout.tsx` ma formularz `POST /api/preview/exit`, który wyłącza tryb.

Logowanie przez route handler

Rozdział 7 trzymał token odświeżania w ciasteczku `httpOnly` ustawianym przez Strapi. Gdy frontend i API mają różne domeny, ciasteczko z `SameSite=Lax` nie poleci z `fetch` strony. Rozwiązaniem jest pośrednik: przeglądarka loguje się do Next, a Next rozmawia ze Strapi serwer-serwer. Strapi w trybie `httpOnly` oddaje token odświeżania tylko w `Set-Cookie`, więc Next wyjmuje go z nagłówka i zapisuje we własnym ciasteczku:

```
// lib/session.ts (fragment)
const strapiCookie = response.headers
  .getSetCookie()
  .find((cookie) => cookie.startsWith(`${STRAPI_COOKIE}=`));
const refreshToken = strapiCookie?.split(';')[0].slice(STRAPI_COOKIE.length + 1) || null;

// storeRefreshToken(token), called by the login and refresh route handlers
(await cookies()).set(REFRESH_COOKIE, token, {
  httpOnly: true,
  secure: process.env.NODE_ENV === 'production',
  sameSite: 'lax',
  path: '/api/auth',
  maxAge: 30 * 24 * 60 * 60,
});
```

`/api/auth/refresh` w Next czyta to ciasteczko i wysyła je do Strapi w polu `refreshToken` ciała. Strapi szuka tokenu najpierw w podpisanym ciasteczku, którego Next nie ma, a potem w ciele (rozdział 7):

```
POST :3011/api/auth/login {"identifier":"anna","password":"Passw0rd!"}
  → 200 {"jwt":"eyJhbGciOi...","user":{"username":"anna",...}}
    set-cookie: redakcja_refresh=...; Path=/api/auth; Max-Age=2592000; Secure; HttpOnly; SameSite=lax
POST :3011/api/auth/refresh             → 200 {"jwt":"eyJhbGciOi..."} + nowe redakcja_refresh
POST :3011/api/auth/refresh (stare)     → 401 {"error":"Session expired"} + skasowane ciasteczko
POST :3011/api/auth/logout              → 200 {"ok":true}
```

Token dostępu trafia do pamięci przeglądarki i idzie do Strapi w nagłówku `Authorization`. Z nim `GET /api/articles/mine` zwróciło artykuły Piotra. Klient z rozdziału 7 działa dalej, wystarczy zmienić adres odświeżania na `/api/auth/refresh`; jego pojedyncze odświeżanie jest tu jeszcze ważniejsze, bo nieudany refresh kasuje ciasteczko. Wylogowanie w Strapi wymaga ważnego tokenu dostępu, więc `/api/auth/logout` najpierw odświeża sesję, a potem ją zamyka. Server Component może ciasteczka czytać, ale nie ustawiać, więc rotacja tokenu musi przejść przez route handler albo Server Function.

CORS

Domyślne `strapi::cors` ma `origin: '*'` i `credentials: true`, a kod middleware zamienia gwiazdkę na origin z żądania:

```
OPTIONS /api/articles/mine  Origin: https://evil.example
→ 204, Access-Control-Allow-Origin: https://evil.example, Access-Control-Allow-Credentials: true
```

> **Pułapka:** to nie jest „wszyscy, ale bez ciasteczek”, tylko „każda strona, z ciasteczkami”. Przeglądarka z sesją z rozdziału 7 wyśle ciasteczko odświeżania z żądaniem każdej strony, której pozwala na to `SameSite` (np. dowolnej subdomeny tej samej witryny), a ta

> strona odczyta odpowiedź – łącznie z nowym tokenem dostępu z `/api/auth/refresh`. Ustaw listę originów jawnie.

```
// config/middlewares.ts (fragment)
{
  name: 'strapi::cors',
  config: {
    origin: env.array('CORS_ORIGINS', ['http://localhost:3000']),
    credentials: false,
  },
},
```

Po zmianie preflight z `evil.example` dostał 200 bez nagłówków `Access-Control-*`, a z `http://localhost:3011` – zgodę z `Authorization` na liście nagłówków. `credentials: false` pasuje do pośrednika z poprzedniej sekcji. Jeśli przeglądarka ma wysyłać ciasteczko Strapi bezpośrednio (rozdział 7), zostaw `credentials: true`, ale z jawną listą originów.

Obrazy

Lokalny provider uploadu zwraca URL względny (`/uploads/policies_cover_ce810c5af8.png`), więc `mediaUrl()` dokleja adres Strapi. `next/image` optymalizuje tylko obrazy z dozwolonych źródeł:

```
// next.config.ts (fragment)
images: {
  remotePatterns: [
    {
      protocol: strapiUrl.protocol === 'https:' ? 'https' : 'http',
      hostname: strapiUrl.hostname,
      port: strapiUrl.port,
      pathname: '/uploads/**',
    },
  ],
  // Next 16 refuses to optimize images from localhost and private networks (SSRF protection).
  dangerouslyAllowLocalIP: process.env.STRAPI_IMAGES_LOCAL === 'true',
},
```

Bez ostatniej opcji `/_next/image?url=http://localhost:1311/uploads/...` zwrócił 400, a log pokazał `hostname resolved to private IP ["127.0.0.1"]`. Po włączeniu – 200 `image/png`, a ścieżka spoza `/uploads/` nadal 400. To lustrzane odbicie walidatora webhooków w Strapi: w Dockerze z adresem `http://strapi:1337` potrzebujesz tej opcji albo publicznego URL-a obrazów. Provider S3 (rozdział 8) zwraca adresy absolutne i wymaga wpisu z hostem bucketu lub CDN.

> **Nuxt.** Konfiguracja i kod pokazane, nieuruchomione (Nuxt 4.5). Token idzie do `runtimeConfig` (tylko serwer, `NUXT_STRAPI_TOKEN`), adres publiczny do `runtimeConfig.public`. Nie ma odpowiednika `revalidateTag`: cache'ujesz funkcją Nitro i czyścisz jej wpisy w trasie webhooka. `usePreviewMode()` włącza się domyślnie parametrem `?preview=true` w URL-u, więc sekret musi sprawdzić trasa serwera, która dokłada `status=draft`. Obrazy obsługuje `@nuxt/image` z `domains` albo providerem `strapi`.
>
> ts // server/utils/articles.ts export const getArticles = defineCachedFunction(async () => { const { strapiToken, public: { strapiUrl } } = useRuntimeConfig(); return $fetch(`${strapiUrl}/api/articles`, { headers: { Authorization: `Bearer ${strapiToken}` } }); }, { name: 'articles', getKey: () => 'all', maxAge: 300 }); // server/api/revalidate.post.ts export default defineEventHandler(async (event) => { if (getHeader(event, 'x-webhook-secret') !== useRuntimeConfig(event).webhookSecret) { throw createError({ statusCode: 401 }); } await useStorage('cache').removeItem('nitro:functions:articles:all.json'); return { ok: true }; });

Podsumowanie

- Dane pobieraj na serwerze Next z API tokenem typu Custom w module z `import 'server-only'`; przeglądarka pyta Strapi bez tokenu, tylko o to, co widzi rola Public.
- Typy odpowiedzi pisz per zapytanie; `@strapi/client` nie przekaże `next.tags`.
- W Next 16 `fetch` nie cache'uje domyślnie: `cache: 'force-cache'`, `next.tags` i `revalidate` jako zabezpieczenie.
- Webhook trafia do route handlera, który porównuje sekret w stałym czasie i woła `revalidateTag(tag, { expire: 0 })`; `'max'` pokazuje po publikacji stary stan.
- Unieważnienia tagów żyją w pamięci jednej instancji – restart i repliki wymagają rewalidacji czasowej albo wspólnego cache handlera.
- Podgląd: sekret, ścieżka tylko lokalna, `draftMode().enable()`, `status=draft` w zapytaniu.

- Logowanie przez route handlery Next trzyma token odświeżania w ciasteczku `httpOnly` frontendu; Strapi dostaje go w ciele żądania.
- Domyślny CORS Strapi odbija każdy origin z `credentials: true`; `next/image` w Next 16 blokuje adresy prywatne.

Co dalej

Frontend działa, ale wszystko sprawdziłeś ręcznie curl-em. Rozdział 12 zamienia takie kontrole w testy: typy generowane przez `strapi ts:generate-types`, testy polityk i middleware na instancji Strapi w teście oraz narzędzia AI dla dewelopera. Jako ćwiczenie możesz dopisać stronę „Moje artykuły”, która korzysta z `/api/articles/mine` i tokenu z `/api/auth/refresh`.

Źródła

- https://docs.strapi.io/cms/features/preview
- https://docs.strapi.io/cms/backend-customization/webhooks
- https://docs.strapi.io/cms/configurations/middlewares
- https://docs.strapi.io/cms/api/client
- https://docs.strapi.io/cms/features/api-tokens
- https://docs.strapi.io/cms/features/users-permissions
- https://nextjs.org/docs/app/api-reference/functions/revalidateTag
- https://nextjs.org/docs/app/guides/caching-without-cache-components
- https://nextjs.org/docs/app/guides/draft-mode
- https://nextjs.org/docs/app/guides/self-hosting
- https://nextjs.org/docs/app/api-reference/functions/cookies
- https://nextjs.org/docs/app/api-reference/components/image
- https://nuxt.com/docs/4.x/api/composables/use-preview-mode
- https://v2.nitro.build/guide/cache
- https://image.nuxt.com/providers/strapi

Rozdział 12. TypeScript, testy i narzędzia AI

> **Cel rozdziału:** zabezpieczyć reguły „Redakcji” przed cichą regresją typami i testami, które naprawdę uruchamiają Strapi, oraz rozsądnie podłączyć asystenta AI do dokumentacji i do własnej instancji.
>
> **Kod:** `examples/12-typescript-testy-ai/`. **Pakiety poza fundamentem:** `npm install -D vitest@4 supertest @types/supertest`.

Polityka `is-owner`, limit komentarzy i wymuszone `moderationStatus: 'pending'` działają, bo w rozdziałach 4 i 5 sprawdziłeś je curl-em – raz. Literówka w nazwie pola, zmiana kolejności middleware albo cotygodniowa aktualizacja Strapi mogą je po cichu wyłączyć, a dowiesz się o tym od czytelnika, który sam zatwierdził swój komentarz. Ten rozdział dokłada typy, które łapią błędy przed startem serwera, testy, które odtwarzają scenariusze z curl-a na prawdziwej instancji, i narzędzia AI – przydatne, o ile wiesz, do czego dajesz im dostęp.

Typy generowane ze schematów

`strapi.documents('api::article.article')` wie, jakie pola ma artykuł, tylko dzięki plikom `types/generated/contentTypes.d.ts` i `components.d.ts`. Tworzy je `npx strapi ts:generate-types` (w „Redakcji” 0,24 s). `contentTypes.d.ts` rozszerza moduł `@strapi/strapi` o rejestr `ContentTypeSchemas`, z którego korzystają `Core.Strapi`, `Data.ContentType<'api::article.article'>` (dokument z polami) i `Modules.Documents.ServiceParams<uid>` (parametry Document Service, użyte w rozdziale 5). Z typami `findOne({ documentId, fields: ['titel'] })` kończy się błędem `Did you mean '"title"'?`. Bez nich literówka się kompiluje, a poprawne `update({ data: { moderationStatus: 'approved' } })` w serwisie komentarzy zgłasza błąd, bo typy zastępcze nie znają pól. Dlatego `types/generated` trafia do repozytorium razem ze zmianą schematu.

> **Pułapka:** według dokumentacji opcja `autogenerate` w `config/typescript.ts` (domyślnie `false`) włącza generowanie typów przy restarcie. W Strapi 5.56 (październik 2026) `strapi develop` w projekcie TypeScript generuje je zawsze: z `autogenerate: false` i usuniętym `types/generated` serwer i tak je odtworzył (`✔ Generating types (290ms)`). Opcja dotyczy tylko projektów JavaScript i jest w nich domyślnie włączona – tak mówi komentarz w typie `Core.Config.TypeScript`. Typy powstają przy `develop`, nie przy `build`, więc to CI powinno sprawdzić, czy ktoś ich nie zapomniał zatwierdzić.

Polityka i middleware z typami

Rozdział 4 zostawił dwa obejścia: `Core.PolicyContext` nie deklaruje `state` ani `params`, a `Core.PolicyHandler` zwraca synchronicznie `boolean`, choć router czeka na wynik polityki (`await handler(...)` w `@strapi/core`). Najczyściej opisać to raz, w jednym pliku:

```ts
// src/types/request-context.ts
import type { Core, Data } from '@strapi/strapi';
export type AuthUser = Data.ContentType<'plugin::users-permissions.user'>;

// Koa types `ctx.state` as `any`; this makes `ctx.state.user` typed in middlewares and controllers.
declare module 'koa' {
  interface DefaultState {
    user?: AuthUser;
  }
}

export type PolicyContext = Core.PolicyContext & {
  state: { user?: AuthUser };
  params: Record<string, string | undefined>;
};

export type Policy<TConfig = unknown> = (
  policyContext: PolicyContext,
  config: TConfig,
  context: { strapi: Core.Strapi }
) => Promise<boolean | undefined> | boolean | undefined;
```

Polityka `is-owner` zaczyna się teraz od `const isOwner: Policy<IsOwnerConfig> = async (policyContext, config, { strapi }) => {` i nie potrzebuje adnotacji w środku. Dzięki rozszerzeniu modułu `koa` z limitu komentarzy znika rzutowanie `ctx.state.user as { id: number }`. Typ nie odwzorowuje jednak wszystkiego: `Core.PolicyContext` dziedziczy po Koa

`query`, więc `policyContext.query` się kompiluje, choć w polityce ma wartość `undefined` (rozdział 4).

Testy na prawdziwej instancji

Dokumentacja proponuje Jest, Supertest i harness, który łata wewnętrzne moduły Strapi (ładowanie `.ts` z `config/`, sterownik `sqlite3`, którego fundament nie ma, bo używa `better-sqlite3`). Prościej zrobić to, co `strapi develop`: skompilować projekt `tsc` do `dist/` i uruchomić Strapi z tego katalogu. Ten sam krok sprawdza typy.

```
// tests/strapi.ts (core of the harness)
export async function startStrapi(): Promise<Core.Strapi> {
  removeDatabase();
  const strapi = createStrapi({ appDir, distDir: path.join(appDir, 'dist') });
  await strapi.load();
  // listen() would mount the routers; supertest needs them without a fixed port.
  strapi.server.mount();
  return strapi;
}
```

`load()` synchronizuje schemat i uruchamia `register`/`bootstrap` bez otwierania portu; Supertest wystawia serwer na losowym porcie na czas żądania. `tests/env.ts` ustawia sekrety testowe (gdy ich brak – w CI nie ma `.env`) i `DATABASE_FILENAME=.tmp/test-<id workera>.db`, więc pliki testów działają równolegle, każdy na pustej bazie, a baza deweloperska zostaje nietknięta. `config/env/test/server.ts` ścisza logi do `warn` i wyłącza sprawdzanie aktualizacji w npm. Skrypt w `package.json` to `"test": "tsc -p . && vitest run"`.

```
// vitest.config.mts (imports from node:module and vitest/config omitted)
const require = createRequire(import.meta.url);
export default defineConfig({
  resolve: { alias: { '@strapi/strapi': require.resolve('@strapi/strapi') } },
  test: { pool: 'threads', setupFiles: ['tests/env.ts'], hookTimeout: 60_000 },
});
```

> **Pułapka:** obie opcje to obejścia sprawdzone na Strapi 5.56. Bez aliasu Vitest ładuje build ESM `@strapi/core`, który pada w Node na `Directory import '.../lodash/fp' is not supported`; alias wskazuje build CommonJS, ten sam, którego używa CLI. Bez `pool: 'threads'` testy przechodzą, ale przebieg kończy się błędem `Worker forks emitted error ... EPIPE`, bo `strapi.destroy()` wywołuje `process.removeAllListeners()` i odcina kanał IPC procesu potomnego. A `vitest@4`, bo Vitest 5 wymaga `@types/node` co najmniej 22, a szkielet Strapi ma `^20`.

Test polityki to scenariusz z rozdziału 4 zapisany kodem. Funkcje z `tests/fixtures.ts` nadają roli Authenticated uprawnienie `update`, tworzą użytkowników i logują ich przez `POST /api/auth/local`, więc w grze jest prawdziwy JWT:

```
// tests/articles.test.ts (one of three cases)
it('rejects an update from another author with 403 PolicyError', async () => {
  const res = await request(strapi.server.httpServer)
    .put(`/api/articles/${articleId}`)
    .set('Authorization', `Bearer ${piotr.jwt}`)
    .send({ data: { title: 'Hijacked' } });
  expect(res.status).toBe(403);
  expect(res.body.error).toMatchObject({ name: 'PolicyError', details: { policy: 'is-owner' } });
});
```

Testy komentarzy sprawdzają, że nowy komentarz ląduje w bazie jako `pending` (odczyt przez Document Service, nie tylko odpowiedź), `moderationStatus` od klienta daje 400, a czwarty komentarz w minucie – 429 z `Retry-After`. Każdy przypadek ma własnego użytkownika, bo limit liczy się per użytkownik. Test jednostkowy serwisu dostaje fałszywy `strapi` z `contentType()` (wymaga go fabryka `createCoreService`) i `documents()` zwracającym obiekty z `vi.fn()`. Wynik `npm test -- --reporter=verbose` (skrócony):

```
✓ tests/comment-service.test.ts > commentService.submit > creates the comment as pending for the given author 5ms
✓ tests/articles.test.ts > is-owner on PUT /api/articles/:id > rejects an update from another author with 403
PolicyError 20ms
✓ tests/articles.test.ts > is-owner on PUT /api/articles/:id > lets the author update their own article 41ms
✓ tests/comments.test.ts > POST /api/comments > answers the 4th comment within a minute with 429 and Retry-After 239ms
 Test Files  3 passed (3)
      Tests  8 passed (8)
   Duration  5.81s (transform 99ms, setup 77ms, import 3.12s, tests 8.84s, environment 0ms)
```

Czy testy coś łapią? Po dopisaniu `|| true` do warunku w `is-owner` przebieg padł z `expected 200 to be 403`. Test jednostkowy jest szybki, ale nie wie nic o trasach, uprawnieniach roli ani kolejności polityk, więc reguły dostępu sprawdzaj testem API.

W CI wystarczą dwa kroki: typy zgodne ze schematem i zielone testy. Workflow GitHub Actions nie był uruchamiany, ale oba polecenia sprawdziłem lokalnie na kopii projektu bez `.env`: typy wyszły identyczne, testy przeszły. W GitLab CI to te same polecenia w `script:`.

```yaml
# .github/workflows/test.yml
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npx strapi ts:generate-types && git diff --exit-code types/generated
      - run: npm test
```

Narzędzia AI dla dewelopera

Dokumentacja dla asystenta

Model zna Strapi głównie z czasów v4 i podsuwa `entityService`. Dokumentacja ma dla narzędzi AI trzy wejścia. `https://docs.strapi.io/llms.txt` to indeks stron z jednozdaniowymi opisami, a każdą stronę pobierzesz jako czysty Markdown, dopisując `.md` do adresu (`https://docs.strapi.io/cms/testing.md`). `llms-full.txt` i `llms-code.txt` to cała dokumentacja i jej przykłady kodu w jednym pliku – do grepowania, nie do wklejania. Trzecie to Docs MCP, oparty na Kapa serwer odpowiadający z aktualnej dokumentacji:

```
claude mcp add --transport http strapi-docs https://strapi-docs.mcp.kapa.ai
```

Składnię potwierdza `claude mcp add --help` (opcje przed nazwą, domyślny zakres `local`). Anonimowe żądanie do serwera dostaje `401` z nagłówkiem `WWW-Authenticate` wskazującym OAuth w `mcp.kapa.ai`, więc po dodaniu trzeba się zalogować przez `/mcp` w Claude Code; tego kroku nie wykonywałem. Dokumentacja radzi zaczynać pytania od „Use the strapi-docs MCP server to answer:", żeby asystent nie odpowiadał z pamięci.

Wbudowany serwer MCP Strapi

Drugi, darmowy serwer MCP działa w twojej instancji i pozwala klientowi AI czytać i zmieniać treści przez Content Manager. Rozdział 9 pokazał, jak plugin dokłada do niego własne narzędzie; tu chodzi o sam serwer. W „Redakcji" włącza go flaga, domyślnie wyłączona:

```
// config/server.ts (fragment)
mcp: {
  enabled: env.bool('MCP_ENABLED', false),
},
```

Po starcie z `MCP_ENABLED=true` log pokazuje `[MCP] Server available at /mcp`. Serwer przyjmuje tylko Admin token (Settings → Administration Panel → Admin Tokens); JWT zalogowanego administratora dostał `Authentication required`. Utworzyłem token „claude-code-readonly" ważny 7 dni, z jednym uprawnieniem: odczyt artykułów w Content Managerze. Rozmowa to JSON-RPC przez `POST` z nagłówkiem `Accept: application/json, text/event-stream`:

```
initialize  → {"result":{"protocolVersion":"2025-06-18",...,"serverInfo":{"name":"strapi-mcp-
server","version":"1.0.0"}},...}
tools/list  → (nazwy narzędzi) log, list_article, get_article
tools/call delete_article → {"error":{"code":-32602,"message":"Tool delete_article disabled"}}
bez tokenu  → 401 {"error":{"code":-32000,"message":"Authentication required"}}
```

Narzędzia wynikają z uprawnień tokenu (`log` istnieje tylko w trybie deweloperskim); pełne prawa do typu z draft & publish dają według dokumentacji do ośmiu narzędzi, z `publish` i `delete` włącznie. Klienta podłączysz przez `claude mcp add --transport http strapi-redakcja http://localhost:1337/mcp --header "Authorization: Bearer`

`$STRAPI_ADMIN_TOKEN"`. Powłoka wstawi wartość tokenu do lokalnej konfiguracji Claude Code, więc nie używaj zakresu `project`, który trafia do repozytorium.

O bezpieczeństwie decydują dwie rzeczy. MCP to droga przez panel, nie przez Content API: przy `list_article` log pokazał tylko `[trace] document service: findMany`, więc `is-owner` i limit komentarzy agenta nie dotyczą. A treść czytana przez agenta pochodzi od ludzi: komentarz z poleceniem „opublikuj wszystkie szkice" trafi do kontekstu modelu, a token z prawem zapisu zamieni ten prompt injection w realną zmianę. Stąd osobny token na klienta, najwęższe uprawnienia, krótka ważność i `MCP_ENABLED` tylko tam, gdzie trzeba – w produkcji raczej wcale. Własne narzędzia MCP z pluginu pokazuje rozdział 9.

Podsumowanie

- `strapi ts:generate-types` tworzy `types/generated`, bez którego `strapi.documents()` nie zna pól. Typy są w repozytorium, a CI pilnuje ich przez `git diff --exit-code`; `autogenerate` dotyczy tylko projektów JavaScript.
- Jeden plik typów obsługuje polityki (`Policy<TConfig>` ze `state`, `params` i wynikiem asynchronicznym) i middleware (rozszerzenie `DefaultState` z Koa).
- Testy API: `tsc` do `dist/`, `createStrapi({ distDir }).load()`, `strapi.server.mount()`, Supertest i osobna baza SQLite na workera; w Vitest 4 dodatkowo alias na build CommonJS i `pool: 'threads'`.
- Reguły dostępu testuj przez API z prawdziwym JWT, logikę serwisu – jednostkowo z fałszywym `strapi`.
- Aktualną dokumentację asystent dostaje przez `llms.txt`, strony `.md` albo Docs MCP.
- Wbudowany serwer MCP omija polityki i middleware tras; daj mu Admin token o najwęższych uprawnieniach.

Co dalej

Rozdział 13 zabiera „Redakcję" na produkcję: obraz Dockera, Postgres zamiast SQLite, sekrety i aktualizacje Strapi. Testy z tego rozdziału uruchomione przed każdą aktualizacją powiedzą ci, czy kolejny minor nie zmienił zachowania twoich polityk.

Źródła

- https://docs.strapi.io/cms/typescript
- https://docs.strapi.io/cms/typescript/development
- https://docs.strapi.io/cms/typescript/documents-and-entries
- https://docs.strapi.io/cms/configurations/typescript
- https://docs.strapi.io/cms/testing
- https://docs.strapi.io/cms/ai/for-developers
- https://docs.strapi.io/cms/ai/docs-mcp-server
- https://docs.strapi.io/cms/features/strapi-mcp-server
- https://docs.strapi.io/cms/features/admin-tokens
- https://docs.strapi.io/cms/plugins-development/extend-mcp-server
- https://docs.strapi.io/llms.txt

Rozdział 13. Produkcja self-hosted: Docker, Postgres, dane, aktualizacje

> **Cel rozdziału:** zbudować z „Redakcji” obraz Dockera, uruchomić go z Postgresem za Traefikiem, przenieść dane z SQLite, robić kopie i bezpiecznie aktualizować Strapi.
>
> **Kod:** `examples/13-produkcja/`. **Pakiety poza fundamentem:** `npm install pg`.

„Redakcja” działa lokalnie na SQLite i `strapi develop`. Produkcja to VPS z Traefikiem, kontener ze Strapi i kontener z Postgresem. Kod przychodzi z repozytorium, konfiguracja ze zmiennych środowiskowych, a dane – z kopii, którą trzeba przenieść. Content-Type Builder jest w produkcji wyłączony, więc każda zmiana modelu przechodzi przez repo i wdrożenie (rozdział 2).

Wszystko poniżej uruchomiłem lokalnie z Postgresem 17, poza Traefikiem i prawdziwą domeną – zachowanie za proxy zasymulowałem nagłówkiem.

```
Internet ──443──→ Traefik (TLS, Let's Encrypt)
                     │ sieć "web", X-Forwarded-Proto: https
                     ▼
              strapi:1337  (node, uid 1000) ── wolumen public/ (uploads)
                     │ sieć projektu
                     ▼
              db:5432  (postgres:17) ── wolumen db-data
```

Konfiguracja produkcyjna

Rozdział 1 pokazał, że przy `NODE_ENV=production` Strapi scala `config/*` z `config/env/production/*`. W pliku produkcyjnym trzymasz więc tylko różnice. Dla bazy różnica jest zasadnicza – zawsze Postgres, niezależnie od `DATABASE_CLIENT`:

```ts
// config/env/production/database.ts (fragment)
const required = (name: string): string => {
  const value = env(name);
  if (!value) {
    throw new Error(`${name} is required in production`);
  }
  return value;
};

return {
  connection: {
    client: 'postgres',
    connection: {
      host: env('DATABASE_HOST', 'db'),
      port: env.int('DATABASE_PORT', 5432),
      database: required('DATABASE_NAME'),
      user: required('DATABASE_USERNAME'),
      password: required('DATABASE_PASSWORD'),
      ssl: env.bool('DATABASE_SSL', false),
    },
    // Docker drops idle connections, so the pool must be allowed to shrink to zero.
    pool: { min: 0, max: env.int('DATABASE_POOL_MAX', 10) },
  },
};
```

Brak hasła zatrzymuje start z czytelnym komunikatem, zamiast próbować domyślnego `strapi:strapi` z pliku bazowego. Kontener nie dostał zmiennej `DATABASE_CLIENT`, a log startowy i tak pokazał `Database: postgres`. Driver `pg` dodajesz sam – fundament ma tylko `better-sqlite3`.

Plik `config/env/production/server.ts` ustawia publiczny adres (`url: env('PUBLIC_URL')`, z tą samą kontrolą obecności) i zaufanie do proxy (`proxy: { koa: true, maxIpsCount: 1 }`) – do tego wrócimy przy Traefiku. Nie kładź obok `middlewares.ts` z tablicą: scalanie tablic element po elemencie (pułapka z rozdziału 1) daje listę, której nie zamierzałeś.

Sekrety pochodzą z pliku `.env` na serwerze, generowanego raz przez `openssl rand -base64 32`, i nigdy nie trafiają do obrazu. Każdy ma inny koszt zmiany:

Zmienna	Co psuje jej zmiana
`APP_KEYS`	podpisy ciasteczek (`strapi_up_refresh.sig`), więc odświeżanie sesji
`JWT_SECRET`	wszystkie tokeny dostępu users-permissions (rozdział 7)
`ADMIN_JWT_SECRET`	sesje panelu admina
`API_TOKEN_SALT`	wszystkie API tokeny – frontend dostaje 401 (rozdział 7)
`TRANSFER_TOKEN_SALT`	tokeny transferu danych
`ENCRYPTION_KEY`	odczyt zapisanych w bazie kluczy API tokenów w panelu

`APP_KEYS` to lista, bo Koa podpisuje pierwszym kluczem, a weryfikuje każdym: nowy klucz dopisujesz na początek, stary usuwasz, gdy wygasną podpisane nim sesje. Pozostałe sekrety rotujesz jednorazowo, wiedząc, kogo to wyloguje.

Obraz Dockera

Strapi nie publikuje oficjalnego obrazu. Przykład z dokumentacji kopiuje do obrazu końcowego cały katalog z etapu budowania. Wersja dla „Redakcji" ma trzy etapy i bierze tylko to, czego Strapi używa w czasie działania:

```
FROM node:22-alpine AS build
WORKDIR /opt/app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
# NODE_ENV only after npm ci, otherwise npm skips devDependencies needed by the build.
ENV NODE_ENV=production
# `strapi build` loads the whole config, so the fail-fast checks in config/env/production need values.
RUN PUBLIC_URL=http://build.invalid DATABASE_NAME=build DATABASE_USERNAME=build DATABASE_PASSWORD=build \
    npm run build

FROM node:22-alpine AS deps
WORKDIR /opt/app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /opt/app
COPY --from=deps --chown=node:node /opt/app/node_modules ./node_modules
COPY --from=build --chown=node:node /opt/app/dist ./dist
COPY --chown=node:node package.json package-lock.json tsconfig.json favicon.png ./
# Sources stay in the image: CLI commands (import, export, console) recompile TypeScript before they run.
COPY --chown=node:node config ./config
COPY --chown=node:node src ./src
COPY --chown=node:node types ./types
COPY --chown=node:node database ./database
COPY --chown=node:node public ./public
USER node
EXPOSE 1337
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s --retries=3 \
  CMD wget -q --spider http://127.0.0.1:1337/_health || exit 1
CMD ["node_modules/.bin/strapi", "start"]
```

Bazą jest `node:22-alpine` jak w dokumentacji (wspierane są Node 22, 24 i 26). Etap `build` instaluje wszystko i kompiluje TypeScript (2,6 s) oraz panel admina (40 s). Etap `deps` instaluje osobno same zależności produkcyjne, a obraz końcowy składa się z nich, z `dist/` i z kilku plików. `.dockerignore` wyklucza `node_modules`, `dist`, `.tmp`, `.env*`, `backups/` i zawartość `public/uploads`, więc lokalna baza ani sekrety nie trafiają do kontekstu budowania. Obraz zajmuje 1,15 GB, z czego 725 MB to produkcyjne `node_modules` samego Strapi (m.in. `typescript` i pakiety panelu).

Pierwszy build padł z błędem `Could not load js config file .../config/env/production/database.js: DATABASE_NAME is required in production`. `strapi build` tworzy pełną instancję Strapi i wczytuje całą konfigurację, więc kontrole obecności zmiennych potrzebują wartości także przy budowaniu. Stąd wartości zastępcze w `RUN` – nie zostają w obrazie. Do paczki panelu trafia tylko ścieżka z `PUBLIC_URL` (tu `/`, kod `create-build-context.js`), więc jeden obraz obsłuży staging i produkcję pod różnymi domenami. Obraz zbudowany z `build.invalid` i uruchomiony z innym adresem zwrócił na `/admin` 200; w przeglądarce panelu nie klikałem.

Wcześniejsza wersja obrazu nie miała źródeł, tylko `dist/`. `strapi start` działał, ale `strapi import` w kontenerze skończył się `error TS18003: No inputs were found in config file '/opt/app/tsconfig.json'`. Komendy CLI (`import`, `export`, `transfer`, `console`) przed startem kompilują projekt TypeScript, więc źródła `config/`, `src/` i `types/` (razem ok. 300 KB)

zostają w obrazie.

`CMD` uruchamia binarkę bezpośrednio, bez `npm run start`. Strapi jest wtedy procesem PID 1 i sam obsługuje `SIGTERM`: wywołuje `destroy()` (rozdział 1) i kończy pracę. `docker compose stop` zatrzymał kontener w 0,27 s, zamiast czekać 10 s na `SIGKILL`. Kontener działa jako `uid=1000(node)`.

Compose z Postgresem

```yaml
services:
  strapi:
    build: .
    env_file: .env
    environment:
      NODE_ENV: production
      DATABASE_HOST: db
    ports:
      - "127.0.0.1:1313:1337"     # local test only
    volumes:
      - public:/opt/app/public    # not public/uploads, see the pitfall below
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: ${DATABASE_NAME}
      POSTGRES_USER: ${DATABASE_USERNAME}
      POSTGRES_PASSWORD: ${DATABASE_PASSWORD}
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
volumes:
  db-data:
  public:
```

Ten sam `.env` zasila Strapi (`env_file`) i podstawianie `${DATABASE_NAME}` w pliku compose, więc hasło do bazy istnieje w jednym miejscu. `$$` przekazuje zmienną do powłoki kontenera, a `service_healthy` czeka na `pg_isready`, nie tylko na start procesu. Nazwany wolumen dostaje przy tworzeniu zawartość katalogu z obrazu razem z właścicielem `node`, więc Strapi zapisuje pliki bez `chown`. Pełny plik w `examples/` ma jeszcze `restart: unless-stopped`, interwał healthchecku i port 15432 dla `psql` z hosta, tylko na `127.0.0.1`.

Po `docker compose up -d` health check odpowiedział po kilku sekundach, a log pokazał `Launched in 3075 ms`, `Environment: production`, `Process PID: 1`, `Database: postgres`:

```
$ curl -i localhost:1313/_health
HTTP/1.1 204 No Content
Content-Security-Policy: connect-src 'self' https:;img-src 'self' data: blob: https://market-assets.strapi.io;...
Strict-Transport-Security: max-age=31536000; includeSubDomains
$ curl localhost:1313/api/articles
{"data":null,"error":{"status":403,"name":"ForbiddenError","message":"Forbidden","details":{}}}
```

403 na pustej bazie to przypomnienie z rozdziału 7: uprawnienia ról to dane, a nie kod. Nowa baza ma role Public i Authenticated bez żadnych uprawnień, dopóki nie nada ich `bootstrap` albo import.

Za Traefikiem

Traefik czyta reguły z etykiet kontenerów. Plik `docker-compose.traefik.yml` nakłada je na lokalną konfigurację i kasuje mapowania portów (`ports: !reset []`), bo na VPS-ie jedynym publicznym wejściem jest Traefik. Tej części nie uruchamiałem – sprawdziłem tylko scaloną konfigurację `docker compose -f docker-compose.yml -f docker-compose.traefik.yml config`.

```
services:
  strapi:
    ports: !reset []
    labels:
      - traefik.enable=true
      - traefik.docker.network=web
      - traefik.http.routers.redakcja.rule=Host(`cms.redakcja.example`)
      - traefik.http.routers.redakcja.entrypoints=websecure
      - traefik.http.routers.redakcja.tls.certresolver=letsencrypt
      - traefik.http.services.redakcja.loadbalancer.server.port=1337
    networks: [default, web]   # plus "db: ports: !reset []" and the external network "web"
```

Strapi mówi zwykłym HTTP, a TLS kończy się na Traefiku. Żeby Strapi wiedział, że klient przyszedł po HTTPS, musi ufać nagłówkom `X-Forwarded-*` – to robi `proxy.koa: true`. `maxIpsCount: 1` każe brać adres klienta tylko z ostatniego wpisu `X-Forwarded-For`, dopisanego przez twojego Traefika. Od tego zależy też limit logowania z rozdziału 7, którego klucz to login, ścieżka i IP. Bez zaufania do proxy wszyscy mają adres Traefika, więc ktoś, kto zgaduje hasło Anny, blokuje logowanie samej Annie.

> **Pułapka:** w produkcji users-permissions ustawia ciasteczko odświeżania z flagą `Secure`, a Koa odmawia wysłania go po nieszyfrowanym połączeniu. Logowanie do kontenera po zwykłym HTTP zwróciło `500 Internal Server Error`, a w logu: `Cannot send secure cookie over unencrypted connection` z `sendRefreshAuthResponse`. To samo żądanie z nagłówkiem `X-Forwarded-Proto: https` dało `200` i `Set-Cookie: strapi_up_refresh...; secure; httponly`. Za Traefikiem bez `proxy.koa: true` Koa ignoruje ten nagłówek, więc logowanie pada mimo HTTPS, a hasło zostało już sprawdzone. Jeśli testujesz obraz produkcyjny lokalnie po HTTP, dodaj nagłówek ręcznie albo licz się z tym 500.

Strapi wysyła własną politykę CSP i HSTS (`strapi::security`, nagłówki powyżej). Nie dokładaj drugiej CSP w middleware `headers` Traefika: przeglądarka egzekwuje wszystkie otrzymane polityki naraz, więc łatwo zablokować panelowi skrypty albo obrazy. Gdy media wychodzą z S3 albo CDN (rozdział 8), rozszerz `img-src` i `media-src` w konfiguracji `strapi::security` w bazowym `config/middlewares.ts`. Limit rozmiaru uploadu ustawia `formidable.maxFileSize` w `strapi::body`, bo Traefik domyślnie nie ogranicza ciała żądania.

Dane: eksport, import, kopie

`strapi export` pakuje treść, relacje, pliki z `public/uploads`, konfigurację i schematy. Z kopii roboczej na SQLite, z jednym obrazkiem w okładce artykułu:

```
$ npx strapi export --key "$EXPORT_KEY" -f backups/redakcja
entities       49   17.3 KB   (api::article.article 7, plugin::users-permissions.user 2, ...)
assets          1    411.0 B
links         105   21.3 KB
configuration  37  125.6 KB
Export archive is in backups/redakcja.tar.gz.enc        # 17 KB, 4.5 s
$ npx strapi export --no-encrypt -f backups/redakcja-plain   # redakcja-plain.tar.gz
```

Archiwum nieszyfrowane rozpakowałem: rekordy użytkowników zawierają e-maile i hashe bcrypt haseł, a konfiguracja – klucz `plugin_users-permissions_grant` z danymi providerów OAuth. Eksport to dane osobowe i sekrety, więc trzymaj go zaszyfrowany (`--key`, AES-128 według dokumentacji) albo nie trzymaj wcale. Nie są eksportowane konta panelu admina ani API tokeny.

Import do kontenera robisz osobnym, jednorazowym kontenerem z tym samym obrazem, wolumenami i siecią:

```
$ docker compose run --rm -v ./backups:/backups:ro strapi \
    node_modules/.bin/strapi import -f /backups/redakcja.tar.gz.enc --key "$EXPORT_KEY" --force
✔ entities: 49 transferred   ✔ assets: 1 transferred   ✔ links: 105 transferred
Import process has been completed successfully!                    # 8.6 s
$ curl -g 'localhost:1313/api/articles?fields[0]=title&populate[cover][fields][0]=url'
... {"id":7,"documentId":"nywq7zs8hidicead440io6t2","title":"Policies in Strapi 5",
     "cover":{"url":"/uploads/docker_cover_e94270a82c.png"}} ...   "total":3
$ curl -s localhost:1313/admin/init
{"data":{"uuid":"0e98cbca-...","hasAdmin":false,"menuLogo":null,"authLogo":null}}
```

`--force` pomija pytanie o skasowanie danych – import zastępuje całą bazę i katalog uploads. `documentId` przeżył przeniesienie, numeryczne `id` nie (8 w SQLite, 7 w Postgresie; rozdział 2). Okładkę serwuje kontener z wolumenu (`200 image/png`), Anna loguje się starym hasłem, a działający serwer pokazał nowe dane bez restartu.

`hasAdmin: false` oznacza, że pierwsza osoba, która otworzy `/admin`, założy konto Super Admina. Zaraz po imporcie utwórz administratora z CLI: `docker compose exec strapi node_modules/.bin/strapi admin:create-user` z tymi samymi opcjami co w rozdziale 7.

> **Pułapka:** dokumentacja Dockera radzi zamontować wolumen na `public/uploads`. Z takim wolumenem import padł: `The backup folder for the assets could not be created inside the public folder. Please ensure Strapi has write permissions on the public directory`. Uprawnienia były w porządku. Import przenosi `uploads/` do katalogu kopii obok (`fs.move`), a punktu montowania nie da się przenieść – ręczne `mv` w kontenerze dało `Resource busy`. Montuj wolumen na całym `public/`. Koszt: zmiany w `public/robots.txt` z nowego obrazu nie przykryją kopii w istniejącym wolumenie.

`strapi transfer --to https://cms.redakcja.example/admin --to-token <token>` przesyła to samo strumieniowo do działającej instancji (`strapi start`, token transferu z jej panelu). Tego wariantu nie uruchamiałem. Jeśli go nie używasz, wyłącz go w produkcji: `transfer.remote.enabled: false` w konfiguracji serwera.

Eksport Strapi przenosi dane między silnikami baz, ale na codzienną kopię lepszy jest `pg_dump`: obejmuje całą bazę z kontami admina i tokenami. Sprawdziłem kopię odtworzeniem do osobnej bazy w tym samym kontenerze:

```
$ docker compose exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc' > backups/redakcja-2026-10-
03.dump
$ docker compose exec -T db sh -c '... createdb restore_check; pg_restore -d restore_check --no-owner;
    psql -d restore_check -tAc "select ... from articles"; dropdb restore_check' < backups/redakcja-2026-10-03.dump
7 articles, 3 published
```

Do zrzutu bazy dołóż archiwum wolumenu `public` (albo bucket z rozdziału 8) i oba wynoś poza serwer.

Zmiany schematu i migracje

Rozdział 2 pokazał, że zmiana nazwy pola kasuje kolumnę przy starcie, a migracja w `database/migrations` ratuje dane, bo działa przed synchronizacją schematu. Ta kolejność ma drugą stronę, ważną przy wdrażaniu na pustą bazę.

> **Pułapka:** na świeżej bazie migracje też się uruchamiają i też przed synchronizacją, czyli zanim powstaną tabele. Migracja testowa wypisała `articles table exists = false`, a Strapi oznaczył ją jako wykonaną (`migrated 2026.10.03T00.00.00.probe.js`). Migracja, która zakłada istnienie tabeli, wywali więc start nowej instancji (nowy staging, odtworzenie z eksportu). Każdą migrację zaczynaj od `knex.schema.hasTable(...)` i `hasColumn(...)`, tak jak przykład z rozdziału 2.

`settings.forceMigration: false` w konfiguracji bazy wyłącza usuwanie tabel i kolumn przy synchronizacji, ale pominięte obiekty przestają być śledzone (dokumentacja). Pewniejszy jest prosty rytuał: `pg_dump` przed każdym wdrożeniem, które zmienia `schema.json`, i próba na stagingu z kopią produkcji.

Aktualizacje

Strapi wydaje nową wersję co tydzień, zwykle w środę: w 2026 roku do 30 września było to 43 wydania, w tym 24 minory (dane z npm, październik 2026). Wszystkie pakiety `@strapi/*` muszą mieć tę samą wersję, więc podbija je razem `@strapi/upgrade`. Fundament jest na najnowszej 5.56.0 (`The project is already up-to-date (minor)`), więc pokaz zrobiłem na kopii `package.json` z wersją 5.40.0:

```
$ npx @strapi/upgrade minor --dry -y
[INFO]  Upgrading from v5.40.0 to v5.56.0
[WARN]  Running the upgrade in dry mode. No files will be modified during the process.
[WARN]  Repository is not clean. Please commit or stash any changes before upgrading
[INFO]  (2/4) Applying the latest code modifications...
[INFO]  (4/4) Installing dependencies...
```

Bez `-y` narzędzie zatrzymuje się na pytaniu o brudne repozytorium. `npx @strapi/upgrade codemods ls` pokazał 13 codemodów, wszystkie dla wersji 5.0.0 i 5.1.0. W obrębie Strapi 5 aktualizacja to więc podbicie wersji i reinstalacja, a ryzyko leży w zmianach zachowania, których codemod nie wyłapie. Sekwencja: gałąź, `npx @strapi/upgrade minor`, `npm run build`, testy z rozdziału 12, nowy obraz, staging z kopią produkcyjnej bazy, wdrożenie. Nie musisz brać każdego wydania; rozsądny rytm to co kilka tygodni i od razu przy poprawkach bezpieczeństwa.

Wydajność w skrócie

Strapi tworzy indeksy na `document_id`, `locale`, `published_at` i na tabelach relacji, ale nie na polach `uid`. Frontend „Redakcji" szuka każdego artykułu po `slug`. Migracja nie nadaje się tu przez pułapkę z pustą bazą, więc indeks zakładasz w `bootstrap`, który działa po synchronizacji schematu:

```
// src/index.ts
async bootstrap({ strapi }: { strapi: Core.Strapi }) {
  await strapi.db.connection.raw('CREATE INDEX IF NOT EXISTS articles_slug_idx ON articles (slug)');
},
```

Na SQLite indeks pojawił się po pierwszym starcie; w Postgresie składnia jest ta sama, ale tam tego kodu nie uruchamiałem. Reszta to zasady z wcześniejszych rozdziałów: `populate` tylko dla potrzebnych pól (rozdział 3) i `maxLimit: 100` w `config/api.ts`. Strapi nie ma cache odpowiedzi, więc cache leży we frontendzie z rewalidacją po webhooku (rozdział 11). Pula połączeń razy liczba replik musi się zmieścić w `max_connections` Postgresa (domyślnie 100).

Checklista bezpieczeństwa

- Publicznie wystawiony jest tylko Traefik; Strapi i Postgres nie mają `ports` na VPS-ie, `.env` ma prawa `600` i nie trafia do obrazu.
- Konto admina z CLI zaraz po pierwszym starcie albo imporcie (`hasAdmin: false` to otwarte drzwi).
- `proxy.koa: true` z `maxIpsCount` równym liczbie twoich proxy, `server.url` z publicznym HTTPS.
- Uprawnienia Public nadawane kodem; dla frontendu token Custom, bo read-only czyta `/api/users` (rozdział 7).
- `allow_register` jest domyślnie `true` – wyłącz rejestrację w *Advanced settings*, jeśli czytelnicy nie zakładają kont.
- `strapi::cors` z listą originów frontendu (rozdział 11), jedna CSP – ze Strapi.
- Webhooki do sąsiednich kontenerów tylko z kodu, bo panel odrzuca w produkcji adresy prywatne (rozdział 8). Transfer wyłączony, jeśli go nie używasz.
- Eksporty szyfrowane, kopie `pg_dump` poza serwerem i regularnie odtwarzane.

Podsumowanie

- `config/env/production/` zawiera tylko różnice: Postgres z kontrolą wymaganych zmiennych, `url` i `proxy`. Tablic middleware tam nie dokładaj.
- `strapi build` wczytuje całą konfigurację, więc kontrole zmiennych potrzebują przy budowaniu wartości zastępczych.
- Obraz wieloetapowy, `npm ci --omit=dev`, użytkownik `node`, `strapi start` jako PID 1; źródła TS zostają dla komend CLI.
- Postgres za healthcheckiem, wolumen na całym `public/`, inaczej import plików padnie.
- Za Traefikiem `proxy.koa: true` jest obowiązkowe – bez niego logowanie zwraca 500.
- `strapi export --key` przenosi dane między bazami, ale zawiera hashe haseł i pomija konta admina. Codzienną kopią jest `pg_dump`, sprawdzany odtworzeniem.
- Migracje działają przed synchronizacją także na pustej bazie, indeksy zakładaj w `bootstrap`, a aktualizacje przepuszczaj przez testy i staging.

Co dalej

Masz całą drogę „Redakcji": od modelu i trasy żądania, przez polityki, Document Service i pluginy, po obraz wdrażany na własny serwer. Następny krok to staging: drugi stos compose z kopią produkcyjnej bazy odtwarzaną co noc z `pg_dump`. Na nim sprawdzisz każdą aktualizację i migrację, zanim zobaczą je czytelnicy.

Źródła

- https://docs.strapi.io/cms/installation/docker
- https://docs.strapi.io/cms/deployment
- https://docs.strapi.io/cms/deployment/guides/traefik
- https://docs.strapi.io/cms/configurations/database
- https://docs.strapi.io/cms/configurations/server

- https://docs.strapi.io/cms/configurations/environment
- https://docs.strapi.io/cms/features/data-management
- https://docs.strapi.io/cms/features/data-management/export
- https://docs.strapi.io/cms/features/data-management/import
- https://docs.strapi.io/cms/features/data-management/transfer
- https://docs.strapi.io/cms/database-migrations
- https://docs.strapi.io/cms/upgrade-tool
- https://docs.strapi.io/cms/upgrades